

Vysoká škola ekonomická v Praze

Fakulta informatiky a statistiky



Využití volně dostupných projektů pro sestavení integrované chytré domácnosti

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Studijní obor: Aplikovaná informatika

Autor: Michal Marhan

Vedoucí bakalářské práce: Ing. Ladislav Luc

Praha, prosinec 2020

Prohlášení

Prohlašuji, že jsem bakalářskou práci *Využití volně dostupných projektů pro sestavení integrované chytré domácnosti* vypracoval samostatně za použití v práci uvedených pramenů a uvedené literatury.

V Praze dne 12. prosince 2020

.....

Podpis studenta

Poděkování

Rád bych poděkoval Ing. Ladislavu Lucovi za profesionální přístup, usměrnění obsahu a přínosné rady pro proces tvorby práce, které vedly k jejímu dokončení a odevzdání.

Abstrakt

Spotřebitelský trh s produkty chytré domácnosti se rozrůstá. V minulosti bylo potřeba pro vytvoření chytré domácnosti drahé proprietární řešení od integrátorské firmy. Dnes najdeme zařízení chytré domácnosti dostupná ke koupi v každém obchodě s elektronikou. Ovšem tento trh je v začátcích. Není zde ještě sjednocený protokol, jazyk, kterým by všechna tato dostupná zařízení komunikovala. Cílem této práce je vytvoření návrhu centrální jednotky chytré domácnosti pomocí volně dostupného softwaru, která už dnes dokáže propojit zařízení napříč různými protokoly. Práce je rozdělena do části teoretické a praktické. Teoretická část začíná analýzou aktuálního stavu trhu, volbou přístupu k návrhu systému a definují se pro něj hlavní požadavky. Poté je provedena analýza protokolů, kterými komunikuje většina zařízení pro chytrou domácnost, jejich volně dostupných implementací a dalšího softwaru potřebného pro zprovoznění systému. Na základě analýzy je vytvořena praktická část, kde je přednesen vlastní návrh řešení a detailně popsána každá vybraná komponenta. V další fázi je vše implementováno a technický postup je zdokumentován. V závěrečné etapě je implementace podrobena testovacím scénářům, které ověřují připravenost implementace do produkčního prostředí. Systém úspěšně prošel testováním a je již aktuálně nasazen v jedné běžící instanci. Výsledkem je lokální centrální jednotka chytré domácnosti, která integruje zařízení napříč protokoly a umožňuje jejich ovládání ze společného grafického rozhraní.

Klíčová slova

Chytrá domácnost, Integrace, Internet věcí, Interoperabilita, Kontejnery, Otevřený software

JEL klasifikace

C88

Abstract

The consumer market with smart home products is expanding. To create a smart home in the past was to use expensive proprietary solutions from smart home integrators. Today, many smart home devices can be bought in ordinary stores with computer electronics. But this market is still in its infancy. There is no single unified protocol, which would be used across all those available devices. The goal defined by the author in this paper is to create a design of a smart home hub using open-source software, which can even today integrate devices across different protocols. Paper is split into theoretical and practice part. It starts with an overview of the current state of the market, then explaining the chosen methodology of system design. The analysis phase looks into details of protocols used in smart home devices, their available open-source implementations, and other software needed to make the smart home hub work. Based on the analysis the paper proceeds into the practical part when a proposal of the solution is devised, and every chosen component of the system is described. In the next phase, every step in the implementation process is documented. The final phase then tests the implementation against testing scenarios, which verifies readiness of the implementation and its ability to be moved into a production environment. The system successfully passes the test phase and is already used in one running instance. The result is a local smart home hub, which integrates devices across protocols and allows their control from one unified graphic interface.

Keywords

Containers, Integration, Internet of Things, Interoperability, Open source, Smart home

JEL Classification

C88

Obsah

Úvod	16
1 Technologie chytré domácnosti	17
1.1 Aktuální stav trhu	17
1.1.1 Příležitosti a výhody.....	17
1.1.2 Nevýhody.....	18
1.1.3 Shrnutí aktuálního stavu.....	22
2 Přístup k tvorbě systému	23
3 Specifikace požadavků	24
3.1 Technické požadavky z pohledu vývojáře systému.....	24
3.1.1 Hardware a operační systém serveru.....	24
3.1.2 Oddělený provoz aplikací	25
3.1.3 Protokoly chytré domácnosti pro integraci.....	25
3.1.4 Bezpečnost	25
3.1.5 Údržba a aktualizace	26
3.1.6 Stabilita provozu, zálohování	26
3.2 Funkční požadavky z pohledu uživatele	26
3.2.1 Grafické rozhraní pro ovládání.....	27
3.2.2 Hlasoví asistenti	27
3.2.3 Základní úroveň samosprávy chytré domácnosti	27
3.2.4 Historie a data o chytré domácnosti	28
3.2.5 Vzdálený přístup.....	28
4 Analýza řešení	29
4.1 Individuální a typový aplikační software	29
4.2 Integrované a komponentové řešení	30
4.3 Operační systém.....	31
4.4 Oddělený provoz aplikací.....	33
4.4.1 Docker	35
4.5 Protokoly chytré domácnosti	36
4.5.1 Transportní a aplikační vrstva.....	36
4.5.2 Zigbee	37
4.5.3 Z-Wave.....	39
4.5.4 RF 433MHz	40
4.5.5 Wi-Fi, Ethernet (TCP/IP).....	41

4.5.6 Proprietární protokoly výrobců	43
4.5.7 Hlasoví asistenti	43
4.6 Přehled softwarových platforem pro automatizaci chytré domácnosti	44
4.7 Historie a data o chytré domácnosti	47
4.8 Vzdálený přístup a správa.....	48
4.9 Bezpečnost	49
5 Návrh řešení.....	53
5.1 Diagramy návrhu	53
5.1.1 Komponentový návrh řešení.....	53
5.1.2 Návrh řešení z pohledu datových toků	55
5.2 Popis návrhu	56
5.2.1 Hardware serveru	56
5.2.2 Operační systém	56
5.2.3 Oddělený provoz aplikací	57
5.2.4 Automatizace chytré domácnosti.....	57
5.2.5 Jednotlivé protokoly chytré domácnosti.....	58
5.2.6 Zálohování	59
5.2.7 Vzdálený přístup a správa	59
5.2.8 Bezpečnost.....	60
5.2.9 Hlasoví asistenti	61
5.2.10 Aktualizace, údržba	61
5.2.11 Historie dat.....	62
6 Implementace návrhu.....	63
6.1 Operační systém	63
6.1.1 Instalace operačního systému Arch Linux.....	63
6.1.2 Nezbytná konfigurace operačního systému a základní software.....	63
6.2 Modulární software.....	66
6.2.1 Instalace a spuštění Docker platformy.....	66
6.2.2 Konfigurace Dockeru.....	67
6.2.3 Nastavení struktury složek.....	67
6.3 Automatizace chytré domácnosti	68
6.4 Jednotlivé protokoly chytré domácnosti	71
6.4.1 Integrace MQTT.....	71
6.4.2 Integrace Zigbee	74
6.4.3 Integrace Z-Wave	77

6.4.4 Integrace RF 433 MHz	79
6.4.5 Integrace Wi-Fi chytrých zařízení	84
6.4.6 Koexistence protokolů	86
6.5 Zálohování	87
6.6 Vzdálený přístup	90
6.7 Vzdálená správa	94
6.8 Bezpečnost	95
6.9 Konfigurace hlasových asistentů	95
6.9.1 Google Assistant	95
6.9.2 Apple Siri	98
6.10 Konfigurace sběru a analýzy historických dat	100
6.10.1 Konfigurace databáze InfluxDB	100
6.11 Konfigurace dashboard aplikace Grafana	102
6.12 Aktualizace a údržba	103
6.13 Přehled spuštěného softwaru a portů	104
7 Testování implementace	105
7.1 Testovací scénáře	105
7.1.1 Test síťové konfigurace	105
7.1.2 Test využití HW prostředků	105
7.1.3 Test zachycení a zpracování MQTT zprávy	107
7.1.4 Test Zigbee zařízení	108
7.1.5 Test Z-Wave zařízení	109
7.1.6 Test HomeKit zařízení	111
7.1.7 Test RF zařízení	111
7.1.8 Test uživatelské samosprávy a kustomizace	112
7.1.9 Test automatizačního scénáře	113
7.1.10 Test zabezpečení vzdáleného přístupu	114
7.1.11 Test zabezpečení vzdálené správy	115
7.1.12 Test ovládání zařízení přes Hlasové asistenty	116
7.1.13 Test databáze časových řad	117
7.1.14 Test zálohování a obnovy	118
7.2 Testovací report	120
Závěr	121
Použitá literatura	123
Přílohy	I

Příloha A: Postup instalátoru distribuce Arch Linux	I
Příloha B: Alternativní připojení Docker kontejnerů do sítě	II
Příloha C: Kontejnerový obraz pro firmware flash Zigbee rádia	III
Příloha D: Servisní skript pro aktualizaci kontejneru	VII

Seznam obrázků

Obr. 1 Prostřední aplikace SmartThings [8]	20
Obr. 2 Možnosti odděleného provozu aplikací.....	34
Obr. 3 Wi-Fi 2.4GHz a Zigbee překrývající spektrum [29]	38
Obr. 4 Webové rozhraní Node-Red pro vytváření automatizačních scénářů [44].....	44
Obr. 5 Výchozí webové rozhraní platformy Domoticz [46]	45
Obr. 6 Výchozí webové rozhraní platformy Home Assistant.....	46
Obr. 7 Webové rozhraní platformy OpenHab	46
Obr. 8 Ukázka vizualizace historických dat v softwaru Grafana	47
Obr. 9 Návrh řešení – Komponenty	54
Obr. 10 Návrh řešení z pohledu komunikace mezi jednotlivými komponentami.....	55
Obr. 11 Konfigurační průvodce Home Assistant	70
Obr. 12 Integrační průvodce MQTT v Home Assistant.....	74
Obr. 13 Zařízení Sonoff RF Bridge.....	80
Obr. 14 Deska Sonoff RF Bridge	81
Obr. 15 Konfigurační webové rozhraní OpenMQTTGateway	82
Obr. 16 Zobrazení aktuálních zpráv na MQTT serveru v nástroji MQTT Explorer.....	83
Obr. 17 Zařízení Xiaomi Desk Lamp podporující HomeKit protokol rozpoznané v Home Assistant.....	85
Obr. 18 Kontrola přepojení Wi-Fi kanálu a koncových zařízení.....	86
Obr. 19 Ukázka přesměrování portů na routeru s firewallem EdgeRouter od Ubiquity.....	90
Obr. 20 Ukázka nastavení DNS záznamů domény	91
Obr. 21 Připojení nakonfigurované integrace Home Assistant do Google Home	98
Obr. 22 Připojení Home Assistant do Apple Home	100
Obr. 23 Nastavení Apple TV jako brány pro HomeKit přes síť Internet	100
Obr. 24 Využití systémových prostředků – CPU, I/O, čím méně, tím lépe.....	106
Obr. 25 Využití operační paměti RAM, čím méně, tím lépe	106
Obr. 26 Úspěšně zachycená MQTT zpráva a správné zpracování v Home Assistant.....	107
Obr. 27 Příklady testovaných Zigbee zařízení IKEA, Aqara	108
Obr. 28 Připojení zařízení do Zigbee sítě ve webovém rozhraní Zigbee2MQTT.....	108
Obr. 29 Integrovaná Zigbee zařízení v Home Assistant – Aqara, Hue, Danfoss, IKEA....	109
Obr. 30 Připojení zařízení do Z-Wave sítě ve webovém rozhraní ZWave2MQTT	110
Obr. 31 Integrované Z-Wave zařízení v Home Assistant – Aeotec Multisensor 6	110
Obr. 32 Test připojení HomeKit zařízení do Home Assistant	111

Obr. 33 Test připojení RF tlačítka do Home Assistant	112
Obr. 34 Samospráva grafického prostředí Home Assistant.....	113
Obr. 35 Aktuální stav automatizačního scénáře v Home Assistant.....	114
Obr. 36 Test kvality implementace HTTPS.....	114
Obr. 37 Test automatické blokace uživatelů s neúspěšnými pokusy o přihlášení v Home Assistant.....	115
Obr. 38 Test hlasového rozhraní Google Assistant	116
Obr. 39 Test hlasového asistenta Apple Siri.....	117
Obr. 40 Generování grafu teplotního čidla z databáze InfluxDB v Grafana	118

Seznam výpisů programového kódu

Výpis 6.1 Zjištění názvu síťové karty	64
Výpis 6.2 Soubor s konfigurací síťové karty	64
Výpis 6.3 Soubor s nastavením DNS serverů	64
Výpis 6.4 Soubor konfigurace časových serverů	65
Výpis 6.5 Zapnutí a kontrola synchronizace systémových hodin	65
Výpis 6.6 Konfigurace zavaděče systému	65
Výpis 6.7 Skript pro vygenerování nové verze zavaděče	66
Výpis 6.8 Instalace a spuštění Dockeru	66
Výpis 6.9 Vytvoření Docker sítě typu bridge	67
Výpis 6.10 Struktura složek pro Docker kontejnery	67
Výpis 6.11 Skript pro spuštění Home Assistant kontejneru	68
Výpis 6.12 Výchozí konfigurační soubory Home Assistant	69
Výpis 6.13 Kontrola nastartovaného kontejneru Home Assistant	69
Výpis 6.14 Konfigurační soubor pro Eclipse Mosquitto aplikaci	71
Výpis 6.15 Skript pro vytvoření nového MQTT uživatele dočasným kontejnerem	72
Výpis 6.16 Skript pro spuštění MQTT kontejneru	73
Výpis 6.17 Kontrola nastartovaného kontejneru MQTT	73
Výpis 6.18 Příklad názvu zařízení, které nelze namapovat do Docker kontejneru	75
Výpis 6.19 Vytvoření stálého symbolického zástupce na USB Zigbee rádio	75
Výpis 6.20 Skript pro vytvoření dočasného kontejneru pro nahrání Zigbee firmwaru do USB rádia	75
Výpis 6.21 Konfigurační soubor pro aplikaci Zigbee2MQTT	76
Výpis 6.22 Skript pro spuštění Zigbee2MQTT kontejneru	77
Výpis 6.23 Kontrola nastartovaného kontejneru Zigbee2MQTT	77
Výpis 6.24 Vytvoření stálého symbolického zástupce na zařízení USB Z-Wave rádio	78
Výpis 6.25 Konfigurační soubor pro aplikaci Zwave2MQTT	78
Výpis 6.26 Skript pro spuštění Zwave2MQTT kontejneru	79
Výpis 6.27 Proces zálohy a flashování firmwaru do Sonoff RF Bridge	81
Výpis 6.28 Ukázková implementace senzoru stisknutí RF domovního zvonku v Home Assistant	83
Výpis 6.29 Ukázková implementace tlačítka pro RF domovní zvonek v Home Assistant ..	84
Výpis 6.30 Kontrola implementace rclone a Google Drive	88
Výpis 6.31 Definice systemd služby pro zálohování	88

Výpis 6.32 Definice systemd časovače pro spuštění zálohování.....	89
Výpis 6.33 Kontrola propagace DNS záznamů	91
Výpis 6.34 Skript pro spuštění Nginx+LetsEncrypt kontejneru	91
Výpis 6.35 Konfigurace reverzního proxy serveru v Nginx.....	92
Výpis 6.36 Nastavení Home Assistant pro proxy server.....	93
Výpis 6.37 Příprava veřejného a privátního klíče	94
Výpis 6.38 Konfigurační soubor pro SSH server	95
Výpis 6.39 Konfigurace integrace Google Assistant v Home Assistant.....	97
Výpis 6.40 Konfigurace exportu zařízení do HomeKit v Home Assistant.....	99
Výpis 6.41 Skript pro inicializaci databáze InfluxDB dočasným kontejnerem	101
Výpis 6.42 Skript pro spuštění InfluxDB kontejneru	101
Výpis 6.43 Konfigurace integrace InfluxDB v Home Assistant	101
Výpis 6.44 Skript pro spuštění kontejneru Grafana	102
Výpis 6.45 Aktivace reverzního proxy serveru pro aplikaci Grafana.....	102
Výpis 6.46 Terminálová zkratka pro aktualizaci systému	103
Výpis 7.1 Test síťové konfigurace.....	105
Výpis 7.2 Definice generického MQTT senzoru v Home Assistant	107
Výpis 7.3 Testované Z-Wave zařízení Aeotec Multisensor 6	109
Výpis 7.4 Připravený automatizační skript v Home Assistant.....	113
Výpis 7.5 Test připojení na SSH server	116
Výpis 7.6 Kontrola provedení automatické zálohy	118
Výpis 7.7 Postup instalace operačního systému Arch Linux instalátorem Anarchy InstallerI	
Výpis 7.8 Vytvoření Docker sítě umožňující přiřazení IP adres domácí sítě kontejneru.....	II
Výpis 7.9 Připojení operačního systému do sítě se systemd-networkd pomocí metody macvlan	II
Výpis 7.10 Souborová struktura pro vytvoření kontejneru na firmware flash Zigbee rádia	III
Výpis 7.11 Definice kontejnerového obrazu pomocí Dockerfile.....	IV
Výpis 7.12 Skript pro flashnutí Zigbee rádia umístěný do kontejneru	IV
Výpis 7.13 Skript pro zahájení vytvoření nového kontejnerového obrazu	V
Výpis 7.14 Ukázkový výpis aplikace úspěšného nahrání Zigbee firmwaru.....	VI
Výpis 7.15 Servisní skript pro aktualizaci kontejneru	VII

Seznam zkratek

API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JSON	Javascript Object Notation
LAN	Local Area Network
RF	Radio Frequency
SSL	Secure Sockets Layer
YAML	YAML Ain't Markup Language

Úvod

Pojem chytrá domácnost je jedním z nejnovějších výstřelů v oblasti spotřebitelské elektroniky. Představa provozu chytré domácnosti je nyní o dost blíže než v minulosti, kdy se pro takové řešení muselo oslovit integrátory a zaplatit velké množství finančních prostředků za hotové komplexní řešení. Tento trh je ale stále na začátku. Je zde několik soutěžících protokolů, které dokáží prvky chytré domácnosti propojit.

Ač se tedy zpočátku může zdát, že investice do volně dostupných prvků chytré domácnosti bude pro spotřebitele nižší, než si objednat celé řešení u integrátora, po nějaké době narazí na současný stav trhu s těmito výrobky. Bez větších technických znalostí je velice těžké dosáhnout vzájemné integrované domácnosti, kde jsou prvky schopny mezi sebou komunikovat a být ovládány z centrálního rozhraní, namísto několika oddělených aplikací.

Rozhodl jsem se pomocí volně dostupného softwaru tento problém vyřešit a sestavit architekturu domácího serveru, který bude plnit roli centrální jednotky chytré domácnosti. Tato jednotka bude schopna integrovat většinu aktuálně používaných technologií na spotřebitelském trhu s doplňky pro stavbu chytré domácnosti.

Pro dosažení tohoto cíle nejprve analyzuji, jaký je současný stav trhu se zařízeními pro chytrou domácnost, co je příčinou neexistence univerzálního standardu, jaké soupeřící technologie se na trhu aktuálně nacházejí. Dále zjistím, zda existují komerční varianty, které se snaží problém vyřešit. Poté představím metodu, kterou přistoupím k návrhu a výstavbě systému. Následně definuji požadavky běhového prostředí; jaké parametry bude softwarová a hardwarová architektura vyžadovat pro zajištění dostatečné úrovně kvality služby a základů pro další rozšiřování a vývoj z pohledu stavitele systému a z pohledu konečného uživatele. Ve fázi analýzy vysvětlím hlavní principy a odlišnosti jednotlivých technologií chytré domácnosti a z toho odvodím, jaké jsou výhody, a naopak jaká úskalí při pokusu o případnou integraci. Také analyzuji typy softwaru, které jsou vhodné pro provoz dalších částí systému.

Ve fázi návrhu řešení popíšu konkrétní použitý software a strukturu serveru, která docílí mnou předem definované požadavky na kvalitu poskytované služby. Ve fázi implementace zdokumentuji postup instalace, propojení jednotlivých komponent, jejichž výsledkem bude zprovozněný systém. Ve fázi testování definuji testovací scénáře, které ověří připravenost mé implementace pro přesun do ostrého produkčního prostředí.

Závěrem zhodnotím, zda se povedlo splnit mé definované cíle, tedy propojení všech IoT prvků pro chytrou domácnost do jednoho centrálního ovládacího panelu, jaké výhody plynou pro koncového uživatele z provozu takového řešení a vyvodím další možné oblasti vylepšení tohoto systému a další možnosti vývoje systému.

1 Technologie chytré domácnosti

Oxfordský slovník popisuje termín Smart Home následovně: „*Jedná se o dům vybavený osvětlením, vytápěním a dalšími elektronickými zařízeními, které mohou být ovládané vzdáleně chytrým mobilem nebo počítačem.*“¹ [1] (překlad autora). Při rozšíření definice do technologického pohledu chytrá domácnost představuje obytné prostředí, které obsahuje přenosovou komunikační síť, technologická zařízení jako senzory, světla, topení, běžné spotřebiče, reproduktory a další zařízení, která mohou být vzdáleně ovládána a také umožňují sběr a odeslání informací o aktuálním stavu. Centrální jednotka chytré domácnosti poté reaguje na přijatá data, aktuální stav a umožňuje určitý stupeň automatizace.

Jednotlivá technologická zařízení v chytré domácnosti spadají do kategorie IoT (Internet věcí), jak udává norma ISO. „*Infrastruktura propojených entit, lidí, systémů a informačních zdrojů, společně se službami, které zpracovávají a reagují na informace z reálného a virtuálního světa.*“ [2] Zařízení pro chytrou domácnost jsou tedy podmnožinou všech zařízení IoT.

Ovšem tento trh ještě není ustálený. Vyskytuje se zde několik technologií a protokolů, které umožňují docílit stejného výsledku – přenosu dat ze senzorů do centrální jednotky a jejich dálkovému ovládání. [3] Tak jako ve video formátech soupeří svobodný AV1 oproti komerčnímu HEVC nebo kdy na trhu současně koexistovaly formáty disků Blu-ray a HD DVD či protokoly a porty pro připojení externích periférií k počítači jako USB a FireWire, tak v dnešní době na trhu s IoT produkty chytré domácnosti soupeří celé řešení jako Zigbee, Z-Wave, Bluetooth Mesh a aplikační vrstvy postavené nad Wi-Fi jako HomeKit nebo MQTT.

1.1 Aktuální stav trhu

1.1.1 Příležitosti a výhody

Report publikovaný organizací OECD zmiňuje právě chytrou domácnost jako část IoT odvětví, se kterým přijde do styku většina spotřebitelů poprvé. [4 s. 4] OECD udává možné pozitivní přínosy chytré domácnosti a hlavní důvody, díky kterým se zákazníci rozhodují pro zakoupení takového zařízení. [4 s. 13–15] Zvýšení pohodlnosti ve svém obydlí je

¹ „A home equipped with lighting, heating, and electronic devices that can be controlled remotely by smartphone or computer.“ (původní znění)

klíčovým prvkem. Je to i hlavní prodejní bod, kterým se snaží výrobci přesvědčit zákazníka. Zařízení chytré domácnosti se prezentuje jako pomocník pro běžné úkony v domácnostech, kterých lze dosáhnout přes různé automatizace. Ovládání zařízení na dálku mimo domov je také velkou příčinou kupování těchto produktů společně s možností ovládání přes hlasové asistenty. Jako menší funkce udává OECD velkou přizpůsobitelnost domácího prostředí. Příkladem mohou být například chytré termostaty, které přizpůsobí teplotu domu dle denní doby nebo přítomnosti uživatele v domě, chytré osvětlení s možností změny teploty a barvy světla či reproduktory pro synchronizované audio přes celý obytný prostor. Dalším z benefitů je i možnost dodatečného vylepšování produktů díky aktualizaci jejich softwaru od dodavatele. OECD se shoduje s reportem Evropské komise o stavu trhu s produkty chytré domácnosti. Liší se pouze uváděnou měrou důležitosti. Report Evropské komise dal jako hlavní dva body zvýšení bezpečnosti a snížení energetické zátěže. Díky zařízením připojeným k síti Internet dokáže uživatel dostat na dálku oznámení o vzniku požáru a naskytují se možnosti jako automatický kontakt záchranných složek. Chytrá domácnost již může být i reaktivní a nejenom informovat. Například při detekci úniku vody může systém uzavřít hlavní přívod. Díky velkému množství dat, která chytré prvky domácnosti umožňují sbírat, se naskytuje velká příležitost na snížení energie. Automatická světla nebo klimatizace, které se vypnou, pokud nebude v místnosti nikdo přítomen. [3]

1.1.2 Nevýhody

Hardware a software v odlišném vlastnictví

OECD ale také poukazuje na potenciální nevýhody aktuálního stavu produktů na trhu. [4 s. 16–17] Některá zařízení chytré domácnosti jsou určena jen pro používání se softwarem výrobce onoho zařízení. Tento koncept mění tradiční pohled na vlastnictví. Když zákazník koupí takové zařízení, v podstatě kupuje dvě části. Samotné fyzické zařízení (hardware) a licenci, která ho opravňuje používat přidružený software výrobce. Většinou se jedná o formu licence pro koncového zákazníka (End User License Agreement), kterou zákazník odsouhlasí při prvním spuštění softwaru v rámci prvotní konfigurace zakoupeného zařízení. Fyzické zařízení je ve vlastnictví zákazníka, ale software zůstává ve vlastnictví výrobce. A protože je takový software nezbytný k běžné funkčnosti, jeho licence ovlivňuje samotné zařízení. To má za následek mnoho implikací. Ne vždy jsou důležité body licence komunikovány srozumitelně pro zákazníka. Součástí smlouvy může být zákaz přeprdeje zařízení, nebo jakákoliv modifikace zařízení. Zákazník pak může smlouvu porušit, aniž by si toho byl vědom. Protože je dost takových typů zařízení připojeno do sítě Internet, výrobce může případy zásahu do hardwaru zařízení zaznamenat a na dálku zrušit licenci pro používání softwaru. To může vyústit v zařízení, které nelze dále používat.

Interopabilita zařízení

Druhým velkým bodem je interopabilita zařízení. Interopabilitou v pojetí informačních technologií se myslí schopnost zařízení vzájemně spolupracovat mezi sebou. V chytré domácnosti by nemělo záležet na výrobci zařízení. V ideálním případě by zákazník koupil termostat od jednoho výrobce, radiátorovou hlavici od jiného a zařízení by se mohla mezi sebou domluvit na společné regulaci teploty v místnosti. Spolupráce mezi zařízeními lze docílit:

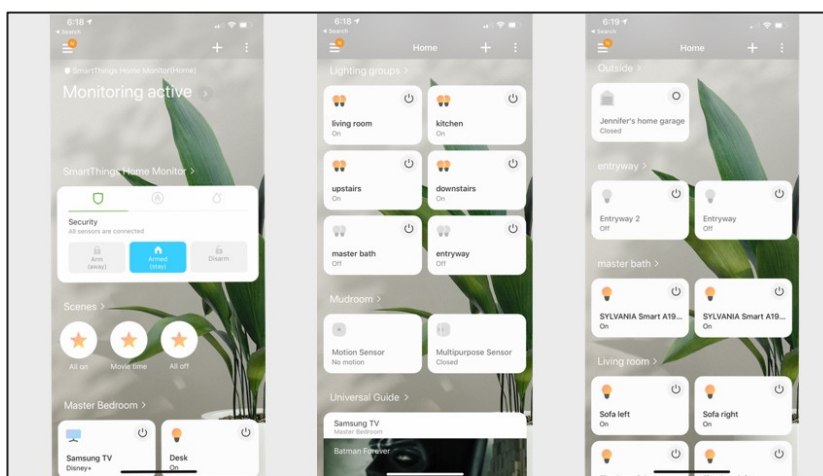
- Použitím jednoho univerzálního protokolu, který bude používat každé zařízení
- Přítomností centrální jednotky, která dokáže přemostit komunikaci mezi různými protokoly.

Aktuálně neexistuje žádný všeobecně přijímaný kompletní standard pro komunikaci mezi zařízeními chytré domácnosti. Jsou zde ale některé protokoly, které se snaží o částečnou standardizaci a interopabilitu. [4 s. 17–18]

Dle Internet Society, na které OECD report odkazuje, tento nedostatek standardizace může vést ke snížení růstu trhu se zařízeními chytré domácnosti. Různé soutěžící protokoly, proprietární protokoly a uzavřené platformy vytvářejí fragmentaci trhu, která nutí zákazníka zůstat u jednoho výrobce. Fragmentace také zvyšuje bariéru pro zákazníka, který se snaží o integraci různých typů zařízení. [5] Příčinu chybějící standardizace Internet Society označuje jako „časové riziko“. Na začínajícím trhu, který ještě není rozvinutý, je velká výhoda být první, kdo uvede své produkty a získá velký podíl. Rychlost vydávání produktů na trh v této době je rychlejší než tvorba patřičných standardů, které by výrobci mohli použít. [6 s. 48]

Problematikou existence více protokolů a jejich interoperabilitou se zabývají i další zdroje. Report vydaný Evropskou komisí z roku 2018 pozoruje jednotlivé protokoly na trhu, které se snaží o vytvoření základní úrovně standardizace a interoperability. Podotýká, že trh se stále nachází v začátku, a vzhledem k širokému záběru produktů, které chytrá domácnost pokrývá, je standardizace velkou výzvou. Jakýkoli nový protokol, který se pokouší o standardizaci, naopak přispívá k ještě větší fragmentaci. Velkým aktuálním hráčem je Zigbee Alliance. Konsorcium s více než 400 členy stojí za protokolem Zigbee. Nejznámějším výrobcem Zigbee zařízení je například IKEA nebo Philips. Zigbee Alliance se snaží o vytvoření celého protokolu. Nejvyšší aplikační vrstva dotdot může být dle Zigbee Alliance použita i v jiných protokolech než Zigbee. [3 s. 4–5] Dalším velkým hráčem, který dosahuje značného počtu zařízení od mnoha výrobců, je protokol Z-Wave od Silicon Labs. Až do minulého roku nebyl tak otevřeným protokolem jako Zigbee, a proto ho neadoptovalo takové množství výrobců. [7] Evropská komise poukazuje i na otevřené protokoly vyšší úrovně pro Wi-Fi/Ethernet zařízení, která pro komunikaci využívají již dostupný přenosový protokol TCP/IP. Například HomeKit od společnosti Apple, který mohou výrobci zařízení doplnit do svých produktů. Nebo také MQTT, protokol implementující aplikační vrstvu od organizace ISO.

Druhým způsobem pro dosažení interoperability jsou centrální jednotky chytré domácnosti, jejichž hlavní účel je poskytnout automatizaci jednotlivých zařízení, které jsou k nim připojené. Tyto jednotky umí pracovat s protokoly Zigbee a Z-Wave a podporují připojení mnoha zařízení Wi-Fi, Ethernet zařízení, která mají dostupné API. Mezi známé produkty patří Samsung SmartThings Hub a Wink Hub. Bohužel k velké části funkcionality vyžadují neustálé internetové připojení, což má své nevýhody, jež jsou popsány dále v sekci. Nabízí ale dobrý pohled, jak může vypadat taková centrální jednotka. Jak lze vidět na obrázku níže, veškerá zařízení jsou dostupná z jednoho grafického prostředí a jedné aplikace. Pro uživatele se jedná o úroveň abstrakce, kdy nemusí řešit různé způsoby ovládání napříč aplikacemi od jednotlivých výrobců zařízení.



Obr. 1 Prostřední aplikace SmartThings [8]

Do úzké podkategorie těchto centrálních jednotek se dají přiřadit i hlasoví asistenti. V oblasti spotřební elektroniky jsou aktuálně na trhu dostupné platformy velkých hráčů. Google Assistant, Amazon Alexa a Apple Siri. Microsoft se také pokusil přijít na trh se svým hlasovým asistentem Cortana, ten však neuspěl a nyní se již dále nerozvíjí. Tito umožňují integrovat zařízení od různých výrobců a ovládat je přes své grafické, a především hlasové rozhraní. Hlasový asistent funguje na principu přeložení lidské řeči do textové podoby, poté pochopení věcného významu řečeného dotazu a vykonání akce. Nevýhodou těchto aktuálních platforem je ale jednostranná integrace. Umožňují zaintegrovat zařízení k sobě, ale nedávají přístup k těmto zařízením dále pomocí svého API. Výjimkou je Apple Siri, která ovládá zařízení přes protokol HomeKit, který je otevřený. Také jsou omezeny jen na zařízení, která se k nim připojí pomocí sítě Internet nebo lokální sítě LAN (Wi-Fi, Ethernet). Pro Zigbee a Z-Wave jsou potřeba další zařízení, která integrují API těchto hlasových asistentů a poslouží jako most pro samotný Zigbee, Z-Wave protokol.

Závislost na výrobcu a síti Internet

Dalším rizikem aktuálního trhu je mnoho zařízení, která používají proprietární protokoly a vyžadují ke své optimální funkčnosti připojení do sítě Internet na servery výrobce. Zde nastává velká závislost na výrobcu i po zakoupení produktu. Výrobce zařízení může vydat

aktualizaci, úpravu, která zapříčiní omezení původní funkcionality výrobku. Nebo může dojít k situaci, kdy se výrobce rozhodne ukončit podporu pro tato zařízení. Dojde k vypnutí serverů, a základní funkcionality zařízení může být znemožněna. Případně může dojít ke změně obchodního modelu, kdy si výrobce začne účtovat za možnost používání zařízení, přestože při zakoupení zařízení tento model nebyl zákazníkovi vůbec komunikován.

Existuje několik případů, kdy se tato situace na trhu stala. Společnost Wink, výrobce centrální jednotky pro chytrou domácnost, prodával zařízení bez žádného modelu předplatného. V květnu 2020 vydal na svém blogu oznámení, že přechází na model předplatného, kdy zákazníci budou muset začít platit měsíční poplatek. Pokud by tak neučinili, zařízení by přestalo zcela fungovat, protože bylo závislé na serverech společnosti. Původní oznámení dokonce počítalo se zahájením předplatného za pouhý týden. [9] Dávalo tak uživatelům velice krátkou dobu na rozmyšlenou. Firma po velké zpětné vazbě oznámení změnila a prodloužila lhůtu před zavedením předplatného. Firma Nest vlastnila společnost Revolv, jejímž produktem byl Revolv hub, centrální jednotka chytré domácnosti, která integrovala několik protokolů. Nest byla odkoupena v roce 2014 společností Alphabet (jedná se o mateřskou společnost firmy Google). V roce 2016 Nest oznámil kompletní zrušení podpory tohoto produktu. [10] S produktem byla deklarována doživotní záruka. Po velké zpětné reakci Nest vrátil zákazníkům peníze z koupě. Nest dokládá i příklad snížení funkcionality přístroje. Jeho hlavní produkt Nest, chytrý termostat, nabízel otevřené API pod programem „Works with Nest“. V květnu 2019 oznámil ukončení tohoto API. Jako náhradu učinil svůj projekt „Works with Google Assistant“, který by nutil zákazníky ke spárování jejich Nest účtu s Google účtem. Největším problémem je, jak přímo uvedl Nest, že nové API umožňovalo pouze ovládání zařízení přes aplikace od Googlu. Integrace s vlastním softwarem přestala být možná. [11] Posledním příkladem změny obchodního modelu je internetová služba IFTT. Tato integrační služba, díky které mnoho zákazníků zvolilo koupi svého chytrého zařízení právě s její podporou, zavedla placený předplatitelský model a silně omezila funkčnost neplacené verze. [12] Zařízení závislá na podpoře výrobce nepodléhají jen změnám obchodního modelu. Může dojít k výpadku serverů, na kterých jsou zařízení závislá. Zařízení Samsung SmartThings Hub mělo několik výpadků; ty na určitou dobu znemožnily funkčnost některých integrací, ale i automatizací, protože je výrobce zpracovával na svých serverech. [13] Chybějící standardy nutí některé firmy do vývoje vlastního softwaru, který již poté nejde udržovat jednorázovou platbou při koupi produktu. A firmy, které se soustředí na sběr dat, nenabídnou otevřené API nebo použití standardního protokolu a úmyslně vytvářejí zařízení, které závisí na připojení k jejich serverům i v případě, kdy lze takovou funkčnost provádět bez připojení k síti Internet.

1.1.3 Shrnutí aktuálního stavu

Shrnutím aktuálního stavu trhu lze konstatovat, že chytrá zařízení dokážou přinést výhody pro koncové uživatele. Chybějící univerzální standardy ale dávají prostor vzniku zařízením, která lze těžko integrovat a jsou závislá na podpoře výrobce, jenž nemusí sledovat zájmy zákazníka. Mnoho zařízení je také uměle závislé na internetovém připojení i pro schopnost základní funkcionality. Jsou již ale vidět pokusy o vytvoření protokolů, které zčásti umožňují interoperabilitu mezi jednotlivými zařízeními. A integrací těchto protokolů do lokální centrální jednotky chytré domácnosti, která je vytvořena z volně dostupného softwaru, se umožní široký výběr z již existujících produktů pro uživatele. Bez závislosti na výrobci a konstantním internetovém připojení a nutnosti používat různé aplikace s odlišnými prostředím pro různá zařízení.

2 Přístup k tvorbě systému

Každý projekt se skládá z několika fází a aktivit, které je nutné postupně provést. U projektů, u nichž dochází k tvorbě softwarového systému, existuje několik možných přístupů, jak postupovat. Hovoříme o modelech životního cyklu softwaru, které popisují kroky od první myšlenky na vytvoření systému až po jeho vyřazení. [14]

Vodopádový model je nejstarší používanou metodou. Bruckner ve své knize vysvětluje význam názvu jako *„jednotlivé fáze následují po sobě a jejich grafické znázornění připomíná vodopád ... celý proces začíná fází specifikace požadavků, poté následuje fáze analýzy, návrhu, implementace, testování a zavedení.“* [15 s. 107] Vodopádový model lze využít v případech, kdy je možné na začátku přesně definovat požadavky na systém. Problém nastává, když se v průběhu vývoje mění požadavky. Změnové požadavky je pak mnohem těžší zpracovat a mohou se odsunout až do další verze, která může přijít až za velký časový úsek.

S nástupem internetu a postupným zrychlováním vývoje trhu začala nastávat potřeba mnohem vyšší obratnosti a agility podniků, tedy schopnost rychle se přizpůsobit a implementovat nové změny. Zde přichází druhý model, který se nazývá iterativní. Již z názvu vyplývá, že je uzpůsoben pro možnost menších inkrementálních změn. Jednotlivé fáze vývoje jsou stejné jako u vodopádového vývoje, jsou ale mnohem kratší a v neustálém cyklu. Na konci každé iterace vzniká plně spustitelný produkt, který rozšiřuje předchozí verzi. Je tedy mnohem jednodušší řešit změnové požadavky a přesvědčit se, jakým směrem se projekt ubíhá po každé iteraci. Iterativní model lze rozdělit do inkrementálního a evolučního. Liší se jen dle toho, zda jsou již na začátku definovány veškeré požadavky. Pokud ano, jedná se o inkrementální model. V případě, že nejsou na začátku přesně známe specifikace požadavků, jde o evoluční model.

Chytrá domácnost s produkty ze spotřebitelského trhu je neustále se vyvíjející entitou. Projekt bude připravený evolučním modelem, který umožní jednodušší přidávání nových integrací. Součástí práce bude první iterace ve fázích:

- Specifikace požadavků
- Analýza řešení
- Návrh řešení
- Implementace návrhu
- Testování

3 Specifikace požadavků

V první fázi životního cyklu modelu je důležité jasně specifikovat, jaké hlavní funkce jsou požadovány po výsledném řešení. Lze se inspirovat hlavním principem MMDIS. Multidimensional Management and Development of Information System je model vyvíjený na fakultě VŠE od 90. let. Tato metodika je primárně určena pro zvýšení znalostí specialistů informačních systémů. [16 s. 3] Nejedná se o přesný detailní popis kroku za krokem, ale o přístup uvažování při výstavbě informačních systémů. [15 s. 124]

Hlavním principem MMDIS je multidimenziálnost. Ta udává, že na problém je dobré se dívat z mnoha pohledů, které tvoří zainteresované strany. Při stavbě informačního systému se střetávají dva hlavní pohledy. Uživatelský a řešitelský. Požadavky jsou tedy rozděleny do dvou pohledů: vývojáře systému a konečného uživatele. Oba budou mít rozdílné interakce se systémem; je třeba dbát na jejich rozdílné potřeby pro tento projekt chytré domácnosti. Vývojář systému, nebo také architekt, má za cíl postavit software, který je pro něj lehce udržovatelný, jde dále rozvíjet, je stabilní a bezpečný. Uživateli softwaru pak jde o co nejlepší uživatelskou zkušenost, tedy jednoduché rozhraní, které mu umožňuje bezproblémovou interakci se systémem bez potřeby znalostí protokolů a technických detailů řešení. [15 s. 128]

3.1 Technické požadavky z pohledu vývojáře systému

Vývojář vytvářející výsledné řešení musí zajistit parametry, které umožní stabilní provoz systému, integraci všech komponent a technologií chytré domácnosti a zajistí budoucí škálovatelnost systému pro možnost adaptability na budoucí nová zařízení chytré domácnosti. Bezpečnost softwarové architektury, která se stará o kontrolu chytré domácnosti, musí být také prioritou, protože dochází k interakci se skutečnými předměty v reálném světě, které zasahují do soukromí uživatele. Požadavky zahrnuté v tomto pohledu by měly být abstraktní pro konečného uživatele takového systému.

3.1.1 Hardware a operační systém serveru

Výpočetní zdroje – Hardware, na kterém bude běžet centrální jednotka chytré domácnosti, musí být naddimenzován. Tedy průměrné vytížení veškerého softwaru nesmí být větší než 30 % dostupných výpočetních zdrojů.

Minimální operační systém – Má být minimalistický a obsahovat jen nezbytné komponenty pro provoz definovaného aplikačního softwaru.

Vzdálená správa – Umožní vzdálený přístup pro správu operačního systému, aby nemusel být architekt systému přítomen fyzicky u hardwaru počítače.

3.1.2 Oddělený provoz aplikací

Zabalení – Každá aplikace a všechny její závislosti budou vždy zabaleny do jednoho celku.

Izolace – Tento celek nebude zasahovat do souborů v operačním systému.

Doručení softwaru – Tento celek by měl být distribuovaný přímo výrobcem softwaru pro maximální kompatibilitu.

3.1.3 Protokoly chytré domácnosti pro integraci

Z průzkumu stavu trhu (viz 1.1) jsou definovány následující protokoly, které je třeba integrovat:

- MQTT
- Zigbee
- Z-Wave
- HomeKit
- 433 MHz rádiové pásmo (RF)
- Další Wi-Fi, Ethernet zařízení (TCP/IP zařízení s dostupným API):
 - Z principu těchto zařízení nelze připravit podporu pro každé, které je na trhu
 - Software centrální jednotky by měl mít jen připravenou platformu na integraci TCP/IP zařízení
- Hlasoví asistenti:
 - Google Assistant
 - Apple Siri (součástí HomeKit)

3.1.4 Bezpečnost

Pro integraci hlasových asistentů jako Google Assistant nebo Apple Siri je nutné, aby bylo API centrální jednotky publikováno do sítě Internet přes webové rozhraní. Webové rozhraní bude také výchozím grafickým rozhraním pro uživatele.

Šifrování s HTTPS – Webové rozhraní musí komunikovat přes HTTPS protokol, aby nebylo možné transparentně sledovat data mezi uživatelem a centrální jednotkou. HTTPS je klasický HTTP protokol obalený šifrovací vrstvou.

Kvalita implementace HTTPS – Pomocí benchmarku SSL Labs od bezpečnostní firmy Qualis musí získat nejméně hodnocení A+. Tento nástroj je obecně uznávanou testovací sadou pro zjištění úrovně a kvality implementace HTTPS.

Autentizace – Rozhraní centrální jednotky publikované do sítě Internet musí používat autentizaci uživatele.

Sítě chytré domácnosti – Jednotlivé protokoly v chytré domácnosti by také měly být zabezpečeny. Wi-Fi, Zigbee, Z-Wave by měly používat jejich dostupné zabezpečení, které zabrání narušení sítě.

Vzdálená správa – Vzdálený přístup ke správě serveru, na kterém bude běžet operační systém centrální jednotky, musí být také zabezpečen proti neautorizovanému přístupu z místní sítě a sítě Internet.

3.1.5 Údržba a aktualizace

Na trh s produkty chytré domácnosti neustále přicházejí nové produkty a mohou se objevit i nové protokoly. Probíhá zde neustálá inovace.

Aktualizace – Použitý software musí být sestaven tak, aby ho bylo možné neustále aktualizovat na nové verze, které budou přidávat nezbytnou funkcionalitu pro budoucí produkty, a nejenom pouze nezbytné bezpečnostní záplaty.

Stabilita – Zároveň ale tento přístup nesmí omezit stabilitu celého systému.

Vrácení verzí – Důležitá je i schopnost jednoduchého rollbacku, což znamená proces vrácení softwaru zpět do předešlé verze.

3.1.6 Stabilita provozu, zálohování

Centrální jednotka chytré domácnosti, která ovládá osvětlení, žaluzie, vytápění domu a mnoho dalších přístrojů, patří do kritické kombinace softwaru a hardwaru, který nemůže být postižen častými výpadky provozu.

Záloha – Bude implementováno zálohování na vzdálené místo. Vzdálená záloha zajistí bezpečí a dostupnost dat v případě katastrofálního selhání hardwaru.

Napájení – Server by měl být zapojený do záložního síťového prvku (UPS) v případě výpadku proudu.

3.2 Funkční požadavky z pohledu uživatele

Technické specifikace požadavků v oblasti stability, bezpečnosti a architektury by měly být transparentní pro koncového uživatele. Uživatel, který bude chytrou domácnost pouze používat a nebude se podílet na její výstavbě, má zcela jiné požadavky než architekt systému. Pro uživatele je nejdůležitější uživatelská zkušenost, tedy přívětivost a přístupnost softwaru.

Požadavky jsou definovány posledními trendy v oblasti způsobů interakce lidí s rozhraním počítače a různými formami počítačů.

3.2.1 Grafické rozhraní pro ovládání

Uživatel by měl mít přístupné grafické rozhraní z mobilního zařízení, tabletu a stolního počítače. Bude potřeba zvolit takový software, který má responzivní rozhraní na různé typy obrazovek.

Responzivní – Rozhraní, které se přizpůsobí napříč různými velikostmi obrazovek.

Konzistentní – Stejně rozhraní napříč zařízeními.

Přístupné – Dostupné z mobilního zařízení, tabletu, stolního počítače.

Abstraktní – Pro uživatele by měly být jednotlivé IoT protokoly chytré domácnosti kompletně abstraktní. V rozhraní centrální jednotky by měl vidět zařízení typu “žárovka, zásuvka, senzor, reproduktor...” a neměl by při interakci přemýšlet, na jakém IoT protokolu dané zařízení pracuje.

3.2.2 Hlasoví asistenti

Dalším typem rozhraním dnešní doby jsou chytrí hlasoví asistenti. Pro chytrou domácnost je pohodlné ovládat její prvky pomocí hlasového rozhraní bez potřeby použití zařízení s displejem v případě, že se jedná o vykonání jednoduché akce.

- Možnost ovládat chytrou domácnost přes Google Assistant
- Možnost ovládat chytrou domácnost přes Apple Siri

3.2.3 Základní úroveň samosprávy chytré domácnosti

Automatizační jednotka chytré domácnosti, která bude uživateli nabízet grafické prostředí, by měla umožňovat uživateli určitou úroveň samostatnosti ve správě chytré domácnosti. Neměla by být potřeba asistence architekta pro nastavení jednoduchých automatizací a úpravy ovládacích prvků dle potřeby uživatele.

Kastomizace – Grafické prostředí by tedy mělo obsahovat intuitivní editor umožňující kastomizace (například přidání přepínače pro novou žárovku nebo nového tlačítka pro přepnutí HDMI vstupu na televizi nebo nastavení automatického zapnutí rádia každé ráno ve všední den).

Automatizace domácnosti – Možnost tvorby jednoduchých automatizací bez znalostí programování v grafickém rozhraní.

3.2.4 Historie a data o chytré domácnosti

Jelikož automatizační software, který bude propojovat jednotlivé protokoly dohromady, musí mít přehled o aktuálním stavu každého zařízení (aby se mohl rozhodnout, zda již například v místnosti není žádný pohyb a může zhasnout světlo), mohou se tato data uchovávat. Chytrá domácnost není jen o aktuálním stavu, ale také o možnosti podívat se na zpětná data.

Samoobsluha – Základní tvorba grafů pomocí grafického rozhraní. Možnost prohlížení historie stavů jednotlivých zařízení v chytré domácnosti.

Užitečnost – Uživatel si může prohlédnout například teplotu vody v bazénu přes celé léto nebo se zaměřit na spotřebu energie a zkontrolovat, zda nemůže více snížit svoji ekologickou stopu.

3.2.5 Vzdálený přístup

Ovládání chytré domácnosti by mělo být možné i mimo její umístění. Velkou výhodou chytré domácnosti je možnost zapnout si topení dříve, než uživatel přijde do domácnosti, mít schopnost rozsvítit světla během dovolené, aby se navodil dojem fyzické přítomnosti, nebo třeba hlídání dětí pomocí přístupu ke vzdáleným kamerám.

Přístup ze sítě Internet – Ovládací panel centrální jednotky bude uživateli přístupný i mimo lokální domácí síť (LAN).

4 Analýza řešení

V této fázi se podrobně prozkoumají možné varianty řešení systému. Při vývoji softwarového řešení pro chytrou domácnost lze v určitých směrech nahlížet jako na podnikový systém. Nejprve se rozhodne o přístupu k výběru softwarových řešení, poté přijde na řadu analýza požadavků, které vyžadují hlubší zkoumání.

4.1 Individuální a typový aplikační software

O výběru softwaru pro podnikové systémy se rozhoduje mezi základními dvěma typy aplikací. Individuální aplikační software (IASW) a typový aplikační software (TASW). Při použití IASW dodavatel nebo podnik samotný vyvíjí software přesně na míru svým potřebám. Takový software může podniku poskytnout konkurenční výhodu při použití know-how, které nemá jiný podnik. IASW je zároveň vysoce optimalizovaný pro procesy daného podniku. Vývoj takového řešení je ale pro podnik výrazně dražší, neboť musí nést celé náklady naprogramování kompletního softwaru, a nikoliv pouze provedení konfigurace či drobné úpravy. IASW se tedy hodí více do konkurenčního prostředí a pro specifické účely, které nejsou běžné napříč většinou podniků. TASW je úplně odlišný přístup k vývoji a provozu aplikací. Výrobce TASW prodává produkt širokému množství zákazníků. Nejedná se tedy o software tvořený na míru podniku. Je to hotový produkt, který podnik zakoupí a implementuje. Nevýhodou řešení může být, že se podnik musí přizpůsobit softwaru. Ale většinou TASW obsahuje nejlepší know-how z dané oblasti a pro podnik se stává naopak výhodou – využije znalosti největších odborníků. TASW je výhodný pro procesy, které mají obecný charakter napříč podniky. Jako příklady softwaru tohoto typu můžeme uvést webový prohlížeč, kancelářské aplikace, emailové klienty, ale i pokročilý software pro řízení zdrojů, účetnictví a další. Běžná vlastnost TASW je možnost kastomizace. Kastomizace umožňuje hlubší přizpůsobení softwaru pro potřeby zákazníka. Lze zde uvést změnu zdrojů dat, ze kterých bude software čerpat, nebo rozšíření funkcionality pomocí systému doplňků, přizpůsobení uživatelského rozhraní (úprava rozvržení obrazovek). [15 s. 81]

Bruckner et al. zařazuje jako specifickou podkategorii TASW také open-source software. Open-source je termín z anglického jazyka pro „otevřený zdrojový kód“. Celý kód softwaru je dostupný k nahlédnutí. Většinou je s open-source softwarem také spjatá nějaká podoba svobodné licence. Ta umožňuje volnou úpravu kódu i využití pro komerční účely (není to ale povinnost). Uvolňování částí softwaru, který byl původně v intelektuálním vlastnictví společnosti do open-source se svobodnou licencí, se stává poslední dobou velice oblíbené u softwarových gigantů jako Google nebo Microsoft. [17, 18] Tento přístup umožňuje spolupráci komunity z celého světa. Firmy tímto dostávají některé příspěvky v podobě úprav a vylepšení zadarmo. Zároveň tento přístup může firmám pomoci při prosazení svého

softwaru jako nového standardu na trhu. Open-source software se nachází mezi typickým IASW a TASW. Umožňuje firmě vzít velký základ a další úpravy doprogramovat. Open-source je v mnoha případech, kromě již zmíněných velkých gigantů, dílem komunity dobrovolníků. Je tedy potřebné zvážit, zda ho firma nasadí pro kritické aplikace, jelikož zde nemusí být přítomná garance kvality a dostupnosti jako od prodejce TASW. Pokud se jedná o open-source projekt, který je tvořený velkou částí komunity, nehrozí velké riziko, že se projekt přestane udržovat a dále vyvíjet. Díky otevřenému zdrojovému kódu se mohou měnit lidé, kteří ho udržují. Zatímco pokud TASW přestane výrobce udržovat, většinou jeho kód neuvolní, a zákazník je nucen po nějaké době přejít na jiný software. Pro použití mimo účely podniku, nekomerční užití, více standardizované účely představuje open-source velice zajímavou alternativu. [15 s. 82]

Funkce, které jsou požadovány po softwaru pro provoz chytré domácnosti, se shodují napříč různými uživateli. Stejnou centrální jednotku chytré domácnosti, která obsahuje platformu pro automatizaci, grafické rozhraní, vzdálený přístup atp., lze použít napříč uživateli, pokud je u ní možná kastomizace. Kastomizací je myšlena možnost přizpůsobení pro individuální potřeby každé chytré domácnosti. Můžeme uvést například pohled pro požadavky automatizace. Jádrem automatizace bude stejné pro všechny uživatele. A každý uživatel si již poté sám vytvoří jednotlivá pravidla, jako „Rozsvítit žárovku Kuchyň-Strop při zaznamenání pohybu čidla Kuchyň“. Tyto funkce není třeba vyvíjet individuálně. V požadavcích jednotlivých uživatelů se ale kombinace zařízení, které je třeba integrovat, mění. Každá chytrá domácnost bude mít jiná zařízení pro integraci. V tomto projektu se ale jedná o sériově vyráběné zboží. Při napsání softwaru pro integraci zařízení do centrální jednotky jedním uživatelem mohou z této komponenty posléze těžit další, kteří zakoupili stejný výrobek. Z těchto důvodů by bylo vysoce neefektivní jít cestou IASW, jelikož potřeby chytré domácnosti se výrazně neliší mezi uživateli, a zvolit TASW v podobě open-source. V open-source variantě se zároveň nejedná o konkurenční prostředí, a každé nově nalezené vylepšení přispěje celé komunitě. Spolupráce dokáže zajistit podporu většího počtu zařízení.

4.2 Integrované a komponentové řešení

Při rozhodovacím procesu, který zahrnuje výběr softwaru, podnik často řeší, zda zvolit integrované řešení, nebo komponenty, které mezi sebou propojí. Bruckner ve své knize hovoří o integrovaném řešení a integrovaných komponentách. U obou přístupů dochází k integraci mezi různými komponentami. U integrovaného řešení jsou veškeré komponenty dodávány v jednom balíku přímo od výrobce softwaru. Integrace mezi jednotlivými částmi balíku je již vyřešena na straně dodavatele. Provozovatel softwaru nemusí vynakládat prostředky na propojení komponent. Riziko tohoto přístupu obnáší stav, kdy výrobce softwaru nezahrne komponenty, které poskytují nejlepší funkcionalitu aktuálně dostupnou jinde. Podnik je pak odkázán pouze na to, co poskytne jeho dodavatel softwaru, jelikož integrované řešení je omezené co do možnosti integrace s jiným softwarem. Při zvolení

komponentového řešení podnik vybere jednotlivé softwarové aplikace a poté je sám integruje mezi sebou. Velkou výhodou tohoto řešení je možnost vybrat přesně ty varianty softwaru, jehož funkcionalitu považuje podnik za tu nejlepší. Zároveň nedochází k závislosti na jednom výrobci softwaru. Mohou se přidávat a měnit další komponenty. Nevýhodou takového řešení je ztráta jisté garance, že jednotlivé komponenty mezi sebou budou vzájemně komunikovat. To musí vyhodnotit sám podnik a zároveň se postarat o samotné propojení všech komponent. [15 s. 87]

V případě využití open-source řešení je komponentové řešení lepší volbou. Většina open-source projektů se soustředí na řešení jednoho problému. Komplexní systém se poté vytvoří složením a integrací několika open-source projektů.

4.3 Operační systém

Jako kandidát operačního systému open-source typu se jeví některá distribuce z GNU/Linux. Důvodů pro výběr tohoto typu operačního systému je několik.

Jedná se o open-source a z velké části svobodný software. Nejenom, že je zcela zdarma pro jakékoliv použití [19], ale je to výsledek spolupráce velkého množství vývojářů po celém světě včetně firem jako Google, IBM, Intel, Red Hat a dalších. [20] Tyto firmy do vývoje GNU/Linux výrazně přispívají, protože ho samy používají ve svých datacentrech, čímž se dostáváme k druhému důvodu. GNU/Linux je velice modulární operační systém. Běžný uživatel se většinou setká pouze s pojmem Linux a považuje tento termín za operační systém. Linux je ovšem pouze jádro operačního systému (anglicky kernel) a veškeré další komponenty potřebné k vytvoření svobodného operačního systému jsou z projektu GNU.

Výsledkem, se kterým se jako uživatelé setkáváme, jsou distribuce GNU/Linux. Jsou to kombinace balíčků softwaru, které mají za cíl splnit cílovou funkcionalitu systému. Tedy například distribuce jako Ubuntu Desktop, Fedora Workstation, Elementary OS jsou mířené pro použití na stolních počítačích k běžné práci jako Microsoft Windows. Obsahují grafické uživatelské rozhraní (GUI), tedy ikony, okna, ovládání myši. Dále například internetový prohlížeč, přehrávač videí, prohlížeč fotek, základní kancelářské programy. Zatímco CentOS, Fedora server, SUSE Enterprise jsou určeny více na použití v serverech. Umožňují instalaci bez grafického rozhraní a dalšího softwaru uvedeného v předchozí variantě, které na serveru není třeba. Jejich softwarová výbava naopak zahrnuje například schopnost provozovat webový, emailový či souborový server.

Některé distribuce se označují jako univerzální (general-purpose), jelikož umožňují již v průběhu instalace určit, jaká kombinace balíčků softwaru půjde nainstalovat. Do této kategorie spadá například Debian. Takové distribuce jsou velice vhodné pro minimalizování hardwarových nároků a zamezení instalace nepotřebného softwaru, který by jen vytvářel

nepotřebný balast v systému a později vytvářel více práce na instalaci aktualizací a údržbu systému. Je ovšem potřeba vyšší znalost GNU/Linux prostředí.

Před výběrem distribuce je třeba ještě zmínit dva různé modely vydávání nových verzí softwaru (aktualizací). GNU/Linux distribuce spadají pod 2 modely aktualizací. Ve většině distribucí je běžný model bodové verze softwaru (point-release). Jednou za delší časový interval se vydá kompletně nová verze systému. Po vydání je další softwarová podpora této verze jen ve formě bezpečnostních aktualizací a zpravidla na ni nevycházejí žádné nové verze programů, které by měnily funkčnost nebo přidávaly nové funkce. Výhoda tohoto modelu je běhové prostředí, které je beze změn. Systém stačí nakonfigurovat jen jednou a není nutné se obávat, že aktualizace bude vyžadovat další konfiguraci. Nevýhodou tohoto přístupu je, že software je zamrzlý v čase vydání systému. Není flexibilní pro podporu nového hardwaru, nepřináší nové softwarové funkce. Pokud chce uživatel získat tyto nové výhody, je potřeba počkat na další velké vydání systému. Tento typ distribucí také běžně používá kastomizované verze softwaru, které zpříjemní uživateli první spuštění nebo usnadní integraci s dalším předinstalovaným softwarem. Nejedná se tedy o čistou verzi přímo od vývojářů daného softwaru. Při aktualizaci na nové velké vydání mohou ale tyto kastomizace způsobit řadu problémů v nekompatibilitě. Následná aktualizace na nové velké vydání tedy většinou vyžaduje velké množství práce a výsledkem nemusí být stabilní běhové prostředí. Běžně se tedy připraví nová čistá instalace a systém se znovu nakonfiguruje od začátku. Analogií k více používaným desktopovým operačním systémům může být Microsoft Windows do verze 10. Jednou za delší časový úsek Microsoft vydal novou verzi operačního systému (Windows 2000, XP, Vista, 7, 8). Během života dané verze Windows nabízel hlavně bezpečnostní aktualizace a nové funkce připravoval do další verze systému. Aktualizace na novou verzi byla komplikovaným procesem. Vyskytovaly se problémy s nekompatibilitou aplikací nebo ovladačů.

Kontinuální vydávání softwaru (rolling-release) představuje velice odlišný pohled na model vývoje a distribuce softwaru. GNU/Linux distribuce s tímto modelem nevydávají žádné velké verze, ale jsou neustále kontinuálně aktualizovány. Uživatel nedostává pouze bezpečnostní záplaty, ale také nové verze instalovaného softwaru. Oproti „point-release“ modelu tento software nebývá nijak dále upravován, než je nezbytně nutné pro zasazení do systému, a uživatel tedy dostává stejný kód aplikace, který vydal jeho tvůrce. Těmto verzím softwaru se říká upstream (verze „přímo od zdroje“). Velký benefit tohoto modelu je, že po prvotní instalaci stačí uživateli provádět malé aktualizace a nikdy nebude muset řešit migraci na nové velké vydání. U tohoto modelu softwaru se může zdát, že sníží stabilitu systému. Je zde domněnka, že nová verze softwaru může rozbít konfiguraci předchozí. Ale protože nainstalovaný software není nijak dále kastomizován správcem distribuce a používá se upstream verze od vývojáře, má v sobě obsažený automatický přechod na novou verzi bez potřeby manuálního zásahu. Stabilitu ale může ohrozit stav, kdy by v systému bylo nainstalované velké množství softwaru, které na sobě závisí. Mohlo by dojít k případu, kdy nová verze podpůrného softwaru způsobí nefunkčnost aplikace, která je na ní závislá. Pokud se ale použije minimální instalace, nezbytná pro provoz operačního systému, a veškerý další

aplikační software se nainstaluje do vlastního sandboxu (viz 4.4), získáme stabilní operační systém, který bude neustále udržovaný na nejnovější verzi softwaru a nikdy nebude potřeba provádět velkou migraci na nové velké vydání operačního systému. Někdo by mohl k tomuto modelu přiřadit také Microsoft Windows 10. Není to ale zcela přesné srovnání. Jednou za půl roku vychází nová velká verze Windows, na pozadí dochází stále k přeinstalaci systému, a i když je marketingové označení jen „Windows 10“ jako poslední verze Windows, jedná se o rozdílné verze. Kadence aktualizací je jen rychlejší a proces aktualizace na novou verzi je více stabilní.

4.4 Oddělený provoz aplikací

Jednotlivé komponenty, které budou tvořit celou centrální jednotku, nebudou instalovány monoliticky do jednoho stromu souborového systému operačního systému. Cílem je co nejvíce použít tzv. sandboxing, kdy všechny nezákladní software (co není potřeba k běhu operačního systému a základních služeb) bude mít svůj vlastní souborový strom a veškeré závislosti zabaleny do jednoho balíčku. Lze si představit, že každá aplikace bude mít svoji složku a na žádné jiné místo nebude mít přístup.

Tímto se dá vyhnout konfliktům se závislostmi softwaru. Závislosti označují podpůrné knihovny potřebné k provozu určité aplikace. Mějme dvě aplikace, které ke svému spuštění potřebují knihovnu X. Jedna z těchto aplikací je kompatibilní pouze s určitou starší verzí knihovny, zatímco druhá aplikace počítá s novější verzí. Při instalaci obou aplikací do jednoho operačního systému může dojít ke konfliktům verzí, kdy aplikace nebudou fungovat zcela správně.

Sandboxový přístup také umožňuje jednoduché aktualizace a tzv. rollback – vrácení k předchozí verze softwaru v případě potřeby. Díky tomu, že všechny soubory, které jsou přidruženy této aplikaci, jsou zabaleny do jednoho celku, je také jednoduchá odinstalace. Zmizí pak problém tzv. pozůstalých balíčků a souborů. To jsou data, která patřila dané aplikaci, ale neodebrala se.

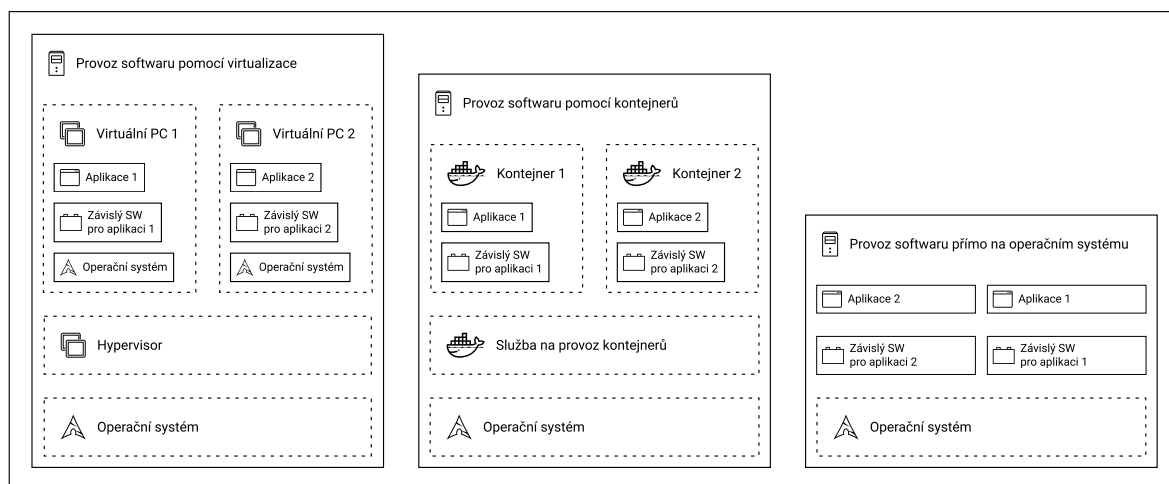
Existují dvě hlavní možnosti, jak docílit sandboxového provozu aplikace. Prvním, déle používaným v podnikové sféře, je virtualizace. Virtualizace je způsob, kdy na počítači s operačním systémem spustíme oddělené prostředí, které se chová jako další počítač. Toto prostředí sdílí výpočetní zdroje s operačním systémem, na kterém běží, ale je od něj kompletně izolováno. Do virtuálního stroje je nutné nainstalovat vlastní operační systém. Funkcionalitu virtuálních strojů zprostředkovává hypervizor, který se stará o rozdělování výpočetních zdrojů, ze kterých vytvoří virtuální hardware. Operační systém uvnitř virtuálního stroje si pak myslí, že běží na hardwaru, který mu poskytuje hypervizor. Virtualizace tedy umožňuje dokonalou izolaci, má ale větší nárok na prostředky systému. Dnešní hypervizory jako Microsoft Hyper-V, VMware ESXi, Xen, Linux KVM mají spolu s evolucí procesorů, které jsou připravené na virtualizaci (Intel VT-x, AMD-V), malou

výpočetní režii, penalizace výkonu činí mezi 1–5 %. Větší penalizaci má operační paměť, síťový výkon a diskový výkon. [21] Za to může většinou právě další operační systém, který běží uvnitř virtuálního stroje. Další nevýhodou je nutná správa operačního systému v každém virtuálním stroji.

Novější variantou, která zažila velký úspěch mezi provozovateli softwarových služeb a nyní se jí daří vstupovat i do konzervativnějších podniků, je tzv. kontejnerizace. Dle Gartneru téměř 30 % podniků již dnes provozuje alespoň jednu aplikaci formou kontejnerizace a do roku 2022 se tento podíl více než zdvojnásobí. [22] Kontejnerizace vznikla za účelem vytvoření stejného softwarového běhového prostředí mezi vývojem a produkcí. Díky distribuci zabalené aplikace s veškerými závislostmi odpadly případy, kdy měl vývojář na svém počítači jinou kombinaci verzí podpůrných knihoven a po nahrání aplikace na produkci se objevily chyby z důvodu odlišných verzí těchto závislostí.

Kontejner tedy není druhem virtualizace. Vše běží na jednom operačním systému a jednom jádře (kernelu). Penalizace výkonu zde není, pouze se lehce zvyšují nároky místa na disku a paměť, což v dnešní době nepředstavuje žádný problém. Typicky se do operačního systému nainstaluje správce kontejnerů, který umožňuje rychlé vypínání a konfiguraci. U kontejnerizace se tedy pouze oddělují soubory aplikace od systému a v některých případech dochází i k oddělení síťových prostředků, aby se mohly lépe konfigurovat porty, na kterých aplikace může běžet.

Kontejnery nabízejí přínos virtualizace s menšími nároky na výpočetní zdroje, konfiguraci a údržbu. Jsou vhodné pro provoz aplikací, které nepotřebují jiný operační systém, než na kterém běží kontejnerová platforma.



Obr. 2 Možnosti odděleného provozu aplikací

Na přiloženém obrázku můžeme vidět graficky znázorněný rozdíl metod odděleného provozu aplikace.

4.4.1 Docker

Docker je jedna z kontejnerových platform. Původně se jednalo o monolitický software. Docker vyřešil spoustu problémů pro vývojáře softwaru tím, že vytvořil nástroje, které podporovaly veškeré procesy potřebné pro stavbu, provoz a distribuci kontejnerů. [23] V roce 2015 Docker vytvořil skupinu *Open Container Initiative (OCI)* a svůj software rozdělil do několika open-source modulů, které mohou využívat i jiné kontejnerové platformy. [24] Jedná se tedy již o standardizovaný software.

Kontejnery, které běží na systému, se vytvářejí z takzvaných obrazů. Vývojář aplikaci zabalí společně se všemi závislostmi do jednoho obrazu, který lze distribuovat. Z obrazu se poté vytvoří běhové prostředí zvané kontejner. Jedná se o instanci obrazu. Kontejnerové prostředí může být různě parametrizováno.

Společnost Docker také provozuje službu DockerHub. Jedná se o repozitář obrazů. Vývojáři mohou svůj software nahrát přímo na DockerHub, který je poté k dispozici uživatelům Dockeru. Přes jediný příkaz *docker pull* je pak možné stáhnout jakýkoliv kontejnerový obraz.

Síťové možnosti Dockeru

Docker odděluje jednotlivé kontejnery od vnější sítě. Pro připojení do domácí sítě LAN a sítě Internet lze použít několik síťových ovladačů, které budou vybírat dle požadavku jednotlivých aplikací běžících v kontejneru.

Bridge – Přemostění (*překlad autora*) je výchozí metoda pro provoz Docker kontejnerů. Při použití této metody Docker vytvoří virtuální místní síť a síťový prepínač (anglicky switch), do kterého připojí jednotlivé kontejnery. Kontejnery tímto mohou komunikovat přímo mezi sebou pomocí svých přidělených IP adres. Pro přístup do vnější sítě (domácí LAN síť nebo síť Internet) Docker použije nastavené primární síťové připojení operačního systému jako router. Tedy veškerou komunikaci mimo kontejnery mezi sebou pošle ven z počítače, na kterém běží. Protože je na cestě router, uživatel musí definovat, jaké porty může kontejner využívat. Tím se otevřou do vnější sítě a budou sdílet IP adresu počítače, na kterém běží Docker. Při spuštění Dockeru se vytvoří jedna výchozí síť typu *bridge*. Pokud kontejner není nastaven jinak, připojí se k této síti. Pokud si uživatel definuje svoji síť typu *bridge*, může poté využít funkcionalitu překladu názvu kontejneru na IP adresu, která je ve výchozí síti vypnuta. Při integraci modulárních komponent domácnosti mezi sebou poté budu moci pouze odkázat na jiný kontejner názvem, například „mqttserver“ místo IP adresy „172.16.0.5“, která se může změnit. V návrhu řešení bude vytvořena síť typu *bridge* pro kontejnery, které budou využívat tohoto typu připojení. Definuje se typ sítě, maska a rozsah IP adres. Výhodou této metody je přesné omezení kontejneru na jednotlivé porty, nemůže dojít ke kolizi, kdy dvě aplikace budou chtít pracovat na stejném portu. Nevýhodou je izolace z vnější sítě. Pokud by aplikace poskytovala funkčnost jako automatické objevování dostupných zařízení na síti, nefungovala by v tomto způsobu připojení.

Macvlan – Tato metoda připojení Docker kontejnerů k síti nebude využita v návrhu řešení. Její popis a teoretické nastavení byly odloženy do přílohy (viz Příloha B:).

Host – Tato metoda odstraní síťovou izolaci mezi kontejnerem a operačním systémem, kde běží Docker. Chová se stejně, jako by byla nainstalována přímo do operačního systému. Protože je připojena přímo do vnější sítě, může otevřít libovolné porty bez definice uživatelem.

None – Žádná síť. Toto nastavení nepřipojí kontejner k síti. Dojde ke kompletní síťové izolaci. Tuto metodu nebudu využívat u žádného kontejneru, protože v tomto návrhu je potřeba komunikace mezi jednotlivými komponentami.

Persistentní data a Docker kontejnery

Z předchozí analýzy bylo zjištěno, že kontejner je oddělený od zbytku souborového systému operačního systému, na kterém běží, veškerá data aplikace ukládá do kontejneru. Kontejner není určen pro ukládání persistentních dat. Persistentní data jsou ta, která mají být vždy zachována (uživatelské nastavení a konfigurace). Při odstranění kontejneru zmizí i veškerá data, která aplikace do kontejneru uchovala. Odstraňování kontejneru je běžný proces v životním cyklu kontejnerového provozu aplikace. Pokud vyjde nová verze aplikace, vývojář připraví nový kontejnerový obraz. Uživatel poté nový obraz stáhne a vytvoří z něho nový kontejner.

Docker umožňuje při tvorbě kontejneru definovat, zda do něj nechceme připojit nějaké složky z operačního systému. Uživatel může přesně definovat, jaká složka z operačního systému má být připojena do kontejneru. Lze zvolit libovolné místo v souborovém stromu kontejneru. Vývojář kontejnerového obrazu přiloží do jeho dokumentace informaci, kam aplikace ukládá svá data, která by měla být persistentní. Aplikace v kontejneru neví, že poté data ukládá ve skutečnosti na jiné místo, vše je pro ni transparentní.

4.5 Protokoly chytré domácnosti

4.5.1 Transportní a aplikační vrstva

Před hlubší analýzou jednotlivých protokolů, které se používají pro koncové produkty v chytré domácnosti, je dobré rozřazení do kategorií podle kritéria, jaké nejvyšší vrstvy referenčního modelu ISO/OSI implementují. V podstatě se jedná o to, kolik práce zbývá na výrobce daného IoT zařízení a o jak vysoce standardizovanou technologii se jedná. To výrazně ovlivňuje kompatibilitu a složitost integrace těchto zařízení do centrální jednotky chytré domácnosti.

Open Systems Interconnection model (OSI) je konceptuální referenční model, jak řešit komunikaci v počítačových sítích. Počítačovou síť rozděluje na několik abstraktních vrstev

– od nejnižší fyzické, která definuje přímo rádio nebo port, který fyzicky pomocí elektronických signálů posílá binární kód, přes transportní vrstvu, která nám definuje, jak budeme přenášet data mezi jednotlivými uzly v síti, adresační vrstvu, která umožňuje tyto jednotlivé uzly identifikovat (např. IP adresy), až po vrstvu aplikační, která zahrnuje konkrétní software, který na dané síti běží. V běžné praxi je příkladem fyzické vrstvy Ethernet port nebo Wi-Fi rádio. Transportní vrstva a adresační vrstva je TCP/IP. Aplikační vrstvou může být například univerzální protokol HTTP, přes který stahujeme webové stránky nebo komunikujeme s různými službami. Nebo specifická vrstva určená pro IoT zařízení, která implementuje přesné podoby zpráv a příkazů pro různé typy zařízení v chytré domácnosti. Například aplikační vrstva dotdot (viz 4.5.2). [25]

Pokud IoT protokol chytré domácnosti implementuje pouze transportní vrstvu nebo univerzální aplikační vrstvu, jen definuje, jak si jednotlivá zařízení v síti mohou vyměňovat data. Ovšem nedefinuje strukturu dat, která se posílají. Pro každé zařízení by tedy byla potřeba individuální integrace podle individuální aplikační vrstvy, kterou používá. Zatímco technologie, která implementuje i aplikační vrstvu, by vyžadovala v ideálním případě napsat jen jednu integraci do centrální jednotky a poté jakékoliv zařízení, které by implementovalo tuto technologii, lze integrovat bez dalšího vývoje. Ukázkovým praktickým příkladem mohou být chytré žárovky. Pokud by všechny používaly stejnou aplikační vrstvu, příkaz na jejich rozsvícení je pro každou identický.

4.5.2 Zigbee

Zigbee je technické řešení pro IoT zařízení, které zahrnuje jak transportní protokol, tak aplikační vrstvu dotdot. Jedná se o otevřený standard. Zastřešuje ho organizace Zigbee Alliance, do které může vstoupit každý výrobce IoT zařízení s technologií Zigbee. Nevýhoda tohoto otevřeného přístupu je, že ne každý výrobce zařízení přesně implementoval aplikační vrstvu podle standardu. Certifikace nebyla povinná. Proto Zigbee Alliance při uvedení protokolu Zigbee verze 3.0 spustila rozsáhlejší certifikační program, který má za cíl vyřešit tento problém s fragmentací. Každé zařízení, které touto certifikací projde, bude mít zaručeno, že je plně kompatibilní s referenční aplikační vrstvou [26] a může na krabici produktu umístit logo Zigbee.

Zigbee je síť založená na smíšené topologii. V síti se vyskytují konečná zařízení, routery (typicky jakékoliv zařízení, které je připojeno k elektrické síti a nemá omezený zdroj energie) a jeden koordinátor, který řídí síť. To znamená, že konečný uzel (zařízení chytré domácnosti) nemusí navázat přímý kontakt s koordinátorem, ale zprávu pošle přes některý z routerů, které předají zprávu dál až ke koordinátorovi. Výhodou tohoto typu sítě je možnost velice rozšířit dosah signálu, a proto není problém pokrýt i velkou domácnost.

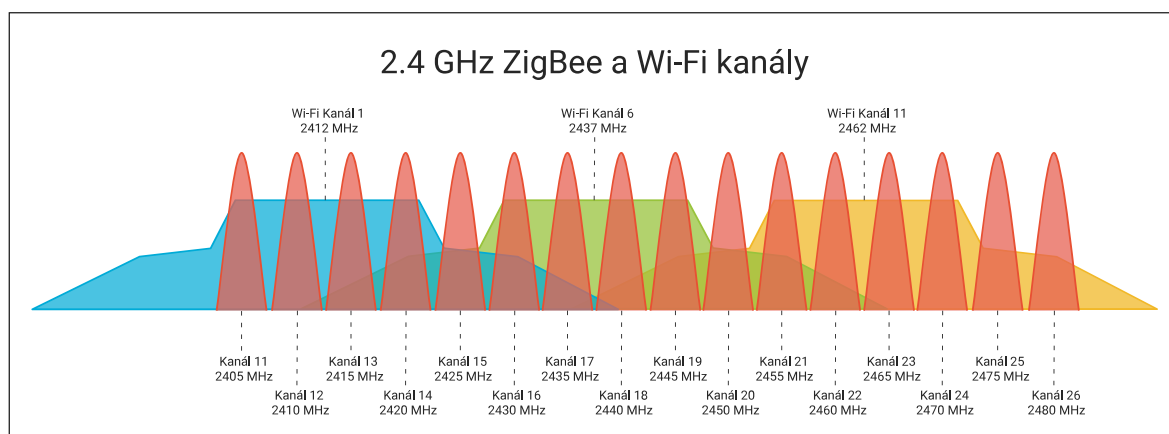
Pro implementaci do centrální jednotky bude potřeba integrace, která implementuje nejen referenční aplikační vrstvu, ale také umožňuje dodatečnou implementaci odchylek pro zařízení, která nejsou plně kompatibilní s referenční vrstvou. Dále bude potřeba najít

zařízení v podobě rádia, které bude sloužit v centrální jednotce jako koordinátor Zigbee sítě, přes který bude možné rozesílat a přijímat data v Zigbee síti.

Známí výrobci na našem trhu se Zigbee zařízeními jsou například Philips se svojí řadou Philips Hue a IKEA s řadou chytré domácnosti IKEA TRÅDFRI. Cenově dostupné jsou také různé senzory od Xiaomi a její sesterské společnosti Aqara [27].

Již v této fázi analýzy je zřejmá výhoda integrace všech Zigbee zařízení pod jednu síť. Každý výše zmíněný výrobce kromě koncových zařízení prodává i svůj koordinátor. Pokud bude chytrá domácnost implementována touto cestou, došlo by k vytvoření několika oddělených Zigbee sítí, které nekomunikují mezi sebou, a nezískaly by se tím výhody topologie smíšené sítě. Tedy větší dosah signálu a robustnost celkové sítě. Zároveň by mohlo dojít k vzájemnému rušení, pokud by jednotlivé sítě byly nakonfigurovány na stejném rádiovém rozsahu.

Zigbee operuje na frekvenci 2.4GHz, což je mezinárodně volně dostupná frekvence spadající do tzv. ISM (industriální, vědecké a zdravotní aplikace) spektra definovaného Mezinárodní telekomunikační unií (ITU) [28]. Díky povaze této frekvence – je nezatížená licencemi a volně dostupná po celém světě, ji používá spousta technologií. V domácnostech a spotřební elektronice je tato frekvence používána hlavně pro Wi-Fi, která může narušit stabilní provoz Zigbee sítě. Proto je nutné dopředu naplánovat, na jakých kanálech bude operovat Zigbee a na jakých Wi-Fi síť. Dnešní verze Wi-Fi protokolu (802.11/ac/ax, také označovaná jako Wi-Fi 5 a 6), již umožňuje i provoz na 5GHz frekvenci. Proto je nejlepší veškerá zařízení, která zvládají frekvenci přes 5GHz přepojit, a 2.4GHz uvolnit pro Zigbee, čímž dojde k minimalizaci případného rušení. Na obrázku lze vidět překrývající se Wi-Fi (modré, zelené, žluté) a Zigbee (červené) kanály (viz Obr. 3).



Obr. 3 Wi-Fi 2.4GHz a Zigbee překrývající spektrum [29]

Open-source implementace Zigbee

Nyní je aktuálně rozvíjených několik open-source projektů zaměřujících se na implementaci koordinátora Zigbee sítě. Všechny projekty mají za cíl zbavit se potřeby používání Zigbee

bran od jednotlivých výrobců a vytvoření univerzální brány pro ovládání těchto zařízení nehledě na výrobce.

Zigbee-herdsman – Projekt naprogramovaný v NodeJS. Podporuje již více než 1000 otestovaných Zigbee zařízení od různých výrobců. Software je tvořený modulárně pomocí několika vrstev. Odchyly od Zigbee standardu zajišťuje pomocí doprovodného projektu Zigbee-herdsman-converters. [30]

Zigbee2MQTT – Projekt od stejného programátora jako Zigbee-herdsman. Jedná se o obal, který umožňuje jednoduchou správu a komunikaci se zařízeními pomocí MQTT (viz 4.5.5). [30]

Zigpy – Python knihovna, která je přepis Zigbee-herdsman. Odchyly od Zigbee protokolu řeší pomocí přídatné knihovny zha-quirks. Jedná se o obecnou implementaci Zigbee.

Deconz-rest-api – Jedná se o doplněk k proprietárnímu softwaru Deconz od německé firmy Dresden Elektronik. Vytváří HTTP API nad tímto softwarem pro správu a používání Zigbee zařízení. Odchyly od Zigbee specifikace se řeší až v tomto API, dle inspekce kódu je mnoho věcí řešeno napevno, odchylky nejsou řešeny modulárně, ale přímo v kódu hlavního API, vede to k horšímu rozšiřování, pro každé zařízení, které není typu světlo, musí být ručně přidána podpora.

4.5.3 Z-Wave

Z-Wave je další technologie, která plně implementuje jak transportní, tak aplikační vrstvu. Až do prosince 2019 se jednalo o uzavřený standard, kde jediným výrobcem rádiových čipů pro Z-Wave byla společnost Silicon Labs. Nyní se otevřela specifikace a Silicon Labs povolila výrobu čipů Z-Wave třetím stranám. [7] To povede k dalšímu rozšíření protokolu. Veškerá zařízení, která se dostanou na trh, musí projít certifikací. Díky povinné certifikaci je teoreticky zajištěna vysoká úroveň kompatibility. Jakékoliv zařízení, které implementuje funkci koordinátora sítě Z-Wave, by mělo být automaticky kompatibilní se všemi zařízeními na tomto protokolu.

Díky standardizovaně definované aplikační vrstvě, která jasně říká, jak data každé Z-Wave zařízení posílá, se získá jednou Z-Wave integrací do centrální jednotky mnoho koncových zařízení chytré domácnosti.

Z-Wave neoperuje na stejné frekvenci po celém světě. Využívá také volné pásmo ISM (viz 4.5.2). Ovšem část 15 tohoto spektra se liší dle regionu světa. Pro Evropu Z-Wave používá 868MHz, zatímco pro USA je to 908MHz. Oproti technologiím operujícím na celosvětově volné frekvenci, jako například 2.4GHz, je nemožné z hlediska legislativy integrovat všechna Z-Wave zařízení. Z legislativních důvodů bude omezena v tomto projektu integrace zařízení, která jsou určena pro evropský trh.

Mezi známé výrobce Z-Wave zařízení pro chytrou domácnost patří například společnost Fibaro nebo Aeotec.

Open-source implementace Z-Wave

Aktuálně existuje jediná pokročilá implementace Z-Wave koordinátora. Jedná se o projekt OpenZWave. Knihovna napsaná v C++, původně reverzně napsaný Z-Wave protokol, po uvolnění Z-Wave specifikace se dá očekávat ještě více stabilní podpora. OpenZWave obsahuje různé wrappery (obaly abstrahující část softwaru), které umožňují jednodušší správu a přístup k zařízením. [31]

Zwave2MQTT – Projekt, který obsahuje OpenZWave knihovnu a zpřístupňuje ji přes MQTT protokol.

Node-openzwave – Tento projekt umožní přístup k OpenZWave přes NodeJS javascriptové prostředí.

4.5.4 RF 433MHz

Volné radiové pásmo (z anglického RF = Radio Frequency) pro zařízení krátkého dosahu operující na 433MHz. Tato frekvence se používá ve spoustě dálkově ovládaných zařízeních. Bezdrátové zvonky, dálkově ovládané rolety a různé další senzory. Pro stavbu nové chytré domácnosti tato zařízení vhodná nejsou, protože tento protokol nemá žádnou kontrolu, zda informace dorazila na cílové místo. U Zigbee, Z-Wave, Wi-Fi funguje kontrola paketů, kdy koncové zařízení dostane odpověď, zda jeho zpráva dorazila do cíle. V případě neúspěchu může zprávu vyslat znovu. Zařízení na této radiové síti nedisponují takovým mechanismem. Některá vyšlou stejnou informaci několikrát za sebou jako způsob proti snížení šance, že cílový bod zprávu nezachytí, to ale není odolné řešení. Pokud se ovšem v domácnosti nějaké dálkově ovládané zařízení na této technologii vyskytuje, je možné ho relativně snadno integrovat.

Do centrální jednotky bude potřeba integrovat zařízení, přes které mohou přijímat a vysílat signály na frekvenci RF 433MHz. U každého zařízení bude potřeba manuálně zaznamenat, jaké signály vysílá a přijímá, ale již nebude potřeba implementovat žádnou další aplikační vrstvu.

Za zmínku stojí projekty RFLink [32] a RFXCOM [33]. Nejedná se o volně dostupné open-source projekty, i když RFLink je dostupný zdarma (freeware). Nabízí ale již vytvořenou interní databázi RF signálů, takže dokážou automaticky rozeznat typ zařízení nebo naopak vysílat signály pro ovládání zařízení bez nutnosti vlastnit ovladač, ze kterého se signály zachytí.

Open-source knihovny pro komunikaci s RF 433MHz

OpenMQTT Gateway – Firmware pro mikordesky typu ESP8266 a ESP32, umožňující provoz MQTT brány různých protokolů jako RF, Infrared, LoRA, Bluetooth LE při připojení patřičných antén. [34]

Tasmota, ESPurna – Firmware pro různá zařízení chytré domácnosti, která používají jako hlavní procesor mikrodisku s čipem ESP8266 nebo ESP32. Umožňuje lokální kontrolu přes MQTT. Podporuje i bránu pro RF protokol. [35] [36]

4.5.5 Wi-Fi, Ethernet (TCP/IP)

Na trhu jsou také zařízení, která k přenášení dat a komunikaci používají prosté Wi-Fi nebo Ethernet připojení. Ač si zařízení může zvolit jinou transportní a aplikační vrstvu, pro komunikaci s dalšími zařízeními používají většinou protokol HTTP nebo COAP přes TCP/IP. Tato zařízení představují nejnižší bariéru pro uživatele pro vstup do kategorie zařízení chytré domácnosti.

Ovšem protokol Wi-Fi je na rozdíl od Zigbee nebo Z-Wave určený pro naprosto jiné použití. Zatímco Zigbee a Z-Wave jsou tzv. nízkoeenergetické sítě, které umožňují provoz koncových zařízení jako pohybových senzorů na jednu baterii CR2032 („knoflíková“ baterie), či dokonce i tzv. Green Power režim, který umožňuje pro některé Zigbee dálkové ovladače získávat energii z pouhého fyzického stisku tlačítka uživatele, tak Wi-Fi je ve srovnání se Zigbee o dost více energeticky náročná. Rádiový modul pro Wi-Fi odebírá značně více energie. Pro zařízení chytré domácnosti na protokolu Wi-Fi je typické, že vyžadují elektrickou síť jako zdroj napájení.

Samotné spojení s Wi-Fi zařízením, které běží na protokolu HTTP nebo COAP na TCP/IP vrstvě je díky dostupnosti softwarové podpory TCP/IP v každém dnešním operačním systému jednoduché. Ovšem úskalí integrace takového zařízení do centrální jednotky chytré domácnosti nastává v tom, že i v případě, kdy každé zařízení zvolí HTTP, je to univerzální textový protokol, kde lze přenášet jakoukoliv zprávu. Každý výrobce Wi-Fi chytrých zařízení si definuje svoji vlastní strukturu zpráv, díky kterým lze ze zařízení konzumovat informace a posílat příkazy. Nelze tedy použít stejný příkaz na zapnutí chytrých Wi-Fi zásuvek od dvou různých výrobců. Některá zařízení ani nemusí komunikovat přes HTTP nebo COAP a používají svůj vlastní protokol nad TCP/IP.

Pro co nejvíce univerzální centrální jednotku je ideální vybrat takové softwarové řešení, které má v sobě již co nejvíce integrací pro TCP/IP zařízení a schopnost dalšího rozšíření. Zde záleží na typu zařízení, zda pro něj existuje open-source integrace.

Existují však aplikační vrstvy nad TCP/IP, které jsou určené pro internet věcí v chytré domácnosti.

HomeKit

HomeKit je protokol pro chytrá zařízení vytvořený společností Apple. Jedná se o implementaci aplikační vrstvy s definovaným typem zpráv, která může běžet nad TCP/IP (Wi-Fi zařízení) nebo Bluetooth transportní vrstvou. Centrální jednotkou je zařízení implementující HomeKit Controller třídu. Z oficiálních zařízení to jsou řady produktů iPhone, iPad, Apple TV. Specifikace tohoto protokolu včetně vývojářských nástrojů je však open-source. Díky tomu jsou k dispozici open-source projekty, které umí emulovat roli centrálního nástroje HomeKit a ovládat zařízení podporující tento protokol. [37] Implementací HomeKit Controller do centrální jednotky chytré domácnosti se zvýší podpora dostupných produktů. Spousta výrobců TCP/IP zařízení, kteří používají své proprietární protokoly, má podporu pro HomeKit jako alternativní ke svému proprietárnímu, což usnadňuje napojení těchto zařízení.

Zároveň otevřená specifikace protokolu umožňuje také opačný scénář. Veškerá připojená zařízení, která budou napojena do centrální jednotky exportovat do sítě jako zařízení typu HomeKit, a tím zpřístupnit možnost ovládání všech zařízení pomocí hlasového asistenta Siri, který je v Apple zařízeních dostupný (viz 4.5.7).

HomeKit Accessory Development Kit – Je open-source implementací HomeKit protokolu pro koncové zařízení přímo od Apple. [38]

Homekit_python, aiohomekit – Open-source Python knihovny, které implementují HomeKit Controller protokol pro ovládání HomeKit zařízení. [39][40]

Hap-controller – Open-source knihovna napsaná v NodeJS, která implementuje HomeKit Controller protokol. [41]

MQTT

MQTT je součástí standardu OASIS/ISO. Jeho účelem bylo vytvoření standardizovaného protokolu pro komunikaci mezi jednotlivými zařízeními internetu věcí. Jedná se o aplikační vrstvu, která běží většinou nad TCP/IP. Umožňuje přenos zpráv mezi jednotlivými zařízeními chytré domácnosti s použitím centrálního zprostředkovatele. Funguje na návrhovém vzoru publisher – subscriber (odesílatel – naslouchač). V síti je přítomen centrální server zvaný zprostředkovatel, tzv. broker. Zařízení internetu věcí v roli odesílatele přenášejí data do centrálního zprostředkovatele pod určitým tématem, která jsou hierarchická a oddělená lomítky. Jiné zařízení internetu věcí, které je v roli naslouchače, poté oznámí centrálnímu zprostředkovateli, o jaká témata mají zájem. Zprostředkovatel poté přepośle zprávy všem naslouchačům. Z praktického příkladu z internetu věcí chytré domácnosti mohou být odesílatelé pohybové senzory. Pohybový senzor umístěný v kuchyni bude odesílat své informace o pohybu do tématu „domov/senzory/pohyb/kuchyň“. Centrální jednotka chytré domácnosti se poté u zprostředkovatele přihlásí, že má zájem o všechny zprávy z tématu „domov“. Tedy dostane veškeré zprávy hierarchicky umístěné v tomto

tématu a pod ním. Není ovšem nijak definované, jaké atributy má přenášet konkrétní typ zařízení. To je již v režii výrobce konkrétního zařízení internetu věcí. MQTT ale pouze definuje platformu pro přenášení jednotlivých zpráv a základní strukturu zpráv.

MQTT se používá více pro vlastnoručně vytvořené projekty nebo v open-source projektech, které umožňují ovládání proprietárních TCP/IP zařízení přes MQTT. Výrobce komerčního produktu chytré domácnosti s MQTT je například společnost Roomba s robotickými vysavači.

Implementací MQTT je mnoho díky tomu, že se jedná o otevřený standard. Seznam je dostupný na stránkách organizace standardu mqtt.org. Používané open-source varianty jsou:

Eclipse Mosquitto – Implementace broker a klienta napsaná v jazyce C. Slouží hlavně pro provoz broker serveru. Zastřešovatelem je Eclipse Foundation.

Apache ActiveMQ – Multiprotokolový server, který také podporuje MQTT.

4.5.6 Proprietární protokoly výrobců

V některých případech je provoz zařízení bez dodavatelem dodávaného koordinátora nereálné. Zařízení mohou používat nezdokumentovaný, zcela proprietární protokol, který se používá jen u konkrétní jedné řady produktů. V tomto případě je neefektivní vývoj otevřeného koordinátora pro tato zařízení, lepší cestou je pokus o integraci do centrální jednotky chytré domácnosti přímo koordinátora dodavatele. Koordinátor většinou slouží pro přemostění koncových zařízení do TCP/IP sítě, aby je mohl zákazník ovládat ze svého mobilního zařízení. Takový koordinátor již komunikuje pomocí standardních protokolů jako HTTP a je možné taková volání odchytil a napsat volně dostupnou implementaci pro ovládání zařízení přes koordinátora dodavatele. Příkladem mohou být chytré termostaty od značky Tado°. Jednotlivé termostaty komunikují na proprietárním protokolu na 868MHz, jejich koordinátor, označovaný jako brána, slouží na přemostění do místní TCP/IP sítě a umožňuje připojení pomocí protokolu HomeKit nebo pomocí API přes síť Internet na straně serveru výrobce.

4.5.7 Hlasoví asistenti

Google Assistant

Pro zprovoznění hlasového asistenta Google Assistant musí centrální jednotka zpřístupnit své zařízení do platformy Google Home, aby mohly být ovládány přes hlasového asistenta. Centrální jednotka musí implementovat API dostupné přes vývojářský balíček Google Actions SDK od společnosti Google. API bude ve formě HTTP REST API. Centrální jednotka bude potřebovat být přístupná na doméně s SSL certifikátem (viz 4.8). [42]

Apple Siri

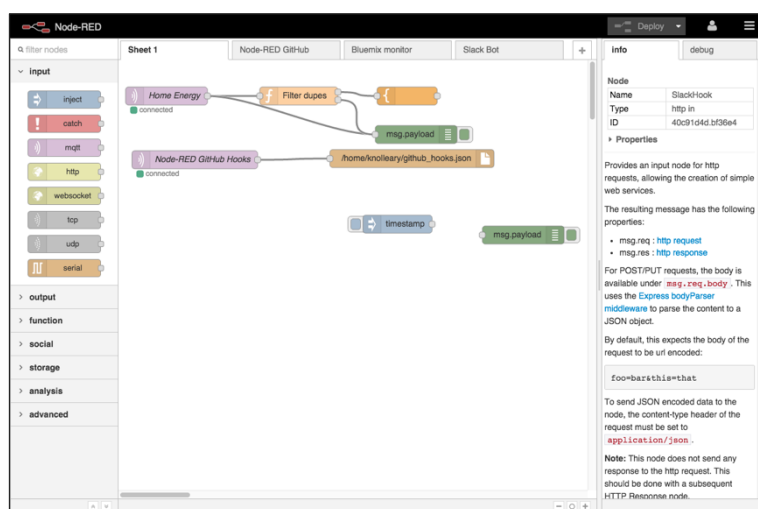
Pro zprovoznění hlasového asistenta Siri je potřeba implementovat v centrální jednotce podporu pro export jejich zařízení do HomeKit protokolu (viz 4.5.5). Poté lze zařízení připojit přes aplikaci Home dostupnou na Apple zařízeních. Apple Siri poté využívá data z aplikace Home pro ovládání zařízení. Pro využití hlasového asistenta Siri mimo domácí síť LAN musí být k dispozici zařízení typu HomeKit Hub, které poskytne přístup k domácím HomeKit zařízením přes servery Applu do sítě Internet. Zařízení typu HomeKit Hub jsou aktuálně: iPad, chytrý reproduktor HomePod, Apple TV (HD nebo 4K model). [43]

4.6 Přehled softwarových platform pro automatizaci chytré domácnosti

Automatizační platforma bude tvořit centrální komponentu softwarového řešení. Do platformy budou propojeny všechny požadované protokoly. Součástí bude také grafické rozhraní pro interakci s uživatelem a možnost vytvářet scénáře pro automatizaci domácnosti.

Node-Red

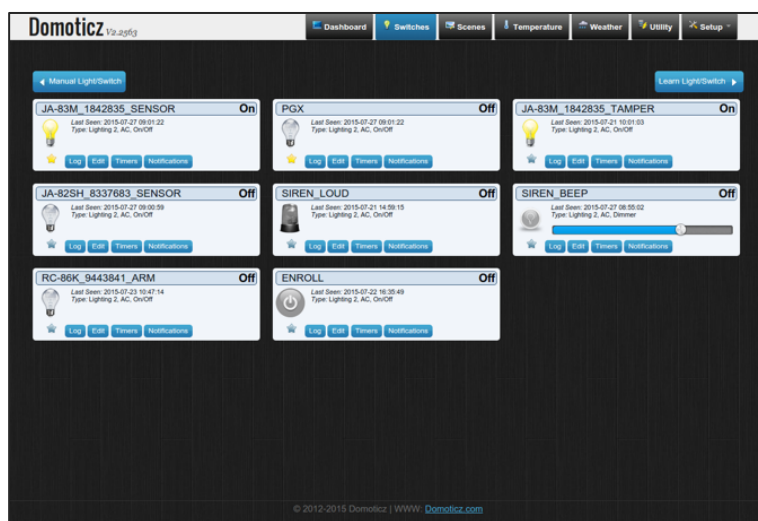
Node-Red představuje automatizační platformu na propojení různých druhů IoT zařízení. Funguje na principu vytváření tzv. flow-based diagramů. Jedná se o vizuální typ programování, kde se prvky propojují vizualizací datových toků. Každý blok může představovat stav IoT zařízení, který lze propojit s jiným pomocí vstupu a výstupu. Například blok *zmáčknuté tlačítko* vyšle signál do *zatáhnout žaluzie*. Node-Red je vhodný doplněk pro architektury k již existující automatizační platformě chytré domácnosti pro vytváření více komplexních automatizačních scénářů. Grafické rozhraní a funkcionalita ale není přímo určená pro vytvoření chytré domácnosti, a proto se nehodí jako hlavní automatizační jednotka pro koncového uživatele chytré domácnosti.



Obr. 4 Webové rozhraní Node-Red pro vytváření automatizačních scénářů [44]

Domoticz

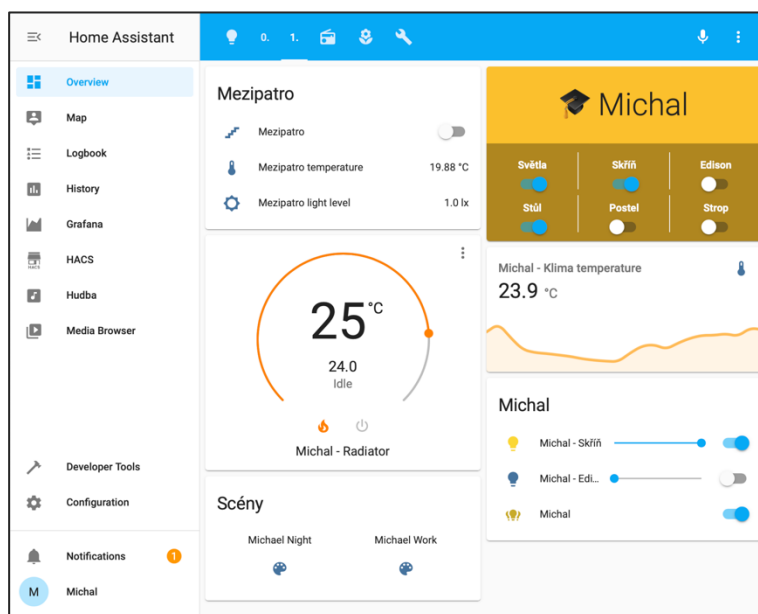
Domoticz je platforma, která se již soustředí přímo na automatizaci chytré domácnosti. Od toho je uzpůsobené grafické rozhraní. Jedná se o webové HTML5 responzivní rozhraní. Kadence vydávání nových verzí je 2x ročně. Dostupná dokumentace není velice propracovaná, chybí poznámky k aktualizacím, zda bude potřeba ruční úprava konfigurace. Bez alternativních rozhraní je možnost kastomizace velice malá. Automatizace lze vytvářet i graficky. Projekt je stále aktivní, dochází k opravám chyb. [45]



Obr. 5 Výchozí webové rozhraní platformy Domoticz [46]

Home Assistant

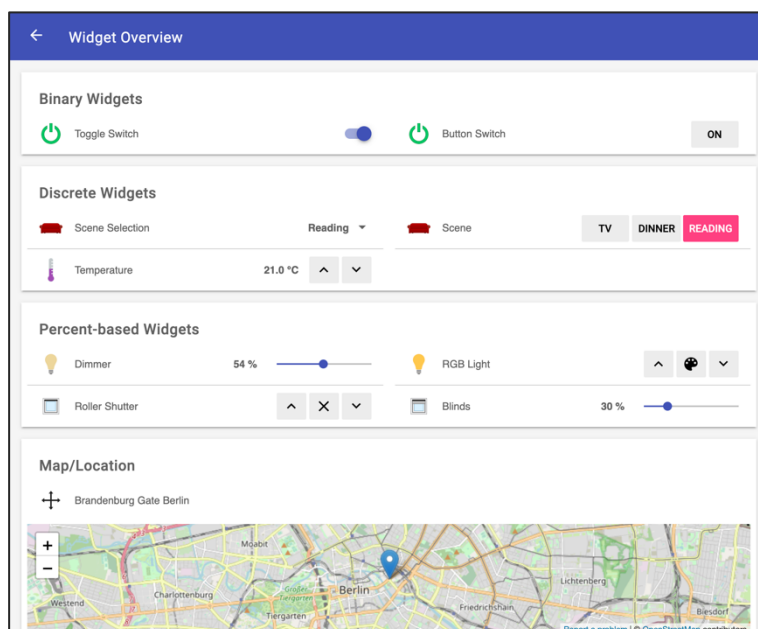
Home Assistant je projekt napsaný v Pythonu. Zaměřuje se na automatizaci domácnosti. Je orientovaný jak na vývojáře, tak posledními verzemi přidává hodně možností konfigurace přes grafické rozhraní. Dále má responzivní, kastomizovatelné webové HTML5 rozhraní přes grafický editor. Jednoduché automatizace je možné vytvořit pomocí průvodce. Je dostatečně přívětivý pro koncové uživatele. Díky velké kastomizovatelnosti lze připravit obrazovky přesně na míru. Umožňuje jednoduché přidávání dalších grafických komponent. Kadence vydávání nových verzí je 6 týdnů. Každé vydání je detailně popsáno. Dokumentace je velice rozsáhlá s popisem konfigurace každé integrace. Ze všech zmíněných projektů v této sekci má aktuálně nejvíce přispěvatelů do kódu dle GitHub statistik. Díky rychlé kadenci stabilních verzí je přidávána podpora i pro nejnovější zařízení chytré domácnosti. [47]



Obr. 6 Výchozí webové rozhraní platformy Home Assistant

OpenHab

Java. Grafické rozhraní je minimalistické, ale velice omezené na kastomizaci. Je ovšem responzivní a v HTML5. Aktuálně je vydána jedna stabilní verze za rok. Výhodou je dlouhý testovací proces, nevýhodou pomalé kadence, kdy bude uživatel déle čekat na podporu nových zařízení chytré domácnosti.



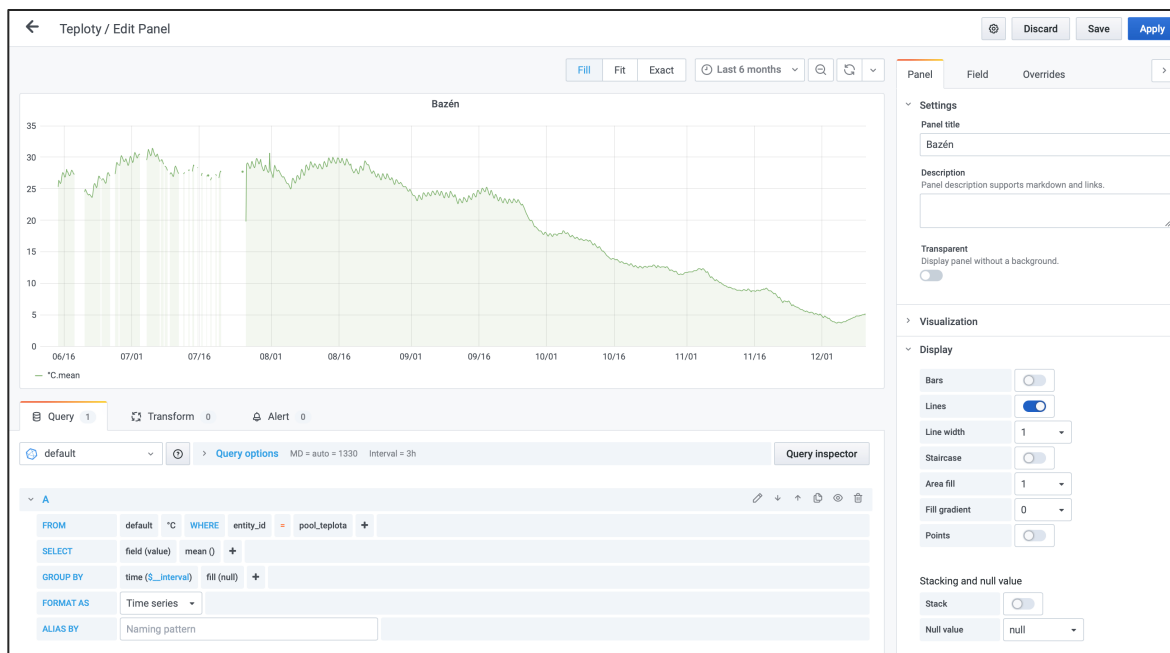
Obr. 7 Webové rozhraní platformy OpenHab

4.7 Historie a data o chytré domácnosti

Pro ukládání a archivaci dlouhodobých dat z chytré domácnosti je vhodné využít druh externí databáze. Archivaci dat do separátní databáze se zrychlí výkon hlavního automatizačního softwaru, a především se umožní jednodušší analýza dat. Další výhodou je uložení do standardizovaného formátu, který je neměnný. V IoT sektoru jsou primárním zdrojem dat sbírané hodnoty ze senzorů. Tato data mají hlavně časový charakter – hodnota senzoru a kdy byla tato hodnota zaznamenána. Pro tento účel byly vytvořeny databáze časových řad. Jsou vysoce optimalizované pro ukládání různých metrik a měření, která mají časové razítko. Díky tomu zabírají mnohem menší místo na disku než v případě použití jiného typu databáze. Oproti typickým relačním databázím jsou omezené jen na data v párovém formátu *čas: hodnota*. Databáze časových řad mají i speciální funkce pro správu časových dat, například převzorkování, které umožňují vytvoření průměrných hodnot za delší časový úsek a ušetření místa na disku.

V práci zaměřené na porovnání databází časových řad popisuje Fadhel et al. několik open-source databází a jejich výkonnost. Zmiňuje: OpenTSDB, InfluxDB, Elasticsearch, Kdb+, RRDtool, Graphite, Prometheus, DalmatinerDB, SQLite3. InfluxDB byl vybrán jako první možnost díky nejmenším nárokům na diskové úložiště a rozumným výpočetním nárokům. [48 s. 9]

Pro následné grafické zobrazení dat v podobě grafů se využívají vizualizační nástroje, které se umí napojit do databáze. Grafana je open-source vizualizační software podporující InfluxDB jako jednu z připojitelných databází. [49]



Obr. 8 Ukázka vizualizace historických dat v softwaru Grafana

4.8 Vzdálený přístup a správa

Vzdálený přístup

Dle analýzy platform pro automatizaci (viz 4.6) a vizualizačních nástrojů pro data chytré domácnosti (viz 4.7) jsou jejich grafická rozhraní ve formě webových aplikací. Každá aplikace běží na svém vlastním portu. Pro přístup aplikace ze sítě Internet lze všechny běžící porty aplikací otevřít firewallem. Nejedná se však o vhodné řešení. Pro zabezpečení by každý interní webový server aplikace musel mít implementováno bezpečné připojení HTTPS (viz 4.9).

Jako druhá varianta pro přístup k více než jedné aplikaci na jednom serveru se využívá tzv. reverzních proxy serverů. Jedná se o webový server, který je přístupný ze sítě Internet (ze strany klienta), a podle příchozích požadavků rozhodne, jakou zdrojovou webovou aplikaci klientovi zpřístupní. Na straně firewallu je otevřený typicky jen jeden či dva výchozí porty (Port 80 pro HTTP a port 443 pro HTTPS). Proxy server poté předává komunikaci z interní sítě ven a naopak. Takové řešení může také jednoduše implementovat bezpečné HTTPS připojení, protože certifikát (viz 4.9) se konfiguruje na jednom místě. [50]

Příchozí požadavek může být ve formě doménového jména (např. aplikace1.mojedomena.cz, aplikace2.mojedomena.cz). A proxy server poté přepoše data ze serveru v domácí síti. Pro tuto funkčnost je nutné mít zakoupenou doménu a odkázat ji v nastavení DNS na IP adresu přípojky, kde bude centrální jednotka chytré domácnosti. DNS je zkratka pro Domain Name System (doménový jmenný systém, překlad autora). Jedná se o základní stavební blok sítě Internet. DNS servery jsou katalog všech domén – ke každé doméně obsahují informaci, na kterou IP adresu doména odkazuje. Při zadání domény, jako www.mojedomena.cz, do webového prohlížeče je kontaktován DNS server, který zpět vrátí IP adresu pro tuto doménu, na kterou se poté prohlížeč připojí.

Je potřeba, aby internetové připojení na místě chytré domácnosti disponovalo veřejnou IP adresou. Taková IP adresa je přístupná přímo ze sítě Internet. Pokud se jedná o neveřejnou, znamená to, že internetová přípojka se nachází v další interní LAN síti, která je pod správou poskytovatele internetového připojení. Veřejná IP adresa přidělovaná poskytovatelem nemusí být statická, ale mění se za interval času. Pro tento případ je třeba využít služby dynamického DNS, kterou poskytují někteří registrátoři domén. Taková služba automaticky mění IP adresu, na kterou doména odkazuje v závislosti na změně poskytované IP adresy. Pro tuto funkčnost se musí na straně přípojky nainstalovat software, který periodicky zjišťuje, jaká je aktuální IP adresa přidělená poskytovatelem, a tuto informaci poté pošle na server služby dynamické DNS. [51]

Nginx je open-source software, který byl přímo vytvořen pro roli webového serveru a reverzního proxy serveru. Dalším open-source softwarem, který lze nakonfigurovat jako reverzní proxy server, je Apache HTTP Server. [52]

Vzdálená správa

Výchozí rozhraní pro interakci a konfiguraci GNU/Linux a unixových systému je textové terminálové rozhraní zvané shell. Pro GNU/Linux byl vytvořen open-source Bourne-again shell (zkráceně bash). Jedná se o výchozí terminál u většiny linuxových distribucí.

Protokol SSH umožňuje vzdálený přístup k terminálu operačního systému z jiného počítače. Jedná se o zkratku pro Secure Shell (Zabezpečený terminál, překlad autora). Další konfigurace a údržba systému se již poté může provádět z jiného počítače v síti a server lze umístit na místo, které nemusí být lehce přístupné. MacOS, Linux a Windows 10 (verze 1709) mají již ssh klienty nainstalované. Pro starší verze Windows lze využít software PuTTY.

Kromě terminálu je součástí SSH také Secure File Transfer Protocol (SFTP). Ten umožní jednoduché posílání a úpravu souborů na serveru z jiného počítače v síti přes SSH protokol. Na Windows je pro SFTP vhodný program WinSCP a na macOS program Cyberduck.

4.9 Bezpečnost

Aby byl navrhovaný systém bezpečný, je třeba zamezit neoprávněným uživatelům vstup do systému, jednotlivých protokolů a zabránit napadnutí komunikace mezi zařízeními.

Vzdálený přístup

Přístup ze sítě Internet bude dle předchozí analýzy (viz 4.8) zajištěn pomocí webového rozhraní. Aby nemohlo dojít k odposlechu komunikace, musí se využít HTTPS protokol, což je HTTP, které šifruje komunikaci pomocí SSL/TLS protokolu.

Princip zabezpečení funguje na asymetrickém a symetrickém šifrování. Každá strana komunikace (klient a server) vygeneruje při začátku spojení kombinaci veřejného a privátního klíče (asymetrické šifrování). Veřejné klíče si mezi sebou pošlou. Zpráva zašifrovaná veřejným klíčem jde dešifrovat pouze privátním klíčem a naopak, tedy zpráva zašifrovaná privátním klíčem jde dešifrovat pouze klíčem veřejným.

Po této fázi si obě strany zabezpečeně vymění náhodně generovaná data a vytvoří symetrický klíč pro symetrické šifrování. Tím dojde k vypnutí asymetrického šifrování a k další komunikaci již bude použitý tento symetrický klíč.

Poté dochází k ověření totožnosti serveru. Tedy, že webová stránka je servírovaná správnou stranou a ne útočníkem. K tomu slouží tzv. SSL certifikáty. Certifikát obsahuje veřejný klíč serveru a identitu vlastníka webové stránky. SSL certifikát si může server vygenerovat sám (tzv. self-signed). Ovšem tyto certifikáty bude jakýkoliv prohlížeč považovat jako nedůvěryhodné.

Pro získání důvěryhodného certifikátu je zapotřebí certifikační autorita, která ověří vlastníka webového serveru (například vlastník musí umístit textový soubor, který autorita vygeneruje, na svůj server). Vlastník serveru poté vygeneruje kombinaci veřejného a privátního klíče. Veřejný klíč zašle certifikační autoritě, která mu vydá certifikát, jehož obsahem je veřejný klíč vlastníka serveru a celý certifikát je podepsaný privátním klíčem certifikační autority. Veškeré používané operační systémy jako Windows, Linux, macOS, iOS, Android mají nainstalovanou sadu veřejných klíčů těchto důvěryhodných certifikačních autorit. Tímto mohou ověřit, zda je certifikát serveru pravdivý, a označit ho jako důvěryhodný (v dnešních grafických rozhraních je to většinou reprezentováno zeleným zámečkem v adresním řádku prohlížeče). Pro certifikát je potřeba vlastnictví domény.

Certifikační autorita, která nabízí zdarma podepsané certifikáty, je služba Lets Encrypt. [53] Při návrhu řešení bude třeba vytvořit řešení, které bude certifikát automaticky průběžně obnovovat.

Uznávanou testovací sadou zabezpečení webu je nástroj SSL Labs tester firmy Qualys. Jejich dokumentace pro nejlepší praktiky správného nasazení a konfigurace HTTPS udává následující parametry, které se týkají bezpečnosti:

Typ protokolu – TLS verze 1.2 a vyšší (jsou zatím bez známých bezpečnostních děr).

Implementace Forward Secrecy – funkce, která zabezpečí veškerou předchozí komunikaci při prolomení privátního klíče na serveru dalším šifrováním.

Strict Transport Security (HSTS) – Tento příznak pošle server klientovi při úplně prvním spojení. Příznak říká, aby klient při veškerých dalších spojeních využíval jen HTTPS spojení. Jedná se o ochranu proti útoku „man-in-the-middle“, kdy se mezi server a klienta postaví útočník, který napodobuje server, ale protože se nemůže ověřit certifikátem, snaží se komunikaci přehodit na nezabezpečený HTTP.

Dále je nutné, aby kromě komunikace byl zabezpečený vstup do aplikace. Moderní přihlašovací metodou je používání dočasných přístupových tokenů. Uživatel při přihlášení do aplikace zadá uživatelské jméno, heslo a server mu vrátí vygenerovaný kód, který bude používat po celou dobu sezení. Kód může mít omezenou platnost a lze ho zablokovat na serveru.

Vzdálená správa

Pro vzdálenou správu je z předchozí analýzy použitý protokol SSH pro vzdálený terminálový přístup k systému. SSH umožňuje používat kombinaci jména a hesla. Toto zabezpečení je dostatečné pro domácí lokální síť (LAN).

V případě publikování SSH serveru do sítě Internet je třeba využít asymetrického šifrování, jak bylo vysvětleno v kapitole výše. Uživatel vygeneruje pár privátního a veřejného klíče. Privátní klíč si nechá a veřejný klíč umístí na SSH server. Při vzdáleném připojení server

vygeneruje matematický úkol, který zašifruje veřejným klíčem uživatele. Jenom SSH klient, který vlastní privátní klíč uživatele, je tedy schopný úkol dešifrovat a poslat serveru odpověď. Tímto se uživatel vyhne útokům „man-in-the-middle“, kdy se za server bude vydávat někdo jiný, který bude odposlouchávat hesla.

Proti tomuto typu útoku je SSH chráněno i bez využití klíče, výše uvedené řešení jen přidává další vrstvu ochrany. Každý SSH klient si totiž při prvním připojení k SSH serveru uloží jeho veřejný klíč, který se využívá vždy. Při každém dalším spojení si SSH klient ověří, zda se připojuje ke stejnému serveru.

SSH server běží na výchozím portu 22. Při otevření do sítě Internet se využívá jiný než výchozí port, aby se dalo vyhnout útokům, které fungují na základě skenování otevřenosti portů, které se používají ve výchozím nastavení pro tyto služby.

Protokoly domácnosti

Sítě Zigbee a Z-Wave jsou ve výchozím stavu šifrované, a není potřeba další konfigurace. Ke zranitelnosti dochází pouze při párování nových zařízení, kdy se síť nachází v otevřeném režimu a může se do nich připojit libovolné zařízení v párovacím režimu. Proto je nezbytné zapínat otevřený režim jen při přidání nových zařízení a poté ho vypnout. Nová Zigbee 3.0 certifikovaná zařízení obsahují instalační kód, který lze využít jako metodu bezpečného připojení do sítě, kdy uživatel přepne zařízení do párovacího režimu a kód zadá do koordinátora. Nemůže tak dojít ke spárování jiného zařízení.

Pro přístup do TCP/IP sítě (domácí síť LAN) se využívá fyzický port Ethernet a Wi-Fi. Jelikož Ethernet je fyzické kabelové připojení, útočník by musel být fyzicky přítomný uvnitř domu, kde se nachází Ethernet port. U bezdrátového připojení Wi-Fi je třeba využít nejnovější druh zabezpečení. Jedná se o WPA2-PSK nebo WPA3. Zkratka znamená *Wi-Fi Protected Access – Pre-shared Key* (Wi-Fi zabezpečený přístup – Předsdílený klíč). Předsdílený klíč představuje heslo, které uživatelé použijí pro přihlášení k Wi-Fi. Při konfiguraci zabezpečené Wi-Fi pro domácí síť LAN je nezbytné zvolit tento druh zabezpečení. Pokud bezdrátový přístupový bod, který poskytuje Wi-Fi, umožňuje vytvořit síť pro hosty (guest network), měla by se tato funkce využít. Síť pro hosty je označení u spotřebitelských routerů s funkcí bezdrátového přístupového bodu pro vytvoření podsítě, která nemá přístup k žádným zařízením v domácí síti LAN. Pouze slouží k přístupu na síť Internet. Zároveň se uživatel vyhne sdílení svého předsdíleného klíče (Wi-Fi hesla).

MQTT umožňuje přenášet data pomocí TLS, který je vysvětlen v sekci výše. Druhou možností je základní autentizace pomocí jména a hesla. Jelikož bude MQTT síť nasazena ve vnitřní LAN síti a nebude otevřena do sítě Internet, lze použít základní autentizaci.

Zařízení využívající HomeKit používají TLS šifrování ve výchozím nastavení. Pro přístup k HomeKit zařízením ze sítě Internet (pro Apple Siri hlasového asistenta) je také automaticky využito TLS šifrování. Každé HomeKit zařízení obsahuje svůj kód (buď

ve formě štítku na zařízení nebo vygenerovaný v aplikaci), který uživatel zadá při párování. Nedochází k otevření sítě ani při párování. [37]

5 Návrh řešení

Na základě analýzy bude vytvořeno následující řešení:

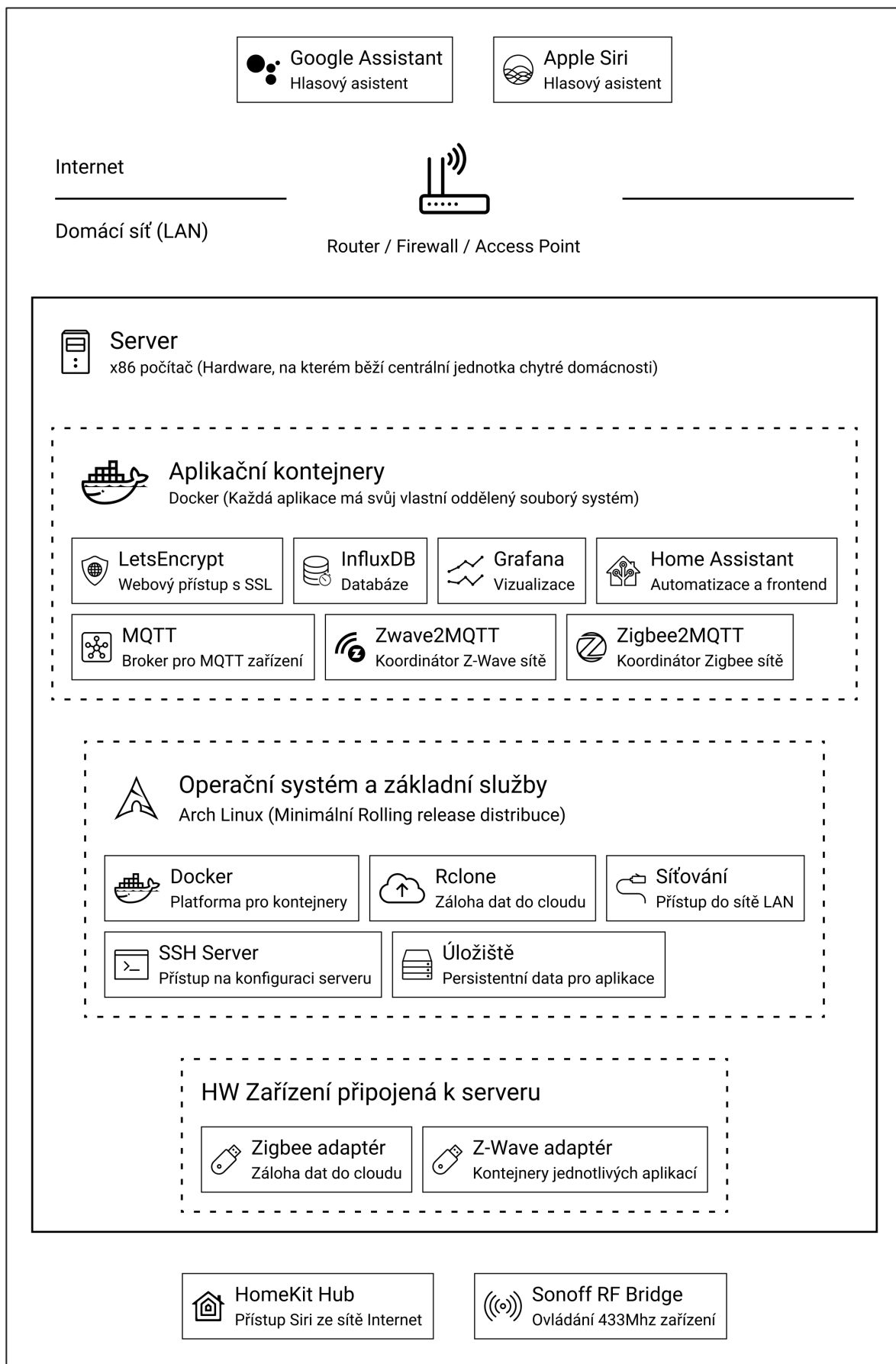
Centrální jednotka chytré domácnosti, která integruje protokoly: Zigbee, Z-Wave, MQTT, HomeKit, 433 MHz a bude mít platformu pro připojení různých TCP/IP zařízení. Centrální jednotka bude umožňovat vytváření automatizací pro integrovaná zařízení. Bude mít responzivní webové grafické rozhraní pro mobilní telefony až počítače. Zároveň bude možné veškerá zařízení ovládat přes hlasové asistenty Google Assistant a Apple Siri. Ovládání přes hlasové asistenty a grafické rozhraní bude dostupné i mimo domácí síť díky zabezpečenému HTTPS rozhraní. Koncový uživatel bude moci vytvářet vlastní jednoduché automatizace a kustomizovat grafické prostředí. Centrální jednotka bude moci ukládat časová data ze senzorů chytré domácnosti do databáze časových řad, nad kterou bude moci koncový uživatel samoobslužně stavět historické grafy. Vše bude běžet na minimalistickém operačním systému Arch Linux v modulárních Docker kontejnerech a bude průběžně zálohováno. Centrální jednotka bude mít dostupnou vzdálenou správu přes zabezpečený SSH protokol.

5.1 Diagramy návrhu

Diagramy jsou velmi vhodné pro grafické znázornění systému. Umožní jednoduchý pohled, který přehledně ukáže jednotlivé části systému a vztahy mezi nimi. V předešlé sekci Fáze analýzy (viz **Chyba! Nenalezen zdroj odkazů.**) byl pro návrh architektury systému zvolen přístup Typového aplikačního softwaru v podobě open-source. Protože se nebude jednat o integrovaný balík, ale komponentové řešení různých softwarů, je vhodné zachytit strukturu veškerého nainstalovaného softwaru a hardwaru. Pro tento účel lze využít inspiraci z UML Diagramů. UML je jazyk, jehož cílem je standardizace různých návrhových diagramů, které pomáhají k vizualizaci a pochopení navrhovaných softwarových řešení.

5.1.1 Komponentový návrh řešení

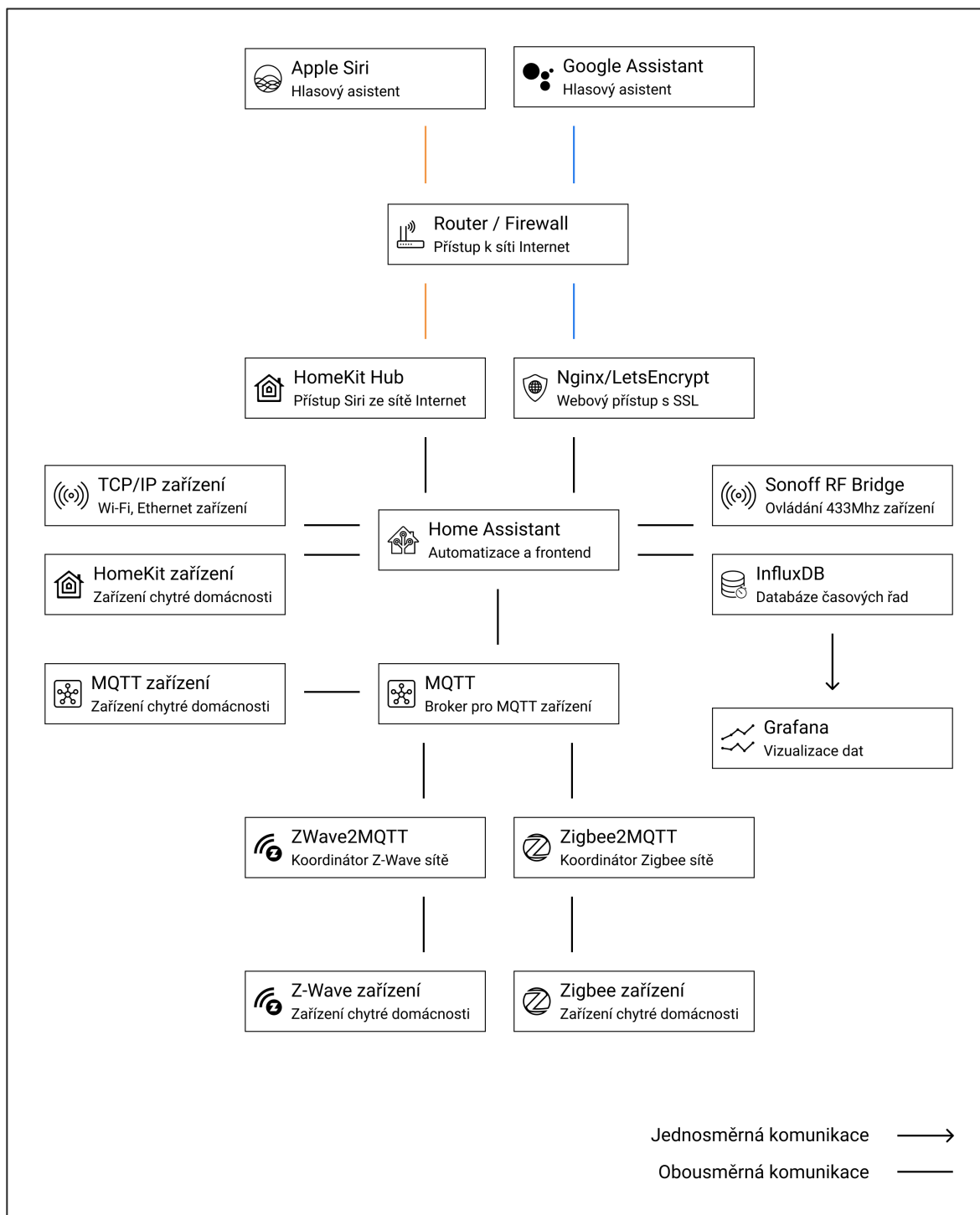
Následující obrázek vychází z principu UML diagramu nasazení, který popisuje hierarchii a zasazení jednotlivých softwarových komponent na hardwarová zařízení. [15 s. 304] Tyto softwarové komponenty tvoří jednotlivé části výsledné centrální jednotky chytré domácnosti.



Obr. 9 Návrh řešení – Komponenty

5.1.2 Návrh řešení z pohledu datových toků

Následující obrázek je diagram datových toků zobrazující, jaké komponenty se mezi sebou integrují. Je zde znázorněn komunikační tok dat mezi jednotlivými komponentami.



Obr. 10 Návrh řešení z pohledu komunikace mezi jednotlivými komponentami

5.2 Popis návrhu

5.2.1 Hardware serveru

Pro provoz veškerých softwarových komponent chytré domácnosti není třeba na dnešní poměry vysoce výkonného hardwaru. Pro plynulý běh řešení, bude implementována následující platforma:

- Procesor: Intel Core i3-4330, @3,5GHz (dvoujádrový procesor z roku 2013)
- Operační paměť: 2x4GB DDR3 na 1300MHz
- Grafická karta: Pouze integrovaná grafika z procesoru (Intel HD 4600)
 - Je potřeba pouze při instalaci systému pro textový výstup na monitor
 - Při použití procesoru bez grafiky je možné dočasně připojit dedikovanou grafickou kartu z jiného počítače a po nastavení SSH Serveru (viz 6.7) ji lze odpojit
- Základní deska: Asus H87M-Pro
- Disk: 120 GB SSD Adata
 - Úložiště pro systém a aplikace
- Zigbee koordinátor – USB rádio s čipem CC2652RB
- Z-Wave koordinátor – Aeotec Z-Stick Gen5 USB
- PCIe Ethernet síťová karta Intel PRO/1000 CT

Návrh míří na splnění požadavku pro průměrné vytížení systémových prostředků na méně než 30 % z dostupných zdrojů (viz 3.1.1).

5.2.2 Operační systém

Pro provoz centrální jednotky chytré domácnosti je zvolena GNU/Linux distribuce s názvem Arch Linux. [54] Jedná se o univerzální distribuci, která má možnost velice minimální instalace. Ve své základní instalaci obsahuje jen absolutně nezbytné balíčky pro provoz operačního systému. Vše ostatní je na výběru uživateli. Spadá do rolling-release modelu. Stabilita rolling-release systému je zajištěna instalací nezákladního softwaru do oddělených kontejnerů. Díky tomuto střídavému přístupu k rolling-release budou výsledkem velice malé nároky na provoz tohoto operačního systému, protože nebudou nainstalované žádné balíčky softwaru navíc, jednoduché aktualizace a bude zajištěna vysoká stabilita systému.

Je třeba zmínit, že Arch Linux patří k nejlépe dokumentovaným distribucím. Jeho dokumentace je tak rozsáhlá, že ji lze využít i při konfiguraci softwaru na zcela jiné distribuci. [55]

Minimální přístup Arch Linuxu má za výsledek, že distribuce neobsahuje vlastního grafického ani textového průvodce instalací. Instalace se běžně provádí zcela ručně pomocí jednotlivých příkazů. Dokumentace provede postupně uživatele nezbytnými kroky. [56] Existuje však projekt s názvem Anarchy Installer, který má za cíl vytvoření průvodce

instalací [57]. Tento instalátor implementuje skript v podobě textového rozhraní, který vykoná potřebné kroky pro minimální instalaci tak, jak jsou popsány v dokumentaci Arch Linuxu.

Budou vykonány následující operace:

- Instalace minimální spustitelné verze Arch Linux pomocí projektu Anarchy Installer
 - Jazykové nastavení, formát disku, instalace balíčků softwaru, vytvoření uživatelů a práv, instalace zavaděče
- Základní konfigurace systému po instalaci
 - Připojení do domácí sítě LAN a do sítě Internet
 - Nastavení synchronizace systémového času
 - Konfigurace zavaděče systému

5.2.3 Oddělený provoz aplikací

Aplikace budou dle analýzy (viz 4.4) provozovány na kontejnerové platformě. Byl vybrán Docker, jelikož všechny aplikace, které se budou provozovat, mají DockerHub jako jednu ze svých metod distribuce softwaru.

Pro každou aplikaci, která běží v Docker kontejneru, bude vytvořena samostatná složka v operačním systému, která se k němu připojí. Do složky budou ukládána veškerá uživatelská data aplikace, která mají být zachována přes aktualizace kontejnerové aplikace. Tento přístup umožní jednoduché zálohování, jelikož budou veškerá data na jednom místě.

5.2.4 Automatizace chytré domácnosti

Z analýzy (viz 4.6) byl pro automatizační platformu a grafické rozhraní chytré domácnosti zvolen Home Assistant.

Bude vytvořen Docker kontejner z oficiálního Home Assistant obrazu. Bude provedena základní konfigurace – vytvoření uživatelského účtu, připravení pro integrace protokolů a možnost vzdáleného přístupu.

Zigbee, Z-Wave, 433 MHz protokoly budou připojeny do Home Assistant přes MQTT. Home Assistant disponuje funkcí MQTT Auto Discovery. Tato funkce umožní integracím, které využívají MQTT, automaticky oznámit přidání nového zařízení do Home Assistant bez další konfigurace.

5.2.5 Jednotlivé protokoly chytré domácnosti

MQTT

Pro MQTT server byl vybrán projekt **Eclipse Mosquitto**. Má jednoduchou konfiguraci a má oficiálně distribuovaný obraz na DockerHub. MQTT poté bude napojen do Home Assistant.

Zigbee

Pro Zigbee koordinátora je zvolen **Zigbee2MQTT** projekt. Bude napojen na MQTT server, na kterém bude poslouchat Home Assistant. Tímto odpadne potřeba párovat znovu všechna zařízení, pokud se vymění v budoucnu automatizační platforma (Home Assistant).

Hardware Zigbee koordinátora je USB rádio na čipu od Texas Instruments (CC2652RB). Jedná se o výkonnější variantu z podporovaných zařízení dle dokumentace Zigbee2MQTT. Do tohoto rádia bude nahrán firmware pro roli Zigbee koordinátora (je tvořen Z-Stack knihovnou od Texas Instruments, která implementuje Zigbee hardwarovou podporu), je dostupný od stejného vývojáře projektu Zigbee2MQTT. Pro tento krok bude implementován vlastní kontejnerový obraz s open-source nástrojem **cc2538-bsl**, který umí nahrát firmware do některých Texas Instruments rádio čipů.

Po připravení USB rádia bude vytvořen Docker kontejner z oficiálního obrazu Zigbee2MQTT na DockerHub a bude provedena základní konfigurace a napojení na MQTT server. Zigbee2MQTT podporuje Home Assistant MQTT Auto Discovery.

Z-Wave

Pro koordinátora Z-Wave sítě je zvolen projekt **Zwave2MQTT**. Z-Wave síť bude ovládána přes MQTT server. ZWave2MQTT podporuje Home Assistant MQTT Auto Discovery.

Dokumentace OpenZWave, projekt, na kterém ZWave2MQTT běží, udává, že každý USB Z-Wave koordinátor má fungovat. Byl zvolen USB koordinátor Aeotec Z-Stick Gen5+, který přímo specifikuje podporu pro tento projekt. [58]

HomeKit

Home Assistant obsahuje integraci knihovny **aiohomekit**, která implementuje třídu HomeKit Controller, umožňující ovládání HomeKit zařízení. Tato integrace bude v Home Assistant spuštěna.

Wi-Fi, Ethernet (TCP/IP)

Díky různorodosti aplikačních vrstev jednotlivých Wi-Fi zařízení nelze přesně definovat integrační postup pro každé zařízení. Pro připojení těchto zařízení bude využita platforma

integrací přímo v Home Assistant, která umožňuje bez dalšího vývoje připojit stovky druhů TCP/IP zařízení. [59]

433 MHz

K ovládání zařízení operujících na 433 MHz bude využitý projekt **OpenMQTTGateway**, který umožňuje ovládat zařízení s 433 MHz rádiem přes MQTT.

OpenMQTTGateway bude nainstalován na zařízení Sonoff RF Bridge. Toto zařízení bude připojené přes Wi-Fi do domácí sítě LAN a nakonfigurováno na spojení s MQTT serverem. Bude implementována ukázková definice bezdrátového tlačítka 433 MHz v Home Assistant.

Koexistence protokolů

Dle analýzy protokolů dochází ke konfliktům u sítí, které operují na 2.4 GHz. Tedy Wi-Fi a Zigbee. Veškerá moderní Wi-Fi zařízení budou odsunuta na novější 5 GHz pásmo. 2.4 GHz pásmo Wi-Fi a Zigbee od sebe budou co nejvíce vzdáleny výběrem kanálů na opačném frekvenčním spektru. Pro Wi-Fi bude použitý kanál 1 a pro Zigbee kanál 25.

Dále budou USB rádia Zigbee a Z-Wave umístěna v bezpečné vzdálenosti minimálně 0,5 m od sebe, aby nedocházelo k vzájemnému rušení. Zigbee rádio bude zapojeno do USB 2.0 portu, jelikož dle reportu společnosti Intel u USB 3.0 může docházet ke generování šumu na frekvenci 2.4 GHz - 5 GHz. [60]

5.2.6 Zálohování

Pro zálohování bude použitý open-source program **rcclone**. Jedná se o příkazovou utilitu, která umožňuje práci se soubory na cloudových úložištích typu Dropbox, Google Drive, OneDrive, vzdálený FTP, WebDAV server atp. Dále bude využita linuxová utilita **tar**, která zabalí do archivu následující data:

- Persistentní data z kontejnerů
- Konfigurace pro spuštění kontejnerů
- Všechny upravené soubory v průběhu implementace

Bude implementován skript, který se spustí pomocí časovače systemd-timer každé ráno ve 3 hodiny. Provede archivaci dat a poté je nahraje na cloudové úložiště.

5.2.7 Vzdálený přístup a správa

Vzdálený přístup

Na místě instalace centrální jednotky chytré domácnosti je k dispozici statická veřejná IP adresa. Bude zakoupena doména a nasměrována na IP adresu domácnosti (viz 4.8). Pro

vzdálený přístup k webovým rozhraním chytré domácnosti bude nainstalován Nginx webový server v konfiguraci jako reverzní proxy server. Na routeru v domácí síti LAN bude otevřen port 80 a 443 pro reverzní proxy server do sítě Internet. Nginx bude běžet ve formě Docker kontejneru z obrazu open-source projektu **linuxserver/swag**. Jedná se o kontejnerový obraz, který v sobě zahrnuje Nginx a automatickou integraci služby LetsEncrypt pro získání doménového SSL certifikátu.

Vzdálená správa

Do operačního systému bude nainstalován základní balíček **openssh**, který poskytuje SSH server (viz 4.8). SSH Server bude spuštěn jako služba běžící na pozadí.

5.2.8 Bezpečnost

Vzdálený přístup

Webový server bude zabezpečen pomocí HTTPS spojení. Díky využití kontejnerového obrazu z open-source projektu **linuxserver/swag** bude nakonfigurován skript pro obdržení aktuálního SSL certifikátu. Dle analýzy (viz 4.9) bude Nginx nakonfigurován následovně:

- Automatické přesměrování veškerých požadavků na zabezpečené spojení. (HTTP na HTTPS).
- Minimální povolené zabezpečení klienta – TLS verze 1.2
- Zapnutá funkce Strict Transport Security (HSTS)
- Povolená Forward Secrecy
- Vytvořen proxy server pro webové rozhraní Home Assistant a Grafana
 - Home Assistant bude přístupný na hlavní doméně
 - Grafana na subdoméně grafana.domena.cz

Vzdálená správa

Pro zpřístupnění SSH server přes síť Internet se vytvoří pomocí **openssh** kombinace privátního a veřejného klíče. Veřejný klíč bude umístěn do konfigurační složky SSH na serveru. Privátní klíč bude uložen na klientském zařízení, které se bude na SSH server připojovat. Z lokální sítě LAN bude povolen přístup pomocí uživatelského jména a hesla. Ze sítě Internet bude povolen přístup jen pomocí privátního klíče. Na routeru bude otevřen náhodný 5místný port, který bude odkazovat na SSH server. Nepoužije se výchozí SSH port 22 z důvodu bezpečnosti. (viz 4.8)

Protokoly chytré domácnosti

Dle analýzy bezpečnosti protokolů (viz 4.9) bude použité následující nastavení:

- Pro MQTT bude využité ověřování pomocí kombinace uživatelského jména a hesla

- Wi-Fi bude zabezpečena protokolem WPA2-PSK
- Zigbee, Z-Wave, HomeKit jsou šifrované automaticky
 - Zigbee, Z-Wave síť bude otevřena jen při připojování nových zařízení

5.2.9 Hlasoví asistenti

Google Assistant

Pro integraci Google Assistant je nejdříve potřeba zprovoznit vzdálený přístup přes HTTPS (viz 5.2.7 a 5.2.8) a vlastnit Google účet, přes který bude používán Google Assistant. Poté bude provedena integrace ve 2 krocích:

- Vytvoření projektu na platformě Actions on Google [42] a odkázání na doménu běžící instance Home Assistant
- Zapnutí Google Assistant integrace v Home Assistant pomocí vygenerovaných autentizačních tokenů v Actions on Google

Apple Siri

Pro integraci Apple Siri je třeba mít založený Apple ID účet. Budou provedeny následující kroky:

- V Home Assistant bude zapnutá integrace HomeKit, která exportuje všechna zařízení integrovaná do Home Assistant do domácí sítě LAN jako HomeKit zařízení
- Na Apple zařízení (iPhone) v aplikaci Home budou přidána Home Assistant zařízení
- Pro ovládání zařízení ze sítě Internet pomocí Apple Siri bude nastavena Apple TV do role HomeKit Hub

5.2.10 Aktualizace, údržba

Pro aktualizaci systému a softwaru je vytvořen postup, který bude proveden vývojářem systému při každé údržbě.

Aktualizace systému

Pro pravidelnou aktualizaci systému bude vytvořen skript, který provede následující kroky:

- Aktualizuje seznam balíčků a jejich aktuálních verzí z repozitáře
- Aktualizuje nainstalované balíčky
- Vyčistí dočasnou paměť od instalačních souborů

Aktualizace aplikací provozovaných v kontejnerech

Bude implementován skript, který vykoná následující kroky:

- Odkáže na stránky vývojáře daného kontejnerového obrazu, kde jsou k nahlédnutí informace o změnách. Uživatel se rozhodne, zda je potřeba vykonat manuální změny v konfiguraci
- Skript se zeptá, zda má pokračovat
- Stáhne se nový kontejnerový obraz
- Ukončí se běžící kontejner
- Z nového kontejnerového obrazu se vytvoří a spustí nový kontejner se stejnou konfigurací

Uživatel ověří funkčnost a v případě nenalezení žádných chyb budou vymazány staré kontejnerové obrazy pomocí příkazu *docker image prune*.

5.2.11 Historie dat

Pro ukládání dat bude využita databáze časových řad InfluxDB dle analýzy (viz 4.7) a pro zobrazení dat byla vybrán software Grafana. Obě komponenty budou zprovozněny pomocí kontejnerů.

InfluxDB

- Databáze využije oficiální kontejnerový obraz **influxdb**
- Proběhne inicializace a vytvoření prázdné databáze *homeassistant* a uživatelského účtu s oprávněními pro zápis do databáze
- V Home Assistant bude nastaveno ukládání dat do databáze pro vybraná zařízení

Grafana

- Grafana využije oficiální kontejnerový obraz **grafana/grafana**
- Bude vytvořen uživatelský účet pro přístup do softwaru Grafana
- Bude vytvořeno napojení do databáze InfluxDB

6 Implementace návrhu

6.1 Operační systém

Předpokladem před instalací operačního systému je plně sestavený hardware z komponent udaných v návrhu hardwaru. (viz 5.2.1)

6.1.1 Instalace operačního systému Arch Linux

Anarchy Installer byl použitý pro instalaci systému.

Instalace je provedena podle postupu, který je detailně dokumentován v příloze (viz Příloha A:).

- Nastavení jazykového prostředí systému (znaková sada klávesnice, časová zóna, jaké aktualizační servery se mají používat)
- Naformátování pevného disku
 - Použiji výchozí GNU/Linux souborový systém ext4
- Vybrání minimální skupiny balíčky softwaru
 - Arch Linux skupina balíčků *base*, která obsahuje Linux jádro, příkazový terminál, nízkoúrovňové komponenty *systemd*, základní utility pro správu souborů, *sudo* pro možnost vykonání administrátorských úkon
- Vybrání zavaděče systému
 - Zvolil jsem *systemd-boot*, protože je již součástí nezbytné instalace
- Vytvoření uživatelů systému
 - Výchozí účet *root* – povinný administrátorský účet s nejvyšším oprávněním.
 - Privilegovaný účet *user* – možnost vykonávat administrátorské úlohy pomocí *sudo* aplikace

Poté lze počítač restartovat a je k dispozici spustitelný minimální linuxový systém, který bude sloužit jako základ softwarové architektury centrální jednotky chytré domácnosti.

6.1.2 Nezbytná konfigurace operačního systému a základní software

Anarchy Installer již provedl část nezbytné minimální konfigurace. Ještě je ovšem potřeba další ruční systémové nastavení.

Připojení systému do domácí sítě (LAN) a sítě Internet

Aby bylo možné instalovat další software, provádět aktualizace, komunikovat s dalšími zařízeními v místní síti, je potřeba nastavit síťové připojení. Konfigurace je provedena

integrovaným softwarem *systemd-networkd*, který umožňuje jednoduchou správu síťových zařízení. Pomocí příkazu *ip link show* je zjištěn název síťové karty.

Výpis 6.1 Zjištění názvu síťové karty

```
[user@SERVER]: ~>$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT
group default qlen 1000
    link/ether 68:05:ca:23:2e:52 brd ff:ff:ff:ff:ff:ff
```

Konfigurace *systemd-networkd* se nachází ve složce */etc/systemd/network*. Je vytvořen soubor dle názvu síťové karty s příponou *.network*. Do souboru je zanesena konfigurace síťové karty. Je vybrána statická IP adresa v domácí síti LAN.

Výpis 6.2 Soubor s konfigurací síťové karty

Soubor */etc/systemd/network/enp1s0.network*

```
[Match]
Name=enp1s0

[Network]
Address=10.0.0.1/16
Gateway=10.0.0.10
```

Do souboru */etc/resolv.conf* se vloží DNS servery, které má operační systém využívat pro překlad doménových jmen. V konfiguraci byly použité veřejné servery Cloudflare a Google.

Výpis 6.3 Soubor s nastavením DNS serverů

Soubor */etc/resolv.conf*

```
nameserver 1.1.1.1
nameserver 8.8.8.8
```

Správcem procesů je v Arch Linux *systemd*. Příkazem *systemctl enable* se povolí zapnutí služby při každém startu systému a přepínač *--now* říká, aby se nečekalo na další start systému a služba se zapnula ihned.

```
[user@SERVER]: ~>$ systemctl enable --now systemd-networkd.service
```

Testovací scénář (viz 7.1.1) poté ověří, zda je systém připojen k síti Internet.

Synchronizace systémového času

Korektní čas je potřeba pro přesné automatizace v chytré domácnosti a další bezchybné fungování systému. Je nastavena integrovaná komponenta *systemd-timesyncd*, která bude automaticky provádět periodickou synchronizaci s časovými servery. V souboru */etc/systemd/timesyncd.conf* je uložena konfigurace časových serverů.

Výpis 6.4 Soubor konfigurace časových serverů

```
Soubor /etc/systemd/timesyncd.conf

[Time]
NTP=0.cz.pool.ntp.org 1.cz.pool.ntp.org
```

Poté již lze zapnout synchronizaci systémových hodin. Pomocí příkazu *timedatectl status* lze také ihned ověřit, zda jsou hodiny synchronizovány.

Výpis 6.5 Zapnutí a kontrola synchronizace systémových hodin

```
[user@SERVER]: ~>$ sudo timedatectl set-ntp true
[user@SERVER]: ~>$ timedatectl status
          Local time: Mon 2020-11-16 21:20:12 CET
          Universal time: Mon 2020-11-16 20:20:12 UTC
             RTC time: Mon 2020-11-16 20:20:12
          Time zone: Europe/Prague (CET, +0100)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

Zrychlení a automatická aktualizace zavaděče systému

Zavaděč systému *systemd-boot*, který se povolil v průběhu instalace systému, má ve výchozím nastavení delší časomíru, která čeká na vstup uživatele při restartu systému. Tento odpočet je pro účely tohoto serveru zbytečný. Bude snížen z 5 s na 0 s. Tímto se docílí rychlejšího restartu serveru při údržbě a menší výpadek chytré domácnosti. Konfigurace zavaděče se nachází v souboru */boot/loader/loader.conf*.

Výpis 6.6 Konfigurace zavaděče systému

```
Soubor /boot/loader/loader.conf

default arch
timeout 0
```

Pokud dojde k aktualizaci balíčku *systemd-boot*, je nutné, aby se vygenerovala nová verze zavaděče operačního systému. Použije se *pacman hook*, což je funkce správce balíčků operačního systému Arch Linux. Tato funkce umí spustit automaticky jakýkoliv program,

pokud dojde k definované události s balíčkem. Následující skript se spustí automaticky při aktualizaci balíčku *systemd*, který obsahuje komponentu *systemd-boot*. Skript poté pustí program *bootctl update*, jež vygeneruje nový zavaděč systému. Skript stačí umístit do složky */etc/pacman.d/hooks/*.

Výpis 6.7 Skript pro vygenerování nové verze zavaděče

```
Soubor /etc/pacman.d/hooks/systemd-boot.hook
```

```
[Trigger]
Type = Package
Operation = Upgrade
Target = systemd

[Action]
Description = Updating systemd-boot...
When = PostTransaction
Exec = /usr/bin/bootctl update
```

6.2 Modulární software

Veškerý nezákladní software je řešený přes Docker kontejnery.

6.2.1 Instalace a spuštění Docker platformy

Instalace Dockeru v Arch Linuxu je velice rychlá. Stačí pouze nainstalovat balíček *docker* z repozitáře. Po instalaci je potřeba Docker službu spustit. Jelikož vytváření a správa kontejnerů vyžaduje vyšší oprávnění, přidá se standardní uživatel systému *user*, který byl vytvořen při instalaci systému, do uživatelské skupiny *docker*. Tímto bude možné spustit Docker příkazy bez potřeby povýšení na administrátorská oprávnění příkazem *sudo*. Po úpravě uživatelských skupin je nutné provést odhlášení a přihlášení k terminálovému sezení, aby se změny projevíly.

Výpis 6.8 Instalace a spuštění Dockeru

```
[user@SERVER]: ~ $ sudo pacman -S docker
[user@SERVER]: ~ $ sudo systemctl enable --now docker
[user@SERVER]: ~ $ sudo usermod -a -G docker user
```

Správnost konfigurace lze rychle ověřit příkazem *docker ps*, který vypisuje aktuální spuštěné kontejnery. I když není žádný kontejner spuštěn, pokud nedojde k chybě, Docker služba běží a uživatel má dostatečná práva.

```
[user@SERVER]: ~ $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
--------------	-------	---------	---------	--------

6.2.2 Konfigurace Dockeru

Před spuštěním prvního kontejneru byla provedena minimální konfigurace síťování v Dockeru. Vytvoří se uživatelská síť typu *bridge*, do které budou připojeny všechny kontejnery nevyžadující přímý přístup do domácí sítě LAN (viz 4.4.1).

Výpis 6.9 Vytvoření Docker sítě typu bridge

```
[user@SERVER]: ~ $ docker network create \
--driver=bridge \
--subnet=172.20.0.0/16 \
--ip-range=172.20.10.0/24 \
--gateway=172.20.10.254 \
user
```

Operační systém bude přístupný kontejnerům v této bridge síti na adrese 172.20.10.254.

6.2.3 Nastavení struktury složek

- Pro ukládání persistentních dat, která mají být zachována přes životní cyklus kontejneru, je pro každou aplikaci vytvořena jedna složka s názvem *storage*, do které budou ukládána všechna její data.
- Dále je vytvořena složka *scripts*, do které se umístí skripty pro spuštění kontejnerů.
- Složka *servicescripts* obsahuje pomocné skripty pro údržbu kontejnerů.
- Složka *builds* obsahuje definice vlastních kontejnerových obrazů. Obě složky jsou umístěny do složky *Docker*, která se nachází v uživatelské domovské složce.

Výpis 6.10 Struktura složek pro Docker kontejnery

Domovská složka /home/user/

```
|-- docker
  |-- builds
    |-- cc2538-bsl
      |-- build.sh
      |-- Dockerfile
      |-- firmware-update.sh
  |-- scripts
    |-- cc2538-bsl.sh
    |-- grafana.sh
    |-- homeassistant.sh
    |-- influxdb.sh
    |-- letsencrypt.sh
    |-- mosquitto.sh
    |-- zigbee2mqtt.sh
    |-- zwave2mqtt.sh
```

```
|-- storage
    |-- grafana
    |-- homeassistant
    |-- influxdb
    |-- letsencrypt
    |-- mosquitto
    |-- zigbee2mqtt
    |-- zwave2mqtt
|-- servicecripts
    |-- addmqttuser.sh
    |-- influxdb_initialize.sh
    |-- updatecontainer.sh
```

6.3 Automatizace chytré domácnosti

Home Assistant automaticky vygeneruje veškeré výchozí konfigurační soubory, pokud nejsou připraveny. Lze rovnou přejít ke spuštění kontejneru.

Vytvoření a spuštění kontejneru

Následující skript se umístí do připravené složky *scripts*.

Výpis 6.11 Skript pro spuštění Home Assistant kontejneru

Soubor /home/user/docker/scripts/homeassistant.sh

```
#!/bin/bash
```

```
docker run -d \
    --name="homeassistant" \
    -v /home/user/docker/storage/homeassistant:/config \
    -v /etc/localtime:/etc/localtime:ro \
    -e TZ=Europe/Prague \
    --network host \
    --restart unless-stopped \
    homeassistant/home-assistant
```

- *docker run* je příkaz pro spuštění kontejneru
- *-d* pustí kontejner na pozadí (detached)
- *--name* umožní přiřadit ke kontejneru název, díky kterému lze na kontejner odkazovat v dalších Docker příkazech
- *-v* slouží k namapování persistentních složek z operačního systému do kontejneru
- *-e* umožňuje nastavit proměnné běhového prostředí
 - *TZ* nastavuje časové pásmo
- *--network host* spouští kontejner v síťovém režimu hosta (viz 4.4.1)

- *--restart unless-stopped* říká, aby byl kontejner vždy restartován, pokud není zastaven uživatelem, například při výskytu chyby
- *homeassistant/home-assistant* je název kontejnerového obrazu na DockerHub

Po vytvoření souboru je potřeba upravit práva souboru a označit ho jako spustitelný. Poté lze skript spustit.

```
user@SERVER: ~/docker/scripts $ chmod +x homeassistant.sh
user@SERVER: ~/docker/scripts $ ./homeassistant.sh
```

Docker automaticky vytvoří složku *homeassistant* ve složce */home/user/docker/storage*. Poté dojde k prvnímu spuštění Home Assistant, který vytvoří výchozí konfigurační soubory v persistentní složce.

Výpis 6.12 Výchozí konfigurační soubory Home Assistant

```
user@SERVER: ~/docker/storage/homeassistant $ ls
automations.yaml  groups.yaml      scenes.yaml  tts/
configuration.yaml  home-assistant.log  scripts.yaml
deps/              home-assistant_v2.db  secrets.yaml
```

Zda došlo k úspěšnému spuštění aplikace Home Assistant, lze ověřit pomocí příkazu *docker logs -f homeassistant*. Příkaz *docker logs* zobrazí textový výstup, který aplikace posílá do standardního linuxového výstupu.

Výpis 6.13 Kontrola nastartovaného kontejneru Home Assistant

```
user@SERVER: ~/docker/scripts $ docker logs -f homeassistant
[fix-attrs.d] applying ownership & permissions fixes...
[fix-attrs.d] done.
[cont-init.d] executing container initialization scripts...
[cont-init.d] udev.sh: executing...
starting version 3.2.9
[15:29:52] INFO: Update udev information
[cont-init.d] udev.sh: exited 0.
[cont-init.d] done.
[services.d] starting services
[services.d] done.
Unable to find configuration. Creating default one in /config
```

Prvotní konfigurace aplikace Home Assistant

Aplikace je nyní dostupná na webovém rozhraní na portu 8123 a IP adrese serveru. Tedy příklad URL je: `http://10.0.0.1:8123`. Při prvním spuštění Home Assistant spustí průvodce, kde dojde ve třech krocích k prvotní konfiguraci:

1. Vytvoření uživatelského účtu
 - Název účtu
 - Přihlašovací jméno
 - Heslo
2. Vybrání aktuální polohy domácnosti a časového pásma. Poloha je využívána pro automatizace, které souvisejí s polohou slunce (západ, východ). Časové pásmo bude aplikace využívat pro zobrazení správného formátu času
 - Název domácnosti
 - Zeměpisná poloha domácnosti
 - Časová zóna – Europe/Prague
 - Nadmořská výška domácnosti
 - Jednotky vzdálenosti
3. Zapnutí automaticky nalezených integrací
 - Tato volba je přeskočena

Na přiloženém obrázku jsou zachyceny vyplněné údaje ve třech krocích konfiguračního průvodce.

The image displays three sequential screenshots of the Home Assistant configuration wizard. The first screenshot shows the 'Create Account' step with fields for Name (Michal), Username (domov), Password, and Confirm Password, followed by a 'CREATE ACCOUNT' button. The second screenshot shows the 'Name of your Home Assistant installation' (Domov) and 'Location' selection. It includes a map of Prague, Czechia, with a pin at 'Praha', and fields for Time Zone (Europe/Prague), Elevation (200 meters), and Unit System (Metric). The third screenshot shows the 'Integrations' screen with a list of detected devices and services: Google Cast, HomeKit, MiDeskLamp1, Printer: Brother DCP-9020CDW, and UPNP. A 'FINISH' button is visible at the bottom right.

Obr. 11 Konfigurační průvodce Home Assistant

Základní konfigurace Home Assistant je nyní dokončena. Následuje implementace protokolů do Home Assistant.

6.4 Jednotlivé protokoly chytré domácnosti

6.4.1 Integrace MQTT

Vytvoření konfiguračních souborů pro Eclipse Mosquitto

Eclipse Mosquitto automaticky nevygeneruje výchozí konfigurační soubory. Před spuštěním kontejneru se tedy ručně vytvoří složka s konfiguračním souborem. Poté je vytvořen prázdný soubor *mosquitto.passwd*, do kterého v dalším kroku dojde k uložení zašifrovaného uživatelského účtu. Nakonec se nastaví správné oprávnění souborů, protože aplikace uvnitř kontejneru běží pod uživatelským a skupinovým identifikátorem 1883 (linuxové UID a GID).

Výpis 6.14 Konfigurační soubor pro Eclipse Mosquitto aplikaci

```
user@SERVER: ~ $ mkdir -p /home/user/docker/storage/mosquitto/config
```

Soubor /home/user/docker/storage/mosquitto/config/mosquitto.conf

```
user mosquitto
max_inflight_messages 40
max_queued_messages 200
persistent_client_expiration 3m
autosave_interval 60
autosave_on_changes false

# Ukládání posledních zpráv na disk
persistence true
persistence_file mosquitto.db
persistence_location /mosquitto/data/

# Výchozí porty aplikace
listener 1883

listener 9001
protocol websockets

# Nastavení výpisu logů
log_dest stderr
log_type all
connection_messages true
log_timestamp true
```

```
# Nastavení autentizace
allow_anonymous false
password_file /mosquitto/config/mosquitto.passwd
```

```
user@SERVER: ~/ $ touch /home/user/docker/storage/mosquitto/config/mosquitto.passwd
user@SERVER: ~/ $ sudo chown -R 1883:1883 /home/user/docker/storage/mosquitto
```

Nastavení zabezpečení

Uživatelské účty se vytvářejí pomocí doprovodného nástroje, který je součástí Eclipse Mosquitto aplikace. Nástroj zašifruje zadané uživatelské jméno a heslo a uloží ho do souboru *mosquitto.passwd*. Je implementován servisní skript, který vytvoří dočasný kontejner z obrazu Eclipse Mosquitto, namapuje persistentní soubor s heslem a umožní uživateli vytvořit nový účet. Poté se kontejner zastaví a vymaže.

Výpis 6.15 Skript pro vytvoření nového MQTT uživatele dočasným kontejnerem

```
Soubor /home/user/docker/servicescripts/addmqttuser.sh

#!/bin/bash

read -p "New username: " userName
echo " "
if [[ -z "${userName// }" ]]; then
    echo "Exiting..."
    exit 1
fi

docker run -it --rm \
-v /home/user/docker/storage/mosquitto/config:/mosquitto/config \
eclipse-mosquitto \
mosquitto_passwd -c /mosquitto/config/mosquitto.passwd "$userName"
```

Přepínač *docker run -it* říká, aby došlo k vytvoření virtuálního terminálu (*-t*) a interaktivního režimu (*-i*). Tímto může aplikace uvnitř kontejneru zachytávat vstup uživatele.

Spuštění kontejneru

Výpis 6.16 Skript pro spuštění MQTT kontejneru

```
Soubor /home/user/docker/scripts/mosquitto.sh

#!/bin/bash

docker run -d \
  --name="mosquitto" \
  -v /home/user/docker/storage/mosquitto/config:/mosquitto/config \
  -v /home/user/docker/storage/mosquitto/data:/mosquitto/data \
  -v /etc/localtime:/etc/localtime:ro \
  -e TZ=Europe/Prague \
  --network user \
  -p 1883:1883 \
  --restart unless-stopped \
  eclipse-mosquitto
```

-p Přepínač slouží k namapování portů z kontejneru do domácí sítě LAN

Výpis 6.17 Kontrola nastartovaného kontejneru MQTT

```
user@SERVER: ~ $ docker logs -f mosquitto
mosquitto version 1.6.12 starting
Config loaded from /mosquitto/config/mosquitto.conf.
Opening ipv4 listen socket on port 1883.
Opening ipv6 listen socket on port 1883.
Opening websockets listen socket on port 9001.
mosquitto version 1.6.12 running
```

Napojení MQTT na Home Assistant

Pro připojení Home Assistant k MQTT serveru stačí provést nastavení nové integrace pomocí průvodce. Kontejner Home Assistant běží v Docker síťovém režimu *host*, proto nelze využít jméno MQTT Docker kontejneru, ale připojí se pomocí otevřeného portu na straně hosta.

1. Hlavní menu Home Assistant -> Konfigurace -> Integrace -> Tlačítko +
2. Ve vyhledávacím poli napsat *MQTT*
3. V okně nastavení připojení zvolit následující parametry:
 - a. Broker – 127.0.0.1
 - b. Port – 1883
 - c. Uživatelský účet s heslem, který byl vytvořen v Nastavení zabezpečení
 - d. Povolit Home Assistant Discovery (viz 5.2.4)

Set up a new integration

Search integrations

MQTT

MQTT

Please enter the connection information of your MQTT broker.

Broker

127.0.0.1

Port

1883

Username

supertajnyuzivatel

Password

.....

☒ Enable discovery

SUBMIT

Success!

Created configuration for 127.0.0.1.

FINISH

Obr. 12 Integrovaný průvodce MQTT v Home Assistant

6.4.2 Integrace Zigbee

Příprava USB Zigbee rádia v operačním systému

Vybrané USB rádio s čipem *Texas Instruments CC2652RB* se připojí do počítače. Pomocí příkazu `dmesg`, který vypíše poslední zprávy z linuxového kernelu, se zjistí ID výrobce a produktu, které se používá pro identifikaci zařízení v systému. Poté se vytvoří soubor, který přidá na zařízení stálého symbolického zástupce `/dev/zigbeestick`, jelikož Docker neumí namapovat zařízení, která mají v názvu nestandardní znaky (viz Výpis 6.18). Po vytvoření *udev rule* souboru stačí zařízení odpojit a připojit do systému a dojde k vytvoření zástupce. Případně je možné vynutit znovunačtení těchto pravidel bez odpojení zařízení příkazem `sudo udevadm trigger`.

Výpis 6.18 Příklad názvu zařízení, které nelze namapovat do Docker kontejneru

```
user@SERVER: ~ $ ls /dev/serial/by-id/  
/dev/serial/by-id/usb-Silicon_Labs_http\:\_slaesh_cc2652_  
_slaesh_s_iot_stuff_00\:\12\:\4B\:\00\:\21\:\A8\:\EB\:\55-if00-port0
```

Výpis 6.19 Vytvoření stálého symbolického zástupce na USB Zigbee rádio

```
user@SERVER: ~ $ dmesg  
usb 1-2: New USB device found, idVendor=10c4, idProduct=ea60, bcdDevice= 1.00  
usb 1-2: New USB device strings: Mfr=1, Product=2, SerialNumber=3  
usb 1-2: Product: XDS110 (03.00.00.07) Embed with CMSIS-DAP  
usb 1-2: Manufacturer: Texas Instruments  
usb 1-2: SerialNumber: L4XXXXXX
```

Soubor /etc/udev/rules.d/50-myusb.rules

```
# USB TI CC2652RB symlink  
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE="0666",  
SYMLINK+="zigbeestick"
```

Flash firmware na USB Zigbee rádio

Byl vytvořen speciální kontejnerový obraz, který připraví běhové prostředí pro open-source nástroj **cc2538-bsl**, který slouží k nahrání Zigbee koordinátor firmwaru do USB rádií s Texas Instruments čipy. Postup implementace vlastnoručně vytvořeného kontejnerového obrazu je dostupný v sekci s přílohami (viz Příloha C:).

Po vytvoření kontejnerového obrazu již stačí vytvořit dočasný jednorázový kontejner, který spustí aplikaci cc2538-bsl a po ukončení aplikace se vymaže. Při této operaci nesmí být spuštěný běžící Zigbee2MQTT kontejner nebo jiná aplikace, která by pracovala se stejným zařízením.

Výpis 6.20 Skript pro vytvoření dočasného kontejneru pro nahrání Zigbee firmwaru do USB rádia

Soubor /home/user/docker/servicescripts/zigbeeupdate.sh

```
#!/bin/bash  
  
docker run -it --rm \  
-v /dev/zigbeestick:/dev/zigbeestick \  
--privileged \  
cc2538-bsl
```

Ukázkový výstup úspěšného flashování pomocí vytvořeného skriptu je dostupný v příloze (viz Výpis 7.14).

Vytvoření konfiguračních souborů pro Zigbee2MQTT

Před spuštěním kontejneru pro úspěšnou inicializaci se vytvoří hlavní konfigurační soubor. Zigbee2MQTT používá formát souboru *YAML*. Zde je důležité správné používání mezer (každá podsekce přidává dvě mezery na řádek). Dojde k definici připojení k MQTT serveru, zapnutí Home Assistant MQTT Discovery, jaké zařízení se má použít na roli koordinátora a na jakém kanálu bude Zigbee síť běžet. Následně je povolené integrované webové rozhraní, jež umožňuje jednoduchou správu a přidávání nových zařízení. Změna oprávnění souborů není u Zigbee2MQTT potřebná, jelikož aplikace běží v kontejneru pod administrátorským *root* účtem.

Výpis 6.21 Konfigurační soubor pro aplikaci Zigbee2MQTT

Soubor /home/user/docker/storage/zigbee2mqtt/configuration.yaml

```
# Zapnutí Home Assistant MQTT Discovery
homeassistant: true

# Zakázání otevření sítě pro nová zařízení
permit_join: false

# Nastavení MQTT připojení
mqtt:
  # Téma pro publikaci zpráv
  base_topic: zigbee2mqtt
  # Adresa serveru
  server: 'mqtt://mosquitto:1883'
  # Uživatelský účet
  user: supertajnyuzivatel
  password: heslo

# Nastavení USB Zigbee rádia
serial:
  # Cesta k zařízení
  port: /dev/zigbeestick

# Zigbee kanál
advanced:
  channel: 25

# Povolení webového rozhraní
frontend:
  port: 8124
experimental:
  new_api: true
```

Spuštění kontejneru

Výpis 6.22 Skript pro spuštění Zigbee2MQTT kontejneru

```
Soubor /home/user/docker/scripts/zigbee2mqtt.sh

#!/bin/bash

docker run -d \
  --name zigbee2mqtt \
  -v /home/user/docker/storage/zigbee2mqtt:/app/data \
  -v /etc/localtime:/etc/localtime:ro \
  -e TZ=Europe/Prague \
  --network user \
  -p 8124:8124 \
  --device="/dev/zigbeestick" \
  --restart unless-stopped \
  koenkk/zigbee2mqtt
```

Pomocí výpisu z aplikace lze rychle ověřit, zda došlo k úspěšnému připojení k MQTT serveru a správné inicializaci USB Zigbee adaptéru. Kontejner nyní běží a je připraven na připojení Zigbee zařízení chytré domácnosti. Správu zařízení lze provádět přes webové rozhraní na portu 8124. Tedy například <http://10.0.0.1:8124>.

Výpis 6.23 Kontrola nastartovaného kontejneru Zigbee2MQTT

```
user@SERVER: ~ $ docker logs -f zigbee2mqtt

Starting Zigbee2MQTT version 1.16.1 (commit #6b32f30)
Starting zigbee-herdsman...
zigbee-herdsman started
Coordinator                               firmware                               version:
'{"meta":{"maintrel":1,"majorrel":2,"minorrel":7,"product":1,"revision":20201026,"transportrev":2},"type":"zStack3x0"}'
Currently 0 devices are joined:
Zigbee: disabling joining new devices.
Connecting to MQTT server at mqtt://mosquitto
Connected to MQTT server
MQTT publish: topic 'zigbee2mqtt/bridge/state', payload 'online'
```

6.4.3 Integrace Z-Wave

Příprava USB Z-Wave rádía v operačním systému

Vybrané Z-Wave USB rádio se dodává již s plně kompatibilním firmwarem koordinátora. Není třeba žádného nahrávání firmwaru. Pouze se jako u Zigbee (viz 6.4.2) provede

vytvoření stálého symbolického zástupce pomocí *udev rule* systému. Do již vytvořeného souboru, který obsahuje definici pro Zigbee USB rádio, lze na další řádek přidat další pravidlo.

Výpis 6.24 Vytvoření stálého symbolického zástupce na zařízení USB Z-Wave rádio

Soubor /etc/udev/rules.d/50-myusb.rules

```
# Aeotec Z-Wave USB Stick symlink
SUBSYSTEM=="tty", ATTRS{idVendor}=="0658", ATTRS{idProduct}=="0200", MODE="0666",
SYMLINK+="aeotec"
```

Konfigurace Zwave2MQTT

Aplikace zvládne první spuštění i bez existujícího konfiguračního souboru. Poté lze konfiguraci provést přes webové rozhraní. Je ale možné vytvořit konfigurační soubor před prvním spuštěním a aplikace ho korektně načte. Konfigurace probíhá ve formátu *JSON*.

Výpis 6.25 Konfigurační soubor pro aplikaci Zwave2MQTT

Soubor /home/user/docker/storage/zwave2mqtt/settings.json

```
{
  "mqtt": {
    "name": "zwave2mqtt",
    "host": "mosquitto",
    "port": 1883,
    "qos": 0,
    "prefix": "zwave2mqtt",
    "reconnectPeriod": 60000,
    "retain": true,
    "auth": true,
    "username": "supertajnyuzivatel",
    "password": "tajneheslo"
  },
  "gateway": {
    "values": [],
    "type": 0,
    "payloadType": 1,
    "nodeNames": true,
    "hassDiscovery": true,
    "discoveryPrefix": "homeassistant"
  },
  "zwave": {
    "port": "/dev/zwavestick",
    "assumeAwake": true,
```

```

    "saveConfig": true,
    "autoUpdateConfig": true,
    "logging": true,
    "pollInterval": 30000,
    "commandsTimeout": 200
  }
}

```

Spuštění kontejneru

Výpis 6.26 Skript pro spuštění Zwave2MQTT kontejneru

```

Soubor /home/user/docker/scripts/zwave2mqtt.sh

#!/bin/bash

docker run -d \
  --name zwave2mqtt \
  -v /home/user/docker/storage/zwave2mqtt/:/usr/src/app/store \
  -v /etc/localtime:/etc/localtime:ro \
  -e TZ=Europe/Prague \
  --network user \
  -p 8125:8091 \
  --device=/dev/aeotec \
  --restart unless-stopped \
  robertslando/zwave2mqtt

```

Správa Z-Wave sítě je nyní dostupná přes webové rozhraní na portu 8125. Příklad URL: *http://10.0.0.1:8125*.

6.4.4 Integrace RF 433 MHz

Příprava zařízení Sonoff RF Bridge

Zařízení Sonoff RF Bridge bylo otevřeno čtyřmi šroubky, které jsou umístěné na spodní straně krytu pod gumovými podložkami. Celé zařízení je na jedné desce, která byla vyjmuta (viz Obr. 13).



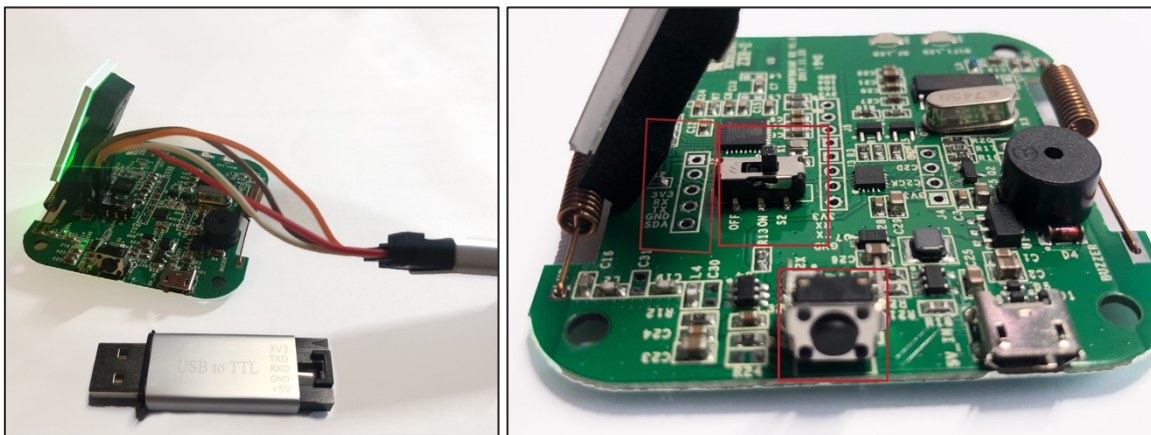
Obr. 13 Zařízení Sonoff RF Bridge

Flashování firmware OpenMQTTGateway

Flashování bylo provedeno na počítači s macOS. Pro připojení Sonoff RF Bridge byl použitý USB převodník na sériový port (USB to TTL). Konkrétně zařízení s čipem Silicon Labs CP2102.

- Do systému byl nainstalován ovladač ze stránek výrobce čipu [61]
- Následně byl nainstalován open-source nástroj esptool, napsaný v jazyce Python, sloužící pro čtení a nahrávání firmwaru do mikrodesek s čipy ESP8266 a ESP32.
- Do Sonoff RF Bridge byly zapojeny piny z USB převodníku
 - Piny nebylo třeba pájet, kontakt byl dostatečný
 - Dle obrázku (viz Obr. 14) se jedná o červeně ohraničený čtverec vlevo. Zapojeno seshora dolů. Verze desky Sonoff RF Bridge V2 1.0 má chybně natištěné popisky pinů RX a TX:
 - 3V3 – 3V3
 - RX na Sonoff RF Bridge – TX na USB převodníku
 - TX na Sonoff RF Bridge – RX na USB převodníku
 - GND – GND
- Přepínač, který zapíná komunikaci ESP8266 čipu s RF 433 MHz čipem, byl přepnutý do pozice OFF. V obrázku (viz Obr. 14) se nachází pozice OFF vlevo. Tím se uvolní datový tok pro sériovou konzoli na flashování
- Na Sonoff RF Bridge bylo přidrženo boční tlačítko a zároveň byl připojen USB převodník do počítače
 - Při flashování není do desky zapojeno napájení do micro-USB portu. Napájí se z USB převodníku na pinu 3V3
- Pomocí příkazu `read_flash` byla provedena záloha původního firmwaru Sonoff RF Bridge, který dodává výrobce, přečtením celé 1 MB flash paměti na desce
- Pomocí příkazu `erase_flash` se vyčistil celý obsah paměti

- Jedná se o bezpečnou operaci. Vždy lze zařízení přepnout do režimu flashování a nahrát nový firmware
- Příkazem `write_flash` byl nakonec nahrán firmware OpenMQTTGateway



Obr. 14 Deska Sonoff RF Bridge

Výpis 6.27 Proces zálohy a flashování firmwaru do Sonoff RF Bridge

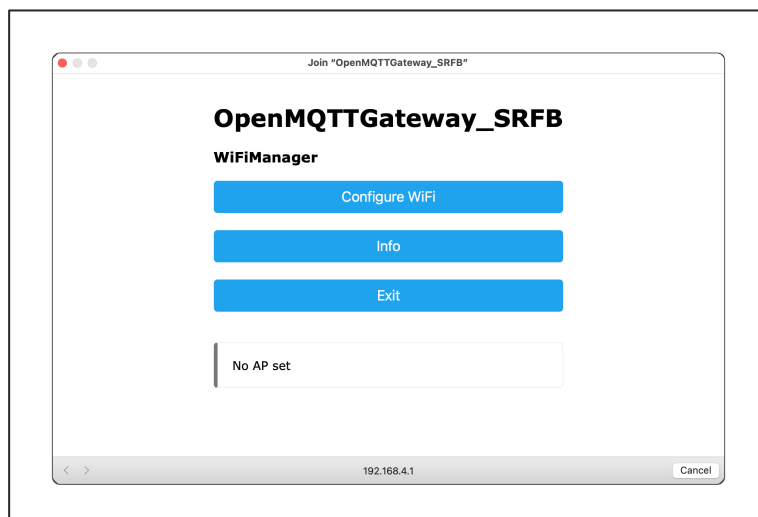
```
// Instalace nástroje esptool do virtuálního Python prostředí
michal@Michals-MBP ~ % python3 -m venv esptool
michal@Michals-MBP ~ % cd esptool
michal@Michals-MBP ~ % source bin/activate
michal@Michals-MBP ~ % pip install esptool

// Záloha flash paměti Sonoff RF Bridge
michal@Michals-MBP ~ % esptool.py --port /dev/ttyUSB0 read_flash 0x000000 0x100000 esp-1MB-backup.bin

// Vymazání a nahrání
michal@Michals-MBP ~ % esptool.py --port /dev/ttyUSB0 erase_flash
michal@Michals-MBP ~ % esptool.py --port /dev/ttyUSB0 write_flash --flash_size 1MB --flash_mode dout 0x000000 firmware.bin
```

Konfigurace OpenMQTTGateway

Po procesu flashování a prvním spuštění zařízení vytvoří dočasný Wi-Fi přístupový bod s názvem *OpenMQTTGateway_SRFB* a heslem *your_password*. Po připojení na Wi-Fi síť se zobrazí tzv. captive portal. Jedná se o webovou stránku, která se zobrazuje například u veřejných Wi-Fi sítí, kde před použitím musí uživatel odsouhlasit podmínky služby. Pokud se stránka sama nezobrazí, je dostupné webové rozhraní na výchozí URL s IP adresou: *http://192.168.4.1*. Zde byla provedena konfigurace Wi-Fi (k jaké Wi-Fi se má zařízení připojit) a MQTT připojení. Po uložení konfigurace je Sonoff RF Bridge restartován a při úspěšném připojení do nakonfigurované Wi-Fi sítě přestane vysílat dočasný přístupový bod. Nastavení lze změnit opětovným vymazáním paměti a nahráním firmwaru.

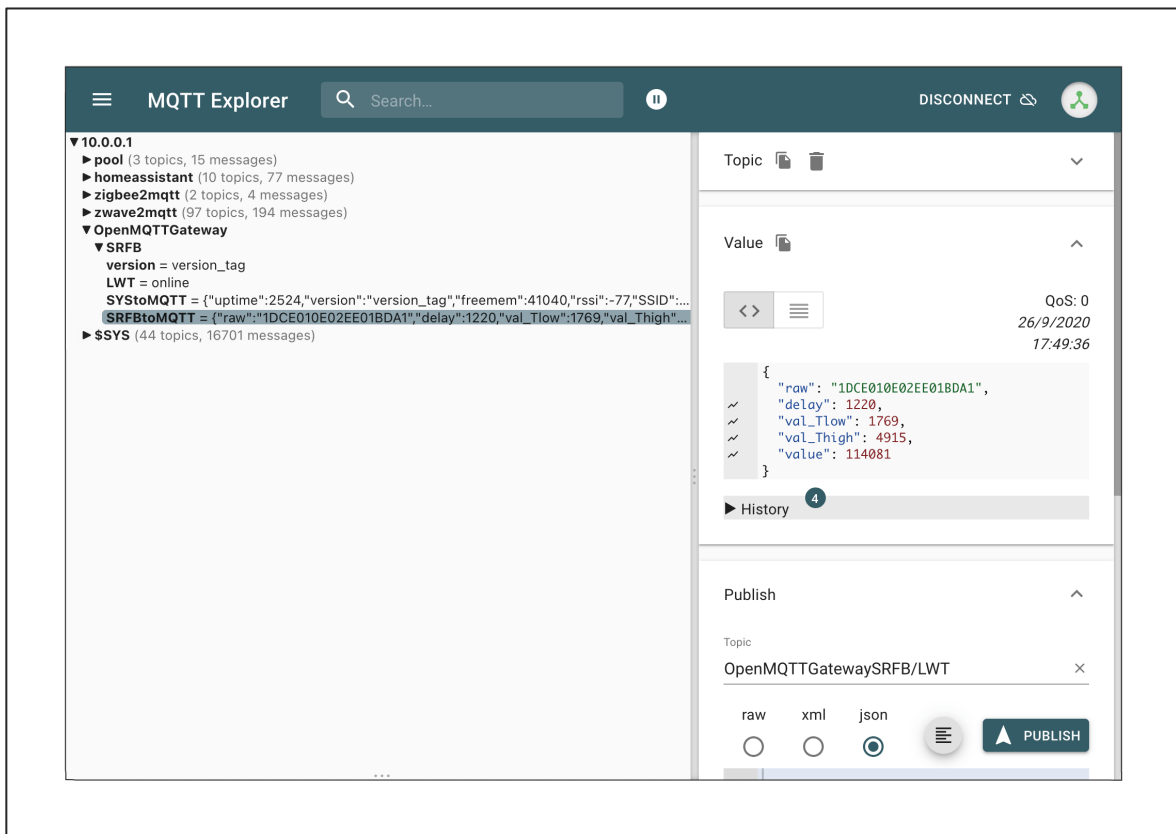


Obr. 15 Konfigurační webové rozhraní OpenMQTTGateway

V sekci Konfigurace Wi-Fi byly nastaveny následující údaje:

- SSID: Název Wi-Fi sítě
 - Lze vybrat ze seznamu nalezených Wi-Fi sítí
- Heslo k Wi-Fi
- MQTT Server – „10.0.0.1“
- MQTT Port – „1883“
- MQTT uživatel – „supertajnyuzivatel“
- MQTT heslo – „supertajneheslo“
- Gateway Name – „SRFB“
 - Jak se bude jmenovat MQTT téma, do kterého bude publikovat zařízení
- Base Topic – „OpenMQTTGateway“
 - Název nadřazeného MQTT tématu
- Kliknuto na „Uložit“

Po restartu bylo zkontrolováno, zda zařízení korektně vysílá do MQTT serveru a zda zachytává RF 433 MHz signály. Byl použitý software MQTT Explorer, který umožňuje grafický stromový pohled do MQTT serveru. Na obrázku (viz Obr. 16) lze vidět, že zprávy se posílají do definovaného tématu *OpenMQTTGateway/SRFB*.



Obr. 16 Zobrazení aktuálních zpráv na MQTT serveru v nástroji MQTT Explorer

Konfigurace Home Assistant

V Home Assistant je nutné ručně vytvořit pro každé zachycené zařízení definici. Na obrázku MQTT prohlížeče (viz Obr. 16) je zobrazeno, že RF signály jsou odeslány do tématu *OpenMQTTGateway/SRFB/SRFBtoMQTT* ve formátu JSON. Atribut *value* je ten, který zobrazuje ID přijímaného signálu, v tomto případě tlačítka domovního zvonku.

Ukázková definice zobrazuje implementaci MQTT binárního senzoru (zapnuto/vypnuto) v Home Assistant pro RF domovní zvonek zachycený přes OpenMQTTGateway.

Výpis 6.28 Ukázková implementace senzoru stisknutí RF domovního zvonku v Home Assistant

Soubor `/home/user/docker/storage/homeassistant/configuration.yaml`

```
binary_sensor:
  - platform: mqtt
    state_topic: 'OpenMQTTGateway/SRFB/SRFBtoMQTT'
    value_template: '{{ value_json.value }}'
    payload_on: '114081'
    off_delay: 5
    device_class: sound
    name: "Zvonek"
```

- *state_topic* – MQTT téma, ve kterém jsou publikována data zachycených signálů
- *value_template* – Definuje formát dat a v jakém atributu se nachází hodnota stavu zařízení
- *payload_on* – Protože MQTT nevysílá zprávu zapnuto/vypnuto, ale číselný kód signálu, je tímto definováno, jakou hodnotu lze považovat pro stav zapnuto
- *off_delay* – Zvonek nevysílá signál vypnutého stavu, jako například kontaktní dveřní senzory. Proto bylo nastaveno automatické přepnutí do stavu vypnuto po uběhnutí 5 s, čímž bylo zároveň zamezeno opětovnému vyvolávání automatizací, pokud někdo zmáčkne zvonek rychle za sebou
- *device_class* – Značí, jak má vypadat zařízení v grafickém rozhraní Home Assistant. Zde bylo zvoleno jako zvukové zařízení (alarm)

Druhá ukázková implementace zachycuje implementaci přepínače, který dokáže vyslat RF signál dveřního zvonku přes OpenMQTTGateway a zároveň dokáže zobrazit aktuální stav, který přebírá z výše definovaného binárního senzoru.

Výpis 6.29 Ukázková implementace tlačítka pro RF domovní zvonek v Home Assistant

```
switch:
- platform: template
  switches:
    zvonek_tlacitko:
      value_template: "{{ is_state('binary_sensor.zvonek', 'on') }}"
      turn_on:
        - service: mqtt.publish
          data:
            topic: 'OpenMQTTGateway/SRFB/commands/MQTTtoSRFB'
            payload: '{ value: 114081 }'
      turn_off:
        - delay: 0
      icon_template: 'mdi:doorbell'
```

- *value_template* – přebírá aktuální stav přepínače z binárního senzoru vytvořeného výše (aby se změnil stav Home Assistant tlačítka v případě, že bylo stisknuté fyzické tlačítko)
- *turn_off* – akce je prázdná, jelikož zvonek je momentální tlačítko

6.4.5 Integrace Wi-Fi chytrých zařízení

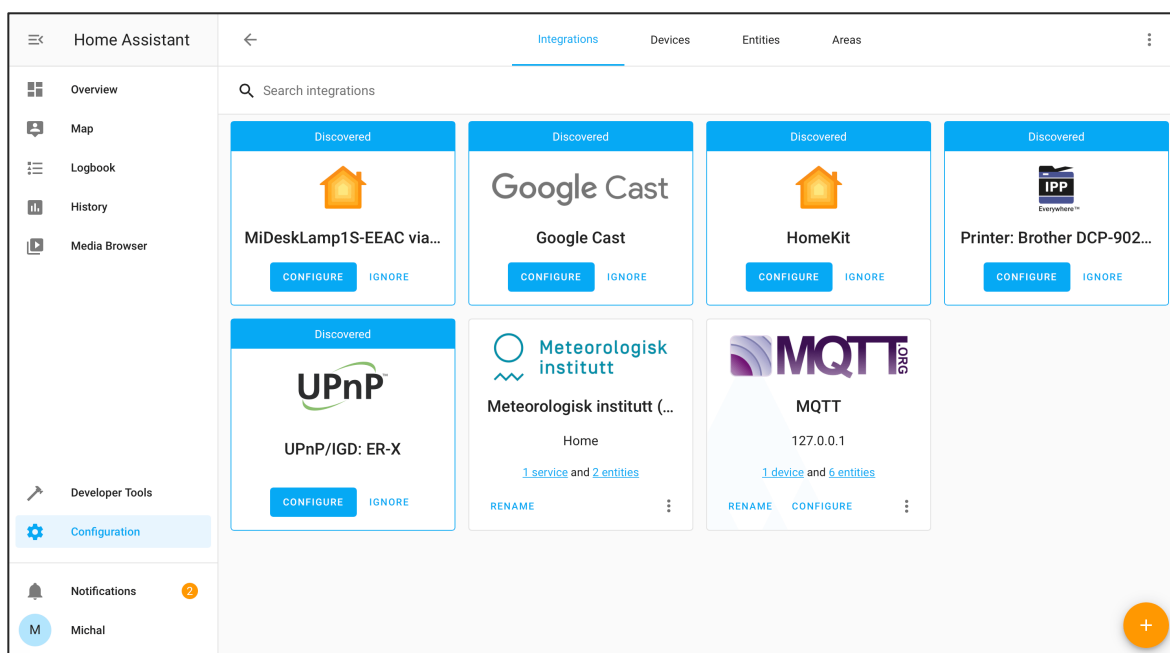
Jak bylo zmíněno v analýze (viz 4.5.6), zařízení připojující se přes Wi-Fi (TCP/IP vrstvu) mají různé formy API na integraci. Nelze implementovat přesný postup pro každé zařízení. Ovšem platforma Home Assistant obsahuje modulární architekturu, která umožňuje přidávání integrací pro všechna zařízení, která mají dostupné API. Na webových stránkách projektu je k dispozici seznam aktuálních integrací. [59] Původně se integrace aktivovaly

přes hlavní konfigurační soubor *configuration.yaml*. Nyní se všechny integrace přesouvají do grafického rozhraní, kde je lze nastavit. Pro zapnutí integrace stačí:

1. Otevřít nastavení Home Assistant
2. Přejít do sekce Integrace
3. Kliknout na tlačítko + (Přidat)
4. Vybrat integraci ze seznamu

Integrace HomeKit zařízení

Dle návrhu je použita implementace třídy HomeKit Controller dostupná přímo v Home Assistant. Součástí výchozího konfiguračního souboru je klíč *default_config*, který zapíná službu *zeroconf*, jež slouží pro automatické objevování HomeKit zařízení na domácí síti LAN. Kontejner Home Assistant byl spuštěn v Docker režimu *--net host*, díky kterému má stejný přístup do sítě, jako by byl nainstalován přímo do operačního systému.



Obr. 17 Zařízení Xiaomi Desk Lamp podporující HomeKit protokol rozpoznané v Home Assistant

6.4.6 Koexistence protokolů

Dle návrhu (viz 5.2.5) a předešlé analýzy se v implementovaném řešení nacházejí 2 bezdrátové protokoly, kde může dojít ke kolizi. Wi-Fi a Zigbee operují na stejném frekvenčním pásmu 2.4 GHz (viz 4.5.2).

Tabulka 1 Implementované bezdrátové protokoly a jejich frekvence

Protokol	Frekvenční pásmo
Wi-Fi	2,4 GHz a 5 GHz
Zigbee	2,4 GHz
Z-Wave	868 MHz
RF	433 MHz

Wi-Fi a Zigbee síť byly nastaveny následovně:

- Wi-Fi vysílače byly nastaveny na vysílání 2 sítí
 - SuperWifi
 - SuperWifi_2.4G
 - Nepoužitím stejného názvu Wi-Fi sítě se docílilo, že se koncová zařízení nemohou sama rozhodnout, k jaké síti se připojit
- Zařízení, která podporují připojení na Wi-Fi s 5 GHz pásmem, byla přepojena na 5 GHz síť
- Wi-Fi kanál 2.4 GHz sítě byl nastaven na 1
- Zigbee kanál byl nastaven na 25 (viz 6.4.2)

Dle návrhu byly USB adaptéry Zigbee a Z-Wave rádia individuálně napojeny na prodlužovací USB kabely o délce 1 m a umístěny od sebe s minimální vzdáleností 0,5 m. Dále byly zapojeny do USB 2.0 portů.

↑	NAME	IP ADDRESS	WIFI EXPERIENCE	WLAN	AP	CHANNEL	WIFI CONNECTION
	a0:6f:aa:58:4d:eb	10.0.0.214	99%	SuperWifi	IW-Pro Obyvák	36	11na (5 GHz)
	BRW28565A53C4F0	10.0.0.231	100%	SuperWifi_2.4G	UAP-nanoHD Patro	1	11ng (2.4 GHz)
	ESP_C0CF82	10.0.0.176	100%	SuperWifi_2.4G	UAP-nanoHD Patro	1	11ng (2.4 GHz)
	iPhone	10.0.0.171	60%	SuperWifi	IW-Pro Obyvák	36	11ac (5 GHz) 
	jbllink-409f38b9a9ad	10.0.0.191	99%	SuperWifi	IW-Pro Obyvák	36	11ac (5 GHz)
	jbllink-409f38e6a73d	10.0.0.205	99%	SuperWifi	IW-Pro Obyvák	36	11ac (5 GHz)
	Lenka-iPad	10.0.0.141	99%	SuperWifi	IW-Pro Obyvák	36	11ac (5 GHz) 
	Lenka-Air	10.0.0.236	98%	SuperWifi	IW-Pro Obyvák	36	11na (5 GHz) 

Obr. 18 Kontrola přepojení Wi-Fi kanálu a koncových zařízení

6.5 Zálohování

Příprava Google Drive účtu pro zálohování

Pro přístup na Google Drive pomocí API je třeba získat autentizační OAuth klíče. S Google účtem, na který bude nahrána každodenní záloha, je provedeno přihlášení do Google Cloud Platform administračního rozhraní [62]. Jsou provedeny následující kroky:

- Vytvoření nového projektu s názvem „chytry-domov“
- Povoleno Google Drive API
 - Menu APIs & Služby -> Knihovna -> Drive API -> Povolit
- Nastavení obrazovky, která se zobrazí při požádání o přístup k datům uživatele. I když tento projekt bude používán stejným uživatelem, jedná se o povinný krok
 - Menu APIs & Služby -> OAuth obrazovka
 - Typ uživatele -> Externí (aplikace bude sloužit pro soukromé použití)
 - První krok
 - Název aplikace – „Chytrá domácnost“
 - Email pro podporu uživatelů – vybrán email Google účtu
 - Kontaktní informace vývojáře – zadán email Google účtu
 - Kliknuto na „Uložit a pokračovat“
 - Druhý krok – Rozsah oprávnění
 - Definice potřebných oprávnění – již byly zapnuté při povolení Drive API. Tento krok lze přeskočit
 - Třetí krok – Testovací uživatelé
 - Přidat uživatele -> emaily Google účtů, které chtějí používat asistenta
- Získání autentizačních klíčů pro přístup k API
 - Menu APIs & Služby -> Pověření -> Vytvořit pověření -> OAuth Client
 - Typ aplikace – Desktopová aplikace
 - Název aplikace – rclone
 - Kliknuto na „Vytvořit“
 - Následně jsou vygenerovány kódy *Client ID* a *Client secret*
 - Kódy jsou zkopírovány do dočasného textového dokumentu

Instalace a konfigurace aplikace rclone

Rclone je aplikace, která nemá žádné závislosti. V návrhu řešení je brán jako základní software. Je instalován přímo do operačního systému. Poté je provedena konfigurace pomocí terminálového textového rozhraní příkazem *rclone config*. Během konfigurace je nastaveno připojení do cloudového úložiště Google Drive.

```
user@SERVER: ~ $ sudo pacman -S rclone
user@SERVER: ~ $ rclone config
```

Pro konfiguraci jsou provedeny následující kroky:

- Nový zdroj

- Jméno zdroje - „gdrive“
- Typ úložiště – Google Drive
- Client_id – okopírovaný kód z Google Console
- Client_secret – okopírovaný kód z Google Console
- Rozsah oprávnění – „drive“ (plný přístup pro čtení a zápis na Google Drive)
- Root_folder_id – nevyplněno (zálohy budou ukládány do kořenové složky)
- Service_account_file – nevyplněno
- Pokročilá konfigurace – Ne
- Použít automatické nastavení – Ne (potřebné při nastavování přes terminál)
- Dojde k vygenerování URL, která se otevře v prohlížeči
- Přihlásí se s Google účtem, na který budou ukládány zálohy
- Objeví se text: „Tato aplikace není ověřená“. Jedná se o soukromou aplikaci, která nebude publikována. Je kliknuto na „Pokračovat“
- Objeví se: „Udělte aplikaci Chytrá domácnost oprávnění“. Je kliknuto na „Povolit“
- Objeví se kód pro zkopírování do aplikace rclone
- Kód je vložen do terminálu a potvrzeno
- Konfigurace rclone je dokončena

Je provedena rychlá kontrola, zda došlo k úspěšné implementaci přístupu k Google Drive cloudovému úložišti. Je vytvořen textový soubor *test.txt* s náhodným textem uvnitř a je zkopírován pomocí *rclone* na Google Drive.

Výpis 6.30 Kontrola implementace rclone a Google Drive

```
user@SERVER: ~ $ rclone copy test.txt gdrive: -v
2020/09/21 17:41:41 INFO : test.txt: Copied (new)
2020/09/21 17:41:41 INFO :
Transferred:      5 / 5 Bytes, 100%, 3 Bytes/s, ETA 0s
Transferred:      1 / 1, 100%
Elapsed time:     2.6s
```

Nastavení denního zálohování

Balíček *systemd*, který se stará o správu služeb, je použitý k vytvoření definice služby, který provede vytvoření zálohy a nahrání do cloudového úložiště. Definice jsou ukládány do */etc/systemd/system/*.

Výpis 6.31 Definice systemd služby pro zálohování

Soubor */etc/systemd/system/backupsync.service*

```
[Unit]
Description=Synchronizace na GDrive

[Service]
Type=oneshot
```



```

User=root
ExecStart=tar -cjf /home/user/smarthomebackup.tar.bz2 \
--exclude=**/home-assistant_v2.db --exclude=**/letsencrypt/log/* \
/home/user/docker \
/etc/systemd/system/backupsync.service /etc/systemd/system/backupsync.timer \
/etc/systemd/network/enp1s0.network /etc/resolv.conf /etc/systemd/timesyncd.conf \
/boot/loader/loader.conf /etc/pacman.d/hooks/systemd-boot.hook \
/etc/udev/rules.d/50-myusb.rules \
/etc/ssh/sshd_config /home/user/.ssh/authorized_keys \
/home/user/.config/rclone /home/user/.bashrc
ExecStart=/usr/sbin/rclone --config /home/user/.config/rclone/rclone.conf \
copy /home/user/spajzbackup.tar.bz2 mirror: -v

```

- *Type=oneshot* umožňuje spustit více příkazů *ExecStart* za sebou
- *Exclude* definuje soubory, které může záloha vynechat
 - Byla vynechána dočasná databáze Home Assistant (která obsahuje stavy všech zařízení za poslední týden) a logy kontejneru LetsEncrypt, které zabírají více místa
- Je nastaveno zálohování složky docker v uživatelské složce a všech upravených souborů v průběhu implementace

Poté je vytvořen systemd časovač, který spustí zálohovací službu každý den ve 3 hodiny ráno.

Výpis 6.32 Definice systemd časovače pro spuštění zálohování

```

Soubor /etc/systemd/system/backupsync.timer

[Unit]
Description=Každé 3 hodiny ráno spustí zálohu souborů

[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=True
Unit=backupsync.service

[Install]
WantedBy=multi-user.target

```

Následně je provedeno znovunačtení souborů se službami pomocí *systemctl daemon-reload* a je spuštěn časovač.

```

user@SERVER: ~ $ sudo systemctl daemon-reload
user@SERVER: ~ $ sudo systemctl enable backupsync.timer

```

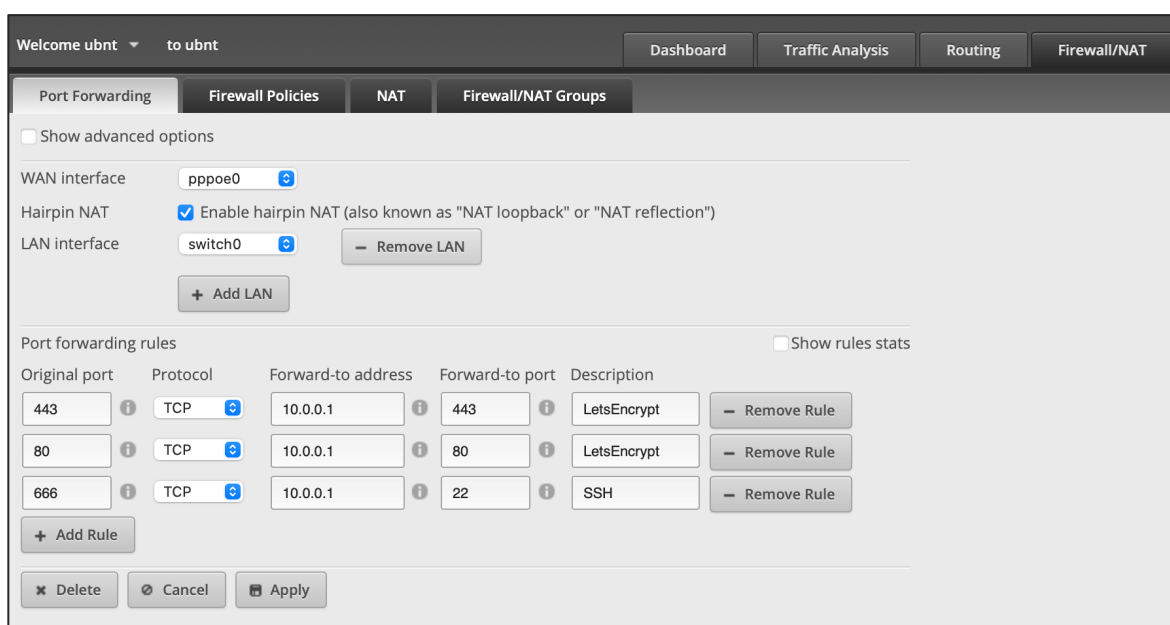
6.6 Vzdálený přístup

Povolení přístupu pro Hlasové asistenty a GUI Home Assistant pro přístup z webu. Konfigurace port forwardingu na routeru. Je potřeba veřejná IP adresa/statická nebo dynamická DNS služba.

Nastavení přesměrování portů na routeru/firewallu

V tomto kroku se povolí přístup ze sítě Internet na výchozích portech pro webové stránky (viz 4.8). Firewall a router je v tomto případě jedno zařízení EdgeRouter od Ubiquiti.

- Port 80 a 443 jsou povoleny ve Firewallu
- Port 80 a 443 jsou na routeru přesměrovány do domácí sítě LAN na IP adresu serveru chytré domácnosti (v tomto případě 10.0.0.1)



Obr. 19 Ukázka přesměrování portů na routeru s firewallem EdgeRouter od Ubiquity

Nastavení DNS záznamů domény

Po zakoupení domény byly nastaveny DNS záznamy.

- DNS A záznam (IPv4) subdomény *domov.tajnadomena.com* byl nasměrován na IP adresu internetového připojení chytré domácnosti
- Zástupný DNS A (IPv4) záznam **.domov.tajnadomena.com* přesměruje jakékoliv požadavky subdomény *domov.tajnadomena.com* na stejnou IP adresu
- TTL (Time-To-Live) byl ponechán na Auto u správce domény. V případě nutnosti lze nastavit na výchozích 3600 s

DNS management for tajnadomena.com				
+ Add record		Search DNS Records		Advanced
Type	Name	Content	TTL	Proxy status
A	*.domov	80.101.102.103	Auto	DNS only Edit ▶
A	domov	80.101.102.103	Auto	DNS only Edit ▶

Obr. 20 Ukázka nastavení DNS záznamů domény

Může trvat i několik hodin, než dojde k propagaci DNS záznamů napříč DNS servery. Pro kontrolu byl použitý příkaz *ping*, který slouží k otestování dostupnosti jakéhokoliv síťového zařízení.

Výpis 6.33 Kontrola propagace DNS záznamů

```
user@SERVER: ~ $ ping domov.tajnadomena.com
PING domov.tajnadomena.com (80.101.102.103) 56(84) bytes of data.
64 bytes from 80.broadband9.iol.cz (80.101.102.103): icmp_seq=1 ttl=64 time=4.73 ms
```

Spuštění kontejneru s Nginx a LetsEncrypt

Výpis 6.34 Skript pro spuštění Nginx+LetsEncrypt kontejneru

```
Soubor /home/user/docker/letsencrypt.sh

#!/bin/bash

docker run -d \
  --cap-add=NET_ADMIN \
  --name=letsencrypt \
  -v /home/user/docker/storage/letsencrypt:/config \
  -v /etc/localtime:/etc/localtime:ro \
  -e TZ=Europe/Prague \
  -e EMAIL=supertajnyuzivatel@gmail.com \
  -e URL=domov.tajnadomena.com \
  -e SUBDOMAINS=grafana \
  -e VALIDATION=http \
  -e PUID=1000 \
  -e PGID=985 \
  --network user \
  -p 443:443 \
  -p 80:80 \
  --add-host docker.host:172.20.10.254 \
  --restart unless-stopped \
  linuxserver/swag
```

- `--cap-add=NET_ADMIN` Jelikož kontejner chce publikovat porty 80 a 443, potřebuje zvýšené oprávnění. Porty pod 1024 jsou považované za privilegované
- `-e` přepínače obsahují proměnné běhového prostředí, které umožňuje nastavit kontejner *linuxserver/swag*
 - Email nastavuje kontaktní adresu, na kterou služba LetsEncrypt bude informovat v případě, že nebude dlouhou dobu zažádáno o prodloužení certifikátu
 - URL definuje hlavní doménu, na kterou je žádán certifikát
 - Subdomains umožňuje přidání subdomén, pro které bude také vytvořen certifikát
 - Validation definuje typ způsobu ověření vlastnictví domény. V tomto případě je automaticky nakonfigurován Nginx webový server na umístění html souboru, který vygeneruje LetsEncrypt službu pro ověření
 - PUID, GUID umožňují u kontejnerových obrazů z dílny projektu *linuxserver* nastavit, pod jakým uživatelem bude aplikace běžet. Při použití stejného ID jako uživatele operačního systému nebude potřeba zvýšené oprávnění při úpravě konfiguračních souborů
- `--add-host` přepínač umožňuje přidat doménová jména pro překlad IP adres do souboru `/etc/hosts` v kontejneru. Doménové jméno *docker.host* bude odkazovat na IP adresu operačního v přemostovací síti *user* (viz 6.2.2), jelikož Home Assistant byl spuštěn v režimu `--net host` (viz 6.3)

Konfigurace Nginx jako reverzní proxy server

Kontejner po prvním spuštění vygeneroval výchozí konfigurační soubory. V hlavním konfiguračním souboru *nginx/site-confs/default* je automaticky zapnuto přesměrování veškerých požadavků na HTTPS. V doprovodných souborech *nginx/ssl.conf* a *proxy.conf* jsou zapnuté bezpečnostní prvky, které byly vytyčeny při návrhu řešení (viz 5.2.8).

V následujícím výpisu je zobrazena upravená část hlavního konfiguračního souboru. Ten obsahuje definici, která se použije, pokud nebude nalezena žádná jiná (například další definice pro subdoménu). Při načtení hlavní domény (*domov.tajnadomena.com*) tedy dojde k využití této direktivy, kde je zapnutý proxy server odkazující na instanci Home Assistant.

Výpis 6.35 Konfigurace reverzního proxy serveru v Nginx

Soubor `/hone/user/docker/storage/letsencrypt/nginx/site-confs/default`

```
...
Server {
...
location / {
    include /config/nginx/proxy.conf;
    resolver 127.0.0.11 valid=30s;
    set $upstream_homeassistant docker.host;
    proxy_pass http://upstream_homeassistant:8123;
```

```

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forward-For $proxy_add_x_forwarded_for;
}

location /api/websocket {
    resolver 127.0.0.11 valid=30s;
    set $upstream_homeassistant docker.host;
    proxy_pass http://$upstream_homeassistant:8123/api/websocket;
    proxy_set_header Host $host;

    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forward-For $proxy_add_x_forwarded_for;
}
...
}

```

Konfigurace Home Assistant

Do hlavního konfiguračního souboru byly vloženy informace, že je Home Assistant provozován pod proxy serverem a může využít hlaviček *X-Forward-For*, do kterých Nginx zapíše původní IP adresu příchozího uživatele. Home Assistant tímto může informovat uživatele v případě, že se někdo pokusil přihlásit se špatným heslem. Při neposílání hlavičky by Home Assistant viděl veškeré požadavky přicházející z IP adresy Docker sítě s LetsEncrypt kontejnerem. Zároveň je nastavena Docker síť jako jediný zdroj, který může posílat tyto hlavičky, a přihlášení uživatele je omezeno na 5 pokusů, po kterých bude IP adresa zablokována. Po vložení této konfigurace byl Home Assistant restartován.

Výpis 6.36 Nastavení Home Assistant pro proxy server

Soubor /home/user/docker/storage/homeassistant/configuration.yaml

```

http:
  use_x_forwarded_for: true
  trusted_proxies:
    - 172.20.10.0/24
  ip_ban_enabled: true
  login_attempts_threshold: 5

```

6.7 Vzdálená správa

Instalace SSH serveru

SSH server je součástí balíčku `openssh`. Je nainstalován do operačního systému.

```
user@SERVER: ~ $ sudo pacman -S openssh
```

Vygenerování privátního a veřejného klíče

Součástí `openssh` balíčku je také utilita `ssh-keygen` pro generování párů klíčů. SSH server se ve výchozím nastavení při autentizaci pomocí veřejného klíče dívá do souboru v domovské složce přihlašovaného uživatele `~/.ssh/authorized_keys`. Při generování klíče bude do složky `.ssh` uložen veřejný klíč pod názvem `id_rsa.pub`. Bude přejmenován na `authorized_key`. Soukromý klíč bude uložen do souboru `id_rsa` ve stejné složce. Klíč je překopírován do klientských zařízení, ze kterých bude prováděna vzdálená správa serverů, a je smazán ze serveru.

Výpis 6.37 Příprava veřejného a privátního klíče

```
user@SERVER: ~ $ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user/.ssh/id_rsa):
Created directory '/home/user/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_rsa
Your public key has been saved in /home/user/.ssh/id_rsa.pub

user@SERVER: ~ $ cd .ssh
user@SERVER: ~ $ mv id_rsa.pub authorized_keys
user@SERVER: ~ $ cat id_rsa
user@SERVER: ~ $ rm id_rsa
```

Konfigurace SSH serveru

Ve výchozím nastavení je již povolena autentizace pomocí veřejného/privátního klíče. Na konec souboru s konfigurací SSH serveru bude přidána definice zakázání ověřování pomocí prostého uživatelského hesla do systému, vyjma domácí sítě LAN. Konfigurace se nachází v souboru `/etc/ssh/sshd_config`. Pozor na podobný soubor `ssh_config`, který se stará o konfiguraci SSH při roli klienta.

```
Soubor /etc/ssh/sshd_config
```

```
PasswordAuthentication no
Match Address 10.0.0.0/16
    PasswordAuthentication yes
```

Spuštění SSH serveru

SSH server je nastaven pro spuštění při každém startu systému. Přepínač `--now` spustí službu okamžitě bez potřeby restartu systému.

```
user@SERVER: ~ $ sudo systemctl enable --now sshd.service
```

6.8 Bezpečnost

Bezpečnostní požadavky byly postupně implementovány v průběhu konfigurace jednotlivých částí řešení.

- Minimální linuxová distribuce snižuje množství potenciálních chyb
- Webový server pro vzdálený přístup používá HTTPS s platným certifikátem. Aplikace dostupné na webovém serveru používají autentizaci uživatele
- Firewall na routeru byl otevřen pouze na porty webového serveru a SSH
- Při konfiguraci SSH serveru bylo zapnuté ověřování pomocí kombinace veřejného/privátního klíče
- MQTT server byl zaheslován
- Zigbee a Z-Wave sítě byly nastaveny, aby neumožňovaly připojení nových zařízení bez ručního otevření sítě
- Wi-Fi byla nakonfigurována do WPA2-PSK

6.9 Konfigurace hlasových asistentů

6.9.1 Google Assistant

Po nastavení Vzdáleného přístupu přes HTTPS je možné napojit Home Assistant, který implementuje Google Actions API pro ovládání chytré domácnosti.

Příprava Google účtu

Dle dokumentace Home Assistant je implementováno nastavení Google účtu, který bude použitý pro Google Assistant [63]. Je provedeno přihlášení do administračního portálu Google Actions console [42] a vykonány následující kroky:

- Nový projekt
 - Jméno projektu – „chytra-domacnost“
 - Jazyk pro hlasové akce – angličtina
 - Země nebo region – Česká republika
- Typ akce
 - Chytrá domácnost
 - Kliknuto na „Začít“
- Záložka Vývoj – Vyvolání (Develop – Invocation)
 - Jméno akce – Home Assistant
 - Tento název bude používán pro některé hlasové příkazy
 - Kliknuto na „Uložit“
- Záložka Vývoj – Akce (Develop – Actions)
 - URL Adresa pro splnění akce (Fullfilment URL) – https://domov.tajnadomena.com/api/google_assistant
 - Zde se použije nastavená doména, která je použita pro vzdálený přístup
 - Google Assistant API je dostupné v Home Assistant na API koncovém bodě `/api/google_assistant`
 - Kliknuto na „Uložit“
- Záložka Vývoj – Propojení účtu (Develop – Account Linking)
 - Nejdříve bylo zjištěno ID Projektu
 - Kliknutím na menu vpravo nahoře a otevřením sekce – Nastavení projektu
 - Client ID – <https://oauth-redirect.googleusercontent.com/r/chytra-domacnost>
 - Client Secret – Zde Home Assistant nic nepotřebuje, stačí náhodný řetězec čísel
 - Autorizační URL – <https://domov.tajnadomena.com/auth/authorize>
 - URL Tokenu - <https://domov.tajnadomena.com/auth/token>
 - Konfigurace klienta – Seznam oprávnění, která bude aplikace žádat po Google účtu
 - email
 - name
 - Kliknuto na „Uložit“
- Záložka Testování – Nastavení
 - Ověřeno, zda je v sekci nastavení zapnuta volba „Testování na zařízeních“
 - Tato možnost povolí připojení na Google Assistant reproduktorech

V tomto momentě je k dispozici v Google účtu integrace Home Assistant do platformy Google Home/Google Assistant. Dále byl vytvořen servisní účet pro Home Assistant, který bude mít přístup k cloudovým službám Google API. Díky tomu bude moci reagovat na přijaté hlasové a jiné povely z Google Assistant. Je použitý stejný projekt v Google Cloud Platform, který byl vytvořen ve fázi implementace zálohování (viz 6.5).

- Je zvolen stejný projekt s názvem „chytry-domov“
- Povoleno HomeGraph API

- Menu API & Služby -> Knihovna -> HomeGraph API -> Povolit
- Vytvoření servisního účtu
 - Menu IAM & Admin -> Servisní účty -> Vytvořit servisní účet
 - První krok – detaily o účtu
 - Název účtu – „Home Assistant“
 - ID účtu – „homeassistant“
 - Kliknuto na „Vytvořit“
 - Druhý krok – oprávnění pro účet
 - Role – generátor tokenů servisních účtů (Service Account Token Creator)
 - Kliknuto na „Pokračovat“
 - Třetí krok – povolení dalších uživatelů k přístupu
 - Přeskočeno
 - Kliknuto na „Dokončit“
 - Přejde se na seznam servisních účtů
 - U vytvořeného účtu je kliknuto na „Akce“ -> Vytvořit Klíč
 - Typ klíče – zvolen formát JSON
 - Dojde ke stažení JSON souboru do počítače

Konfigurace Home Assistant

Vytvořený JSON soubor obsahující autentizační údaje pro přístup ke Google HomeGraph API a umožňující export zařízení chytré domácnosti do platformy Google Assistant, je umístěný do kořenové konfigurační složky Home Assistant `/home/user/docker/homeassistant/google_service_account.json`. V hlavním konfiguračním souboru `configuration.yaml` pro Home Assistant je poté definována integrace Google Assistant:

Výpis 6.39 Konfigurace integrace Google Assistant v Home Assistant

Soubor `/home/user/docker/storage/homeassistant/configuration.yaml`

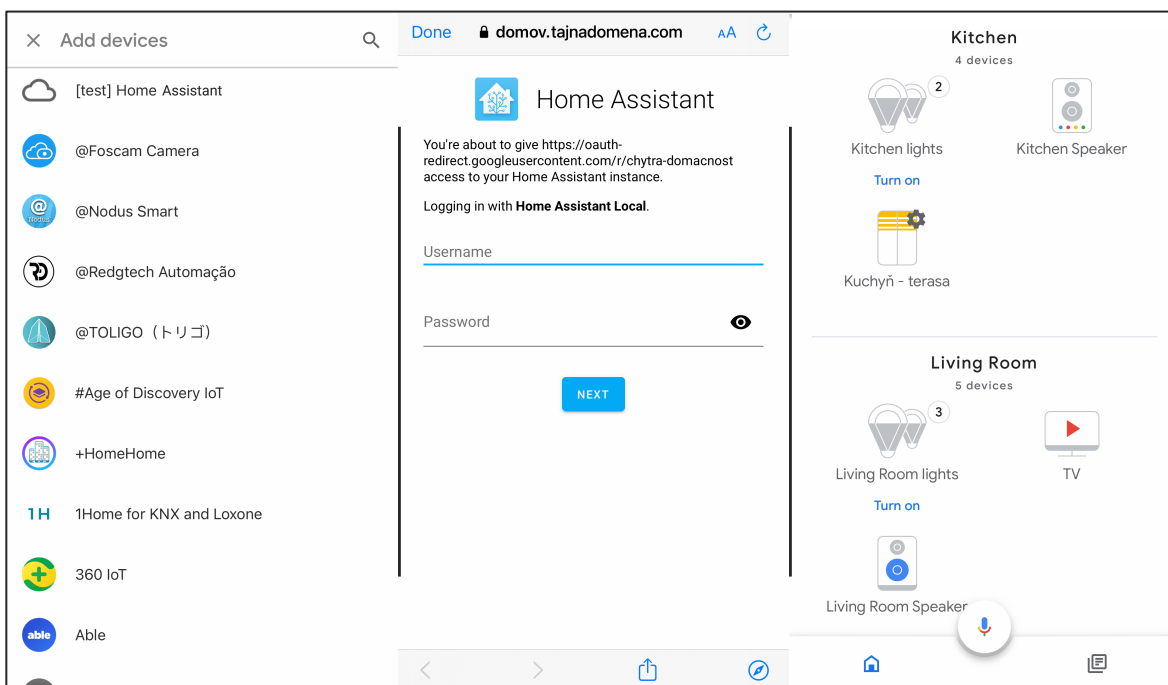
```
google_assistant:
  project_id: chytry-domov
  service_account: !include google_service_account.json
  expose_by_default: true
  exposed_domains:
    - script
    - light
    - sensor
    - climate
    - cover
  entity_config:
    cover.chodba:
      name: Hallway Blind
```

- *project_id* – označuje ID projektu z Google Actions
- *service_account* – je odkaz na stažený soubor z Google Cloud Platform rozhraní
- Využilo se možnosti filtrování, jaká zařízení mohou být exportována, a zařízeními byly přiděleny jiné názvy, které jsou více vhodné pro anglické hlasové ovládání

Konfigurace v Google Home

Aplikace Google Home, která je dostupná pro iOS a Android zařízení, je grafické rozhraní pro konfiguraci služeb Google Assistant. Byl použitý nakonfigurovaný Google účet s vytvořenou integrací pro Home Assistant a tato integrace byla v aplikaci zapnuta. Poté již došlo k importu všech zařízení do Google Assistant, a je možné hlasové ovládání.

1. Tlačítko + (Přidání nových zařízení) -> [test] Home Assistant
 - Integrace je označená jako test, jelikož se jedná o vlastní integraci



Obr. 21 Připojení nakonfigurované integrace Home Assistant do Google Home

6.9.2 Apple Siri

Konfigurace Home Assistant

V Home Assistant v hlavním konfiguračním souboru *configuration.yaml* byla povolena integraci HomeKit pro export zařízení. Dále byly použity dodatečné filtry pro zakázání exportu nepotřebných zařízení. Nakonec byla zařízení přejmenována do anglických názvů pro korektní hlasové ovládání. Po restartu Home Assistant byla integrace aktivní.

Výpis 6.40 Konfigurace exportu zařízení do HomeKit v Home Assistant

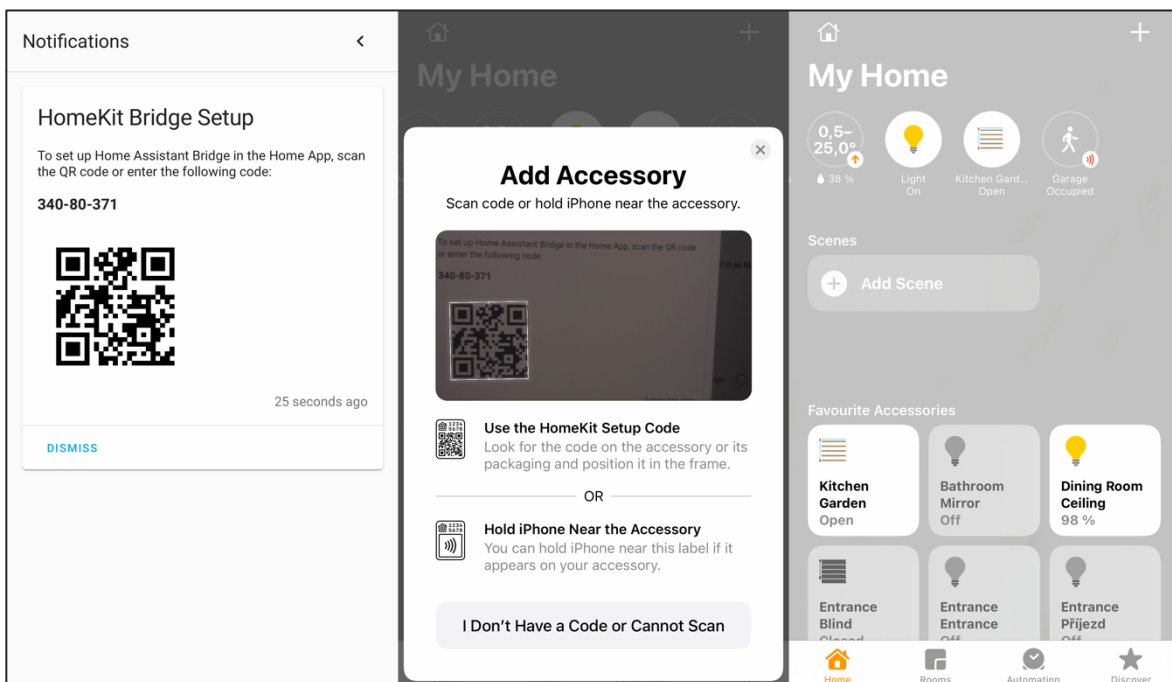
Soubor /home/user/docker/storage/homeassistant/configuration.yaml

```
homekit:
  filter:
    exclude_entities:
      - light.jidelna_strop_1
      - light.jidelna_strop_2
      - light.jidelna_strop_3
    exclude_domains:
      - device_tracker
      - script
      - media_player
    include_entities:
      - media_player.tv
  entity_config:
    sensor.pool_teploata:
      name: Pool Water
    cover.predsina:
      name: Entrance Blind
    light.predsina:
      name: Entrance
```

Konfigurace Apple Home

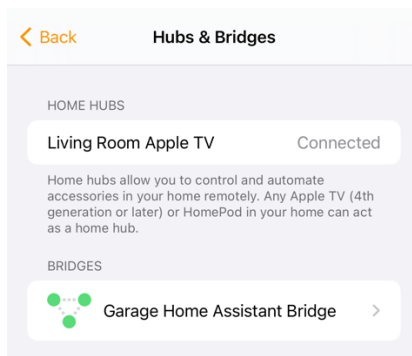
Po aktivaci integrace je k dispozici v notificačním panelu Home Assistant QR HomeKit kód pro napojení do Apple Home. Byla provedena konfigurace v Apple Home aplikaci:

2. Byla spuštěna aplikace Apple Home
3. Kliknuto na + (přidání nového příslušenství)
4. Naskenován QR kód
5. Povoleno, že se jedná o necertifikované HomeKit zařízení
6. Připojená zařízení z Home Assistant se roztřídila do místnosti



Obr. 22 Připojení Home Assistant do Apple Home

Nyní je možné ovládat všechna zařízení přes hlasového asistenta Apple Siri. Do domácí sítě LAN byla připojena Apple TV, která slouží jako brána pro přístup k zařízením pomocí hlasového asistenta ze sítě Internet. Tento stav lze ověřit v aplikaci Apple Home.



Obr. 23 Nastavení Apple TV jako brány pro HomeKit přes síť Internet

6.10 Konfigurace sběru a analýzy historických dat

6.10.1 Konfigurace databáze InfluxDB

Inicializace databáze

Před spuštěním trvalého kontejneru byla nejdříve provedena jednorázová inicializace databáze, která vytvoří uživatelské účty a prázdnou databázi s názvem *homeassistant*. Po inicializaci byla ze skriptu vymazána uživatelská hesla.

Výpis 6.41 Skript pro inicializaci databáze InfluxDB dočasným kontejnerem

```
Soubor /home/user/docker/servicescripts/influxdb_initialize.sh
```

```
#!/bin/bash

docker run --rm \
  -v /home/user/docker/storage/influxdb:/var/lib/influxdb \
  -e TZ=Europe/Prague \
  -e PUID=1000 \
  -e PGID=1000 \
  -e INFLUXDB_DB=homeassistant \
  -e INFLUXDB_ADMIN_ENABLED=true \
  -e INFLUXDB_ADMIN_USER=admin \
  -e INFLUXDB_ADMIN_PASSWORD=supertajneheslo \
  -e INFLUXDB_USER=hass -e INFLUXDB_USER_PASSWORD=dvere \
  -e INFLUXDB_USER=grafana -e INFLUXDB_USER_PASSWORD=dvere \
  influxdb /init-influxdb.sh
```

Poté byl spuštěn standardní kontejner. Byl otevřen port do domácí sítě a ponechán na standardní hodnotě 8086, kterou používá databáze InfluxDB jako výchozí.

Výpis 6.42 Skript pro spuštění InfluxDB kontejneru

```
docker run -d \
  --name="influxdb" \
  -v /home/user/docker/storage/influxdb:/var/lib/influxdb \
  -v /etc/localtime:/etc/localtime:ro \
  -e TZ=Europe/Prague \
  -e PUID=1000 \
  -e PGID=1000 \
  --network user \
  -p 8086:8086 \
  --restart unless-stopped \
  influxdb
```

Napojení na Home Assistant

Výpis 6.43 Konfigurace integrace InfluxDB v Home Assistant

```
influxdb:
  host: 127.0.0.1
  database: homeassistant
  username: supertajnyuzivatel
  password: supertajneheslo
  include:
    entities:
```

- sensor.bazen_teplosta
- sensor.trava_vlhkost

6.11 Konfigurace dashboard aplikace Grafana

Spuštění kontejneru

Výpis 6.44 Skript pro spuštění kontejneru Grafana

```
docker run -d \  
  --name="grafana" \  
  -v /home/user/docker/storage/grafana:/var/lib/grafana \  
  -v /etc/localtime:/etc/localtime:ro \  
  -e TZ=Europe/Prague \  
  --network user \  
  -p 3000:3000 \  
  --restart unless-stopped \  
grafana/grafana
```

Konfigurace aplikace Grafana

Po spuštění je v aplikaci Grafana vytvořen uživatelský účet a bylo nakonfigurováno spojení s InfluxDB databází.

7. Konfigurace -> Datové zdroje -> Přidat datový zdroj
 - Jméno – „Home Assistant“
 - Jazyk dotazování – „InfluxQL“
 - HTTP – „http://influxdb:8086“
 - Detaily InfluxDB databáze
 - Databáze – „homeassistant“
 - Uživatel a heslo
 - Kliknuto na „Uložit a otestovat“
 - Připojení bylo navázáno

Vzdálený přístup do aplikace Grafana

Po spuštění a nastavení vzdáleného přístupu (viz 6.6) byl v kontejneru nastaven Nginx reverzní proxy server i pro aplikaci Grafana na doméně *grafana.domov.tajnadomena.com*. V kontejnerovém obrazu *linuxserver/swag* byla aktivována přednastavená definice pro proxy server Grafana.

Výpis 6.45 Aktivace reverzního proxy serveru pro aplikaci Grafana

```
user@SERVER: ~ $ cd /home/user/docker/storage/letsencrypt/nginx/proxy-confs/  
user@SERVER: ~ $ mv grafana.subdomain.conf.sample grafana.subdomain.conf  
user@SERVER: ~ $ docker restart letsencrypt
```

Soubor /home/user/docker/storage/letsencrypt/nginx/proxy-confs/grafana.subdomain.conf

```
...
location / {
...
    set $upstream_app grafana;
    set $upstream_port 3000;
    set $upstream_proto http;
    proxy_pass $upstream_proto://$upstream_app:$upstream_port;
}
```

V druhé části výpisu lze vidět, že soubor s konfigurací pro proxy server byl již nastaven na doménové jméno *grafana*. Protože se lze odkazovat na další Docker kontejnery jejich jmény, nepotřeboval soubor žádnou další konfiguraci.

6.12 Aktualizace a údržba

Operační systém

Pro rychlou aktualizaci operačního systému byla implementována zkratka do *.bashrc*. Tímto po přihlášení uživatele postačí zadat příkaz *up*.

1. Nejdříve upozorní uživatele na kontrolu hlavních webových stránek distribuce Arch Linux, zda není potřeba manuální intervence u nějakého balíčku
2. Poté provede stažení a instalaci všech nových verzí balíčků
3. Vyčistí dočasné instalační soubory

Výpis 6.46 Terminálová zkratka pro aktualizaci systému

Soubor /home/user/.bashrc

```
alias up='echo "Zkontroluj www.archlinux.org" && sudo pacman -Syu && sudo pacman -Scc'
```

Kontejnery

Byl implementován servisní skript *updatecontainer.sh* pro aktualizaci kontejneru. V příloze je dostupný celý zdrojový kód skriptu (viz Příloha D:). Skript provede následující kroky:

1. Zobrazí aktuálně běžící kontejnery a dotáže se uživatele na výběr kontejneru k aktualizaci
2. Zastaví a odstraní aktuální kontejner
3. Stáhne nejnovější kontejnerový obraz
4. Vytvoří a spustí nový kontejner

6.13 Přehled spuštěného softwaru a portů

Tabulka 2 Přehled běžícího softwaru na serveru s otevřenými porty

Aplikace	Kontejner/balíček	Otevřený port	Otevřen do sítě Internet
Home Assistant	homeassistant/home-assistant	8123	Ne
Eclipse Mosquitto	eclipse-mosquitto	1883, 9001	Ne
Zigbee2MQTT	koenkk/zigbee2mqtt	8124	Ne
ZWave2MQTT	robertslando/zwave2mqtt	8125	Ne
InfluxDB	influxdb	8086	Ne
Grafana	grafana/grafana	3000	Ne
Nginx	linuxserver/swag	80, 443	Ano
SSH Server	openssh ^a	22	Ano (přes port 666)

Pozn.^a Nainstalován přímo do operačního systému

7 Testování implementace

Ve fázi testování je ověřena funkčnost provedené implementace návrhu. Testovací scénáře definují očekávanou funkčnost každé komponenty a ověří, zda došlo k úspěšné implementaci. Na základě jednotlivých výsledků je vytvořen souhrnný tabulkový report o stavu implementace.

7.1 Testovací scénáře

7.1.1 Test síťové konfigurace

Příkazem *ping* lze rychle ověřit, zda je systém úspěšně připojen k síti Internet. Proběhne ke kontaktování serveru google.com, pokud dojde k přeložení domény na IP adresu a dojde k přijetí paketů, konfigurace byla úspěšná.

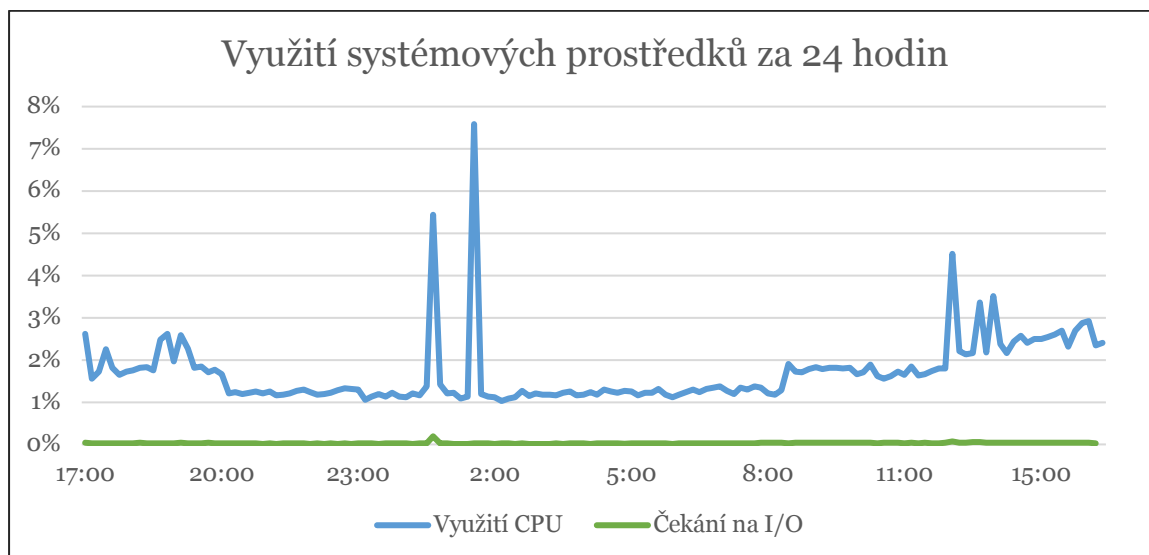
Výpis 7.1 Test síťové konfigurace

```
user@SERVER: ~ $ ping google.com
PING google.com (172.217.23.206) 56(84) bytes of data.
64 bytes from prg03s05-in-f14.1e100.net (172.217.23.206): icmp_seq=1 ttl=57 time=7.02 ms
64 bytes from prg03s05-in-f14.1e100.net (172.217.23.206): icmp_seq=2 ttl=57 time=6.44 ms
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 6.439/6.731/7.024/0.292 ms
```

Doména byla úspěšně přeložena na IP adresu a pakety se úspěšně vrátily. Internetové připojení v operačním systému bylo korektně nakonfigurováno.

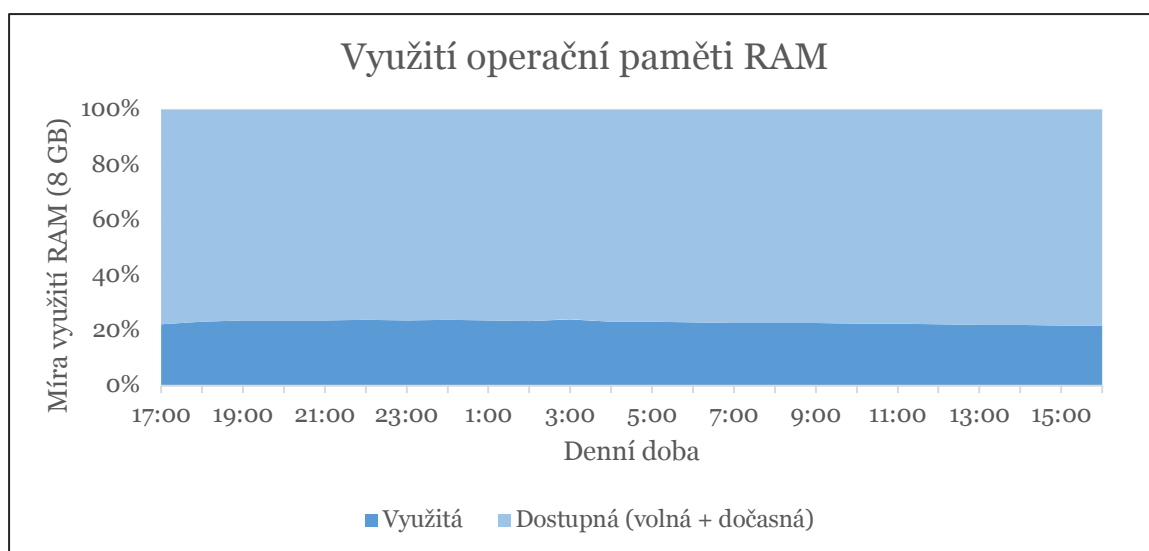
7.1.2 Test využití HW prostředků

Po implementaci všech komponent bylo spuštěno pozorování vytížení hardwaru pomocí aplikace Netdata [64] v kontejnerovém prostředí. Dle specifikace požadavků (viz 3.1.1) se očekává průměrné vytížení systému pod 30 % výkonu.



Obr. 24 Využití systémových prostředků – CPU, I/O, čím méně, tím lépe

Graf výše ukazuje metriky využití procesorového výkonu a kolik času strávil procesor při čekání na vstupní a výstupní data (například při čtení dat z pevného disku). Na grafu lze vidět výkyv, kdy byl mezi 23. a 2. hodinou proveden údržbový restart systému. Výkyv mezi 11. a 15. hodinou zaznamenal restarty kontejnerů při aktualizaci. Při pohledu na levou osu je patrné, že celkové zatížení systému je při běžném provozu pod 5 % celkového výkonu. Volba operačního systému s minimální stopou se vyplatila pro vysoce optimalizovaný běh aplikací a je hluboce pod očekávanou hranicí zatížení.



Obr. 25 Využití operační paměti RAM, čím méně, tím lépe

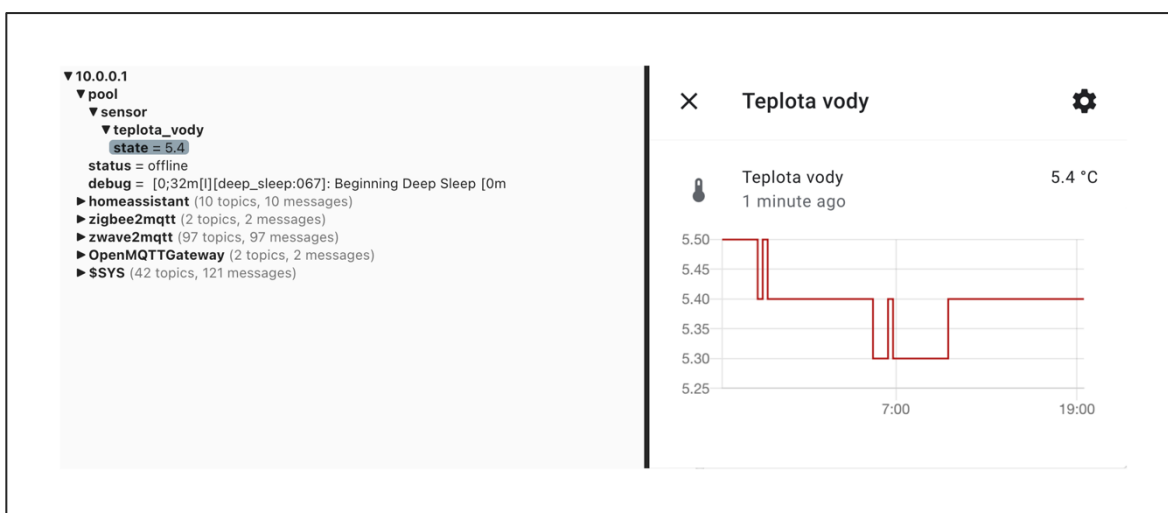
Ukázalo se, že aplikace Netdata k ukládání dat o vytížení systémových prostředků využívala operační paměť RAM. Aplikace byla překonfigurována na ukládání informací přímo na SSD disk a byl spuštěn druhý celodenní test pro monitorování operační paměti RAM. Z grafu výše je patrné, že využitá paměť se pohybuje okolo hranice 20 % celkové velikosti paměti,

tedy okolo 1,6 GB. SWAP paměť (odkládání dat z operační paměti na pevný disk, které se využívá při nedostatku fyzické operační paměti) nebyla po celou dobu využita.

Testovací scénář využití systémových prostředků byl úspěšný ve všech parametrech. Pokud by nebylo plánované žádné rozšíření a vývoj systému, mohl se zvolit méně výkonný procesor.

7.1.3 Test zachycení a zpracování MQTT zprávy

Pro test MQTT serveru a integrace do MQTT bylo použité vlastnoručně vytvořené teplotní voděodolné čidlo DS18B20 připojené do mikordesky s čipem ESP32. Na desku byl nahrán firmware ESPHome, jenž má podporu pro vysílání zpráv přes MQTT. Do MQTT tématu `pool/sensor/teplota_vody/state` posílá periodicky každých 20 minut teplotu z čidla.



Obr. 26 Úspěšně zachycená MQTT zpráva a správné zpracování v Home Assistant

Výpis 7.2 Definice generického MQTT senzoru v Home Assistant

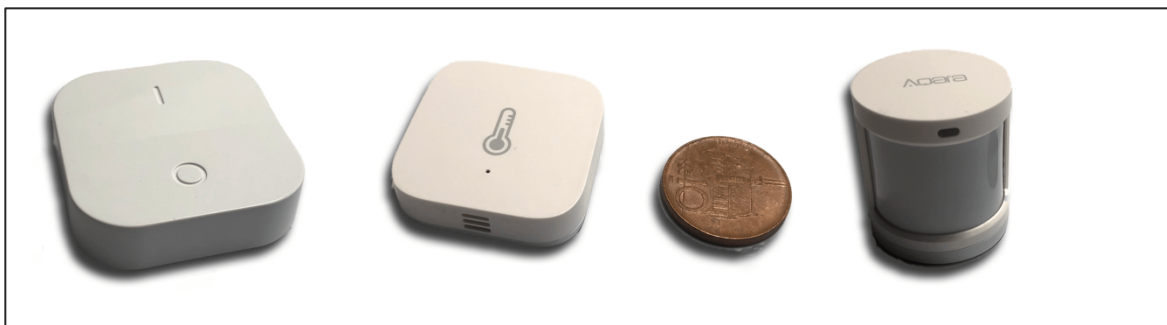
Soubor `/home/user/docker/homeassistant/configuration.yaml`

```
sensor:
  - platform: mqtt
    state_topic: "pool/sensor/teplota_vody/state"
    device_class: temperature
    unit_of_measurement: °C
    name: "Teplota vody"
    unique_id: pool-30aea416078c
    force_update: true
```

force_update – vynutí, aby Home Assistant aktualizoval stav senzoru i v případě, že se nezměnila jeho hodnota od minulého měření

Na obrázku (viz Obr. 26) lze v levé části vidět úspěšně přijaté zprávy MQTT serverem (viděno z aplikace MQTT Explorer). V pravé části je poté výsledek úspěšného testu integrace MQTT v Home Assistant.

7.1.4 Test Zigbee zařízení

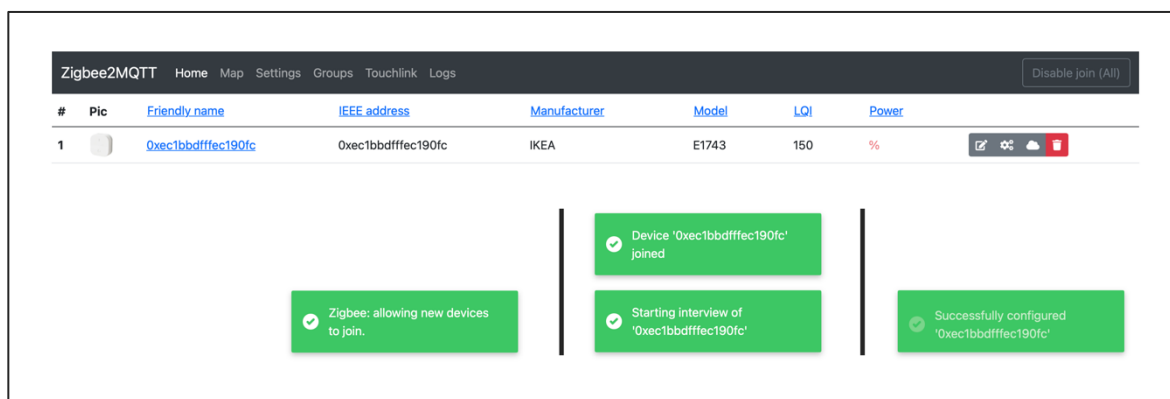


Obr. 27 Příklady testovaných Zigbee zařízení IKEA, Aqara

Díky větším odchylkám od standardu u Zigbee zařízení (viz 4.5.2) je vybrána větší testovací sada Zigbee zařízení od různých výrobců, která ověří kompatibilitu zprovozněného systému. Testovací scénář předpokládá úspěšné připojení všech zařízení do Zigbee sítě, poté správné reportování všech dat z jejich senzorů v průběhu jednoho dne a korektní posílání příkazů na ovládání u zařízení, které to umožňují.

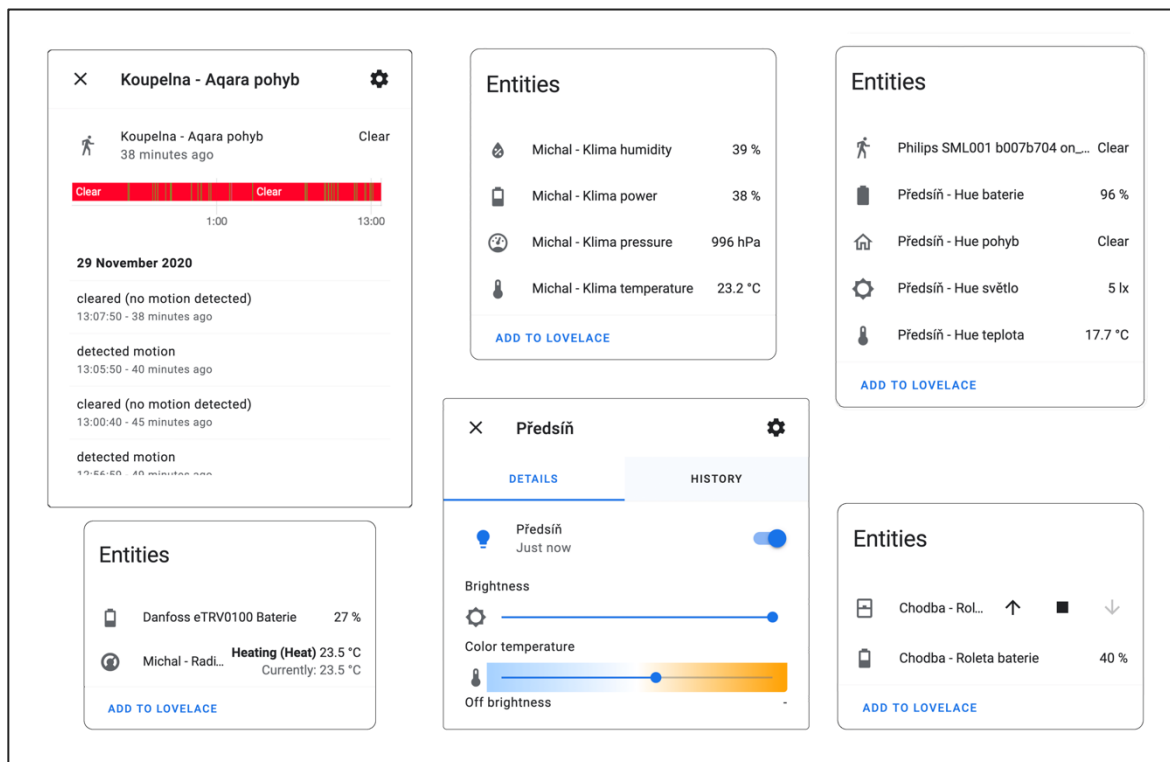
Seznam testovaných Zigbee zařízení

- Aqara senzor teploty, vlhkosti a hydrostatického tlaku
- Aqara pohybové čidlo
- Danfoss Ally Termostatická hlavice pro radiátory
- Gledopto ovladač pro LED pásek (Zigbee světlo)
- Philips Hue pohybové čidlo
- IKEA okenní roleta Fyrtur
- IKEA tlačítko On/Off



Obr. 28 Připojení zařízení do Zigbee sítě ve webovém rozhraní Zigbee2MQTT

Test probíhal otevřením Zigbee sítě, čímž bylo povoleno připojení nových zařízení, a spárováním každého konečného zařízení se sítí. Tento proces byl prováděn z implementovaného webového rozhraní, které je ukázané na obrázku výše společně s notifikacemi, které informují o průběhu párování každého zařízení.



Obr. 29 Integrovaná Zigbee zařízení v Home Assistant – Aqara, Hue, Danfoss, IKEA

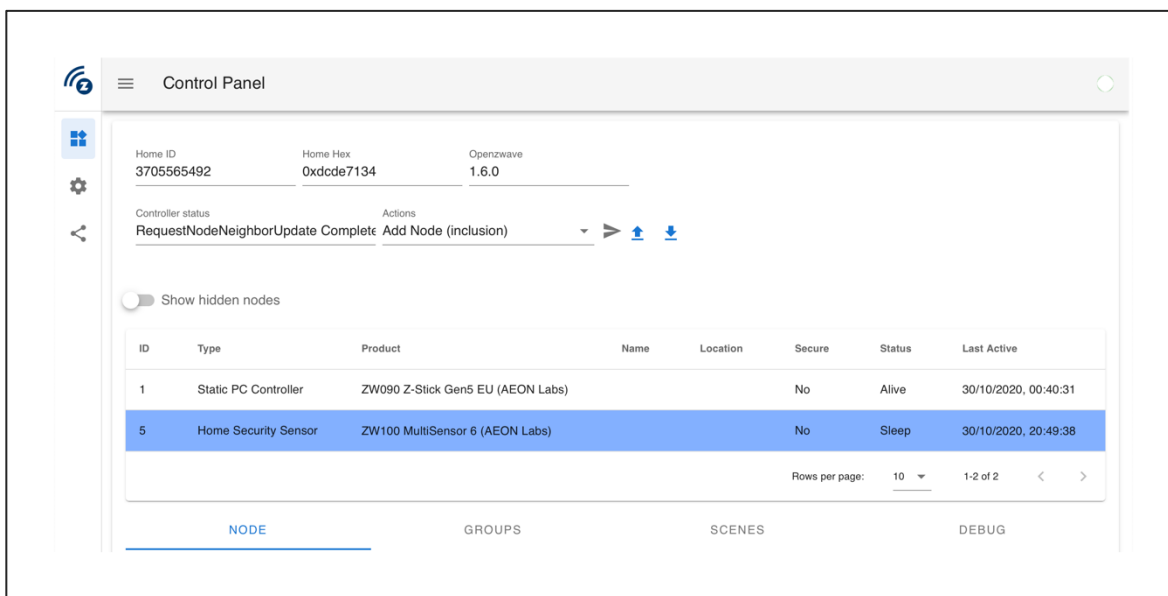
Všechna testovaná zařízení správně reportovala své hodnoty do Zigbee2MQTT a v Home Assistant se správně propisaly. Senzory reportovaly své hodnoty po celou dobu testovacího intervalu. Příkazy pro ovládání zařízení poslané z Home Assistant se projeví. Došlo k zapnutí/vypnutí světel, úspěšné změně teploty na termostatické hlavici a vysunutí/zasunutí chytré rolety. Testovací scénář proběhl úspěšně.

7.1.5 Test Z-Wave zařízení



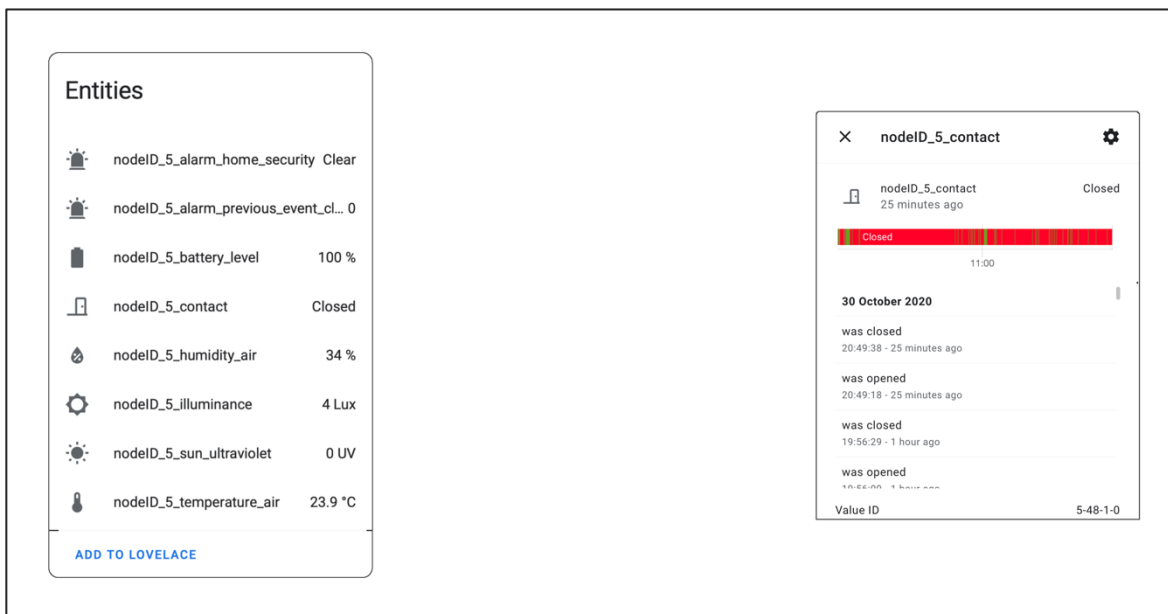
Výpis 7.3 Testované Z-Wave zařízení Aeotec Multisensor 6

Pro připojení k Z-Wave síti byl vybrán všestranný senzor Aeotec Multisensor 6, který měří teplotu, vlhkost, světlo, UV záření a pohyb. Díky důslednému certifikačnímu procesu výrobců Z-Wave zařízení bylo použito jen jedno zařízení, jelikož všechny komerční produkty musí přesně implementovat specifikaci protokolu (viz 4.5.3). Testovací scénář očekává reportování všech hodnot senzorů v Home Assistant v testovacím intervalu jednoho dne.



Obr. 30 Připojení zařízení do Z-Wave sítě ve webovém rozhraní ZWave2MQTT

Na obrázku výše je vidět webové rozhraní pro konfiguraci Zwave2MQTT. Síť byla přepojena do režimu připojení nového zařízení (inkluze). Aeotec Multisensor byl přepnutý do režimu párování a úspěšně se připojil do sítě.

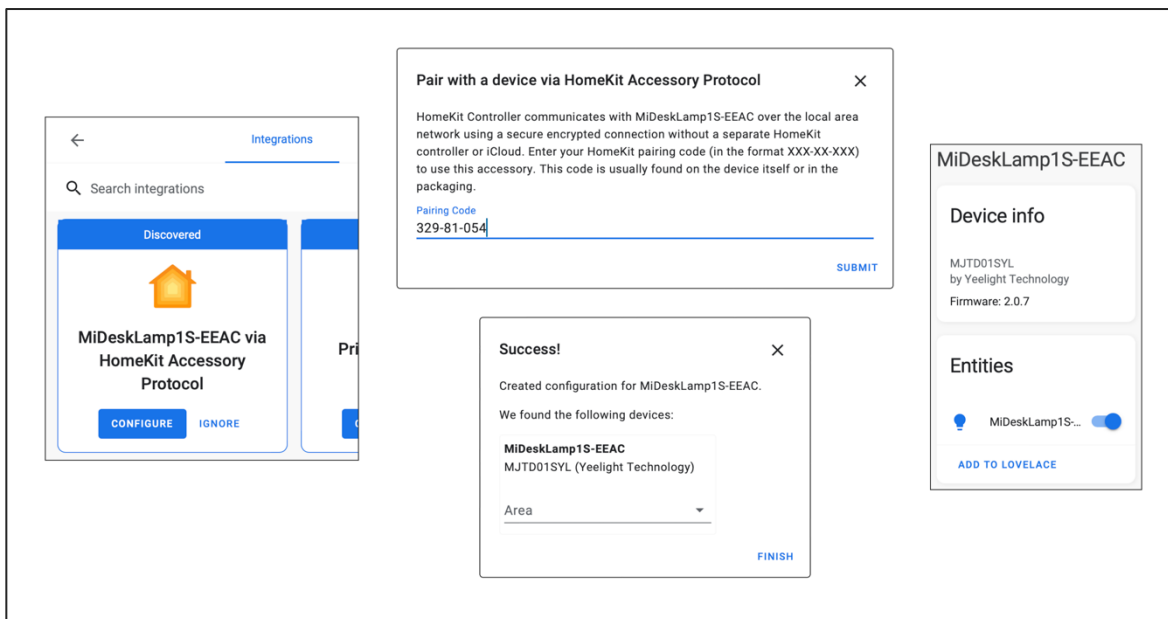


Obr. 31 Integrované Z-Wave zařízení v Home Assistant – Aeotec Multisensor 6

V Home Assistant, jak je vidět na obrázku výše, se poté korektně propsaly veškeré hodnoty 24 hodin testovaného intervalu. Testovací scénář proběhl úspěšně.

7.1.6 Test HomeKit zařízení

Pro test integrace HomeKit protokolu byla použita stolní lampa Xiaomi Desk Lamp. Testovací scénář předpokládá automatické objevení HomeKit zařízení v domácí síti LAN a korektní reportování aktuálního stavu zařízení.



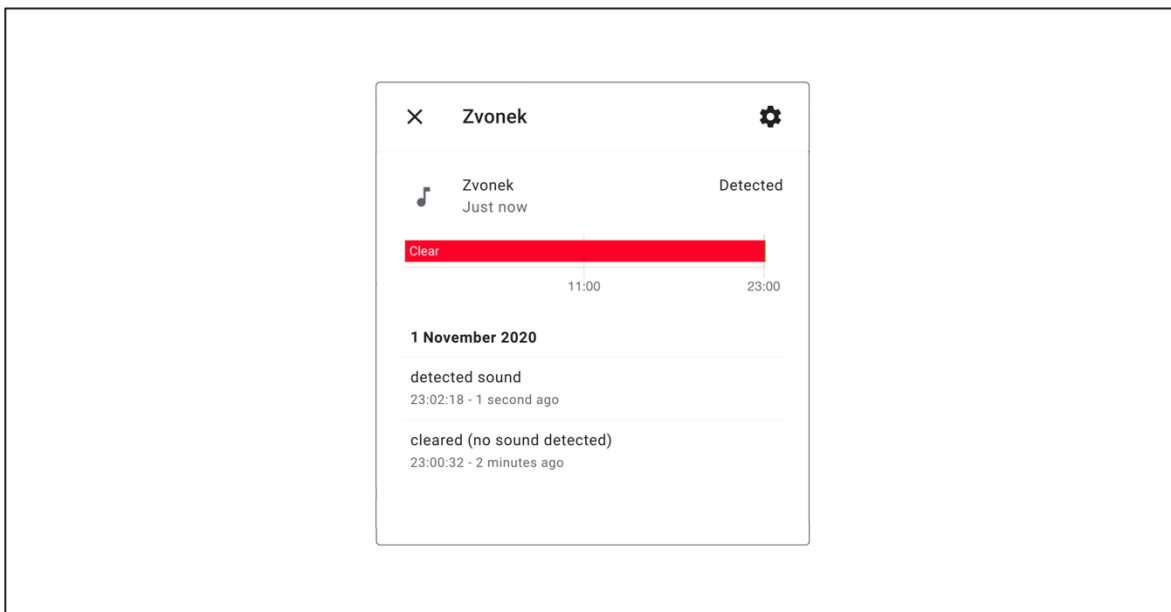
Obr. 32 Test připojení HomeKit zařízení do Home Assistant

Zařízení bylo objeveno jako dostupné k párování v sekci *Integrace*. Po kliknutí na konfigurovat a opsání HomeKit kódu pro párování, který se nacházel na zařízení, došlo k přidání zařízení do Home Assistant. Reportování stavu fungovalo korektně – při zapnutí/vypnutí světla ručně se změna stavu ihned projevila v Home Assistant. Naopak posílání příkazu z Home Assistant bylo také provedeno bez odezvy. Testovací scénář proběhl úspěšně.

7.1.7 Test RF zařízení

Testovací scénář pro ověření správné integrace RF zařízení využívá bezdrátový domovní zvonek. Scénář předpokládá:

- Zachycení vždy stejného bezdrátového kódu při stisknutí tlačítka na MQTT serveru
- Zobrazení tohoto stavu do Home Assistant
 - V Home Assistant byla ručně vytvořena senzorová entita, výpis kódu je obsažen v sekci implementace RF (viz Výpis 6.28)



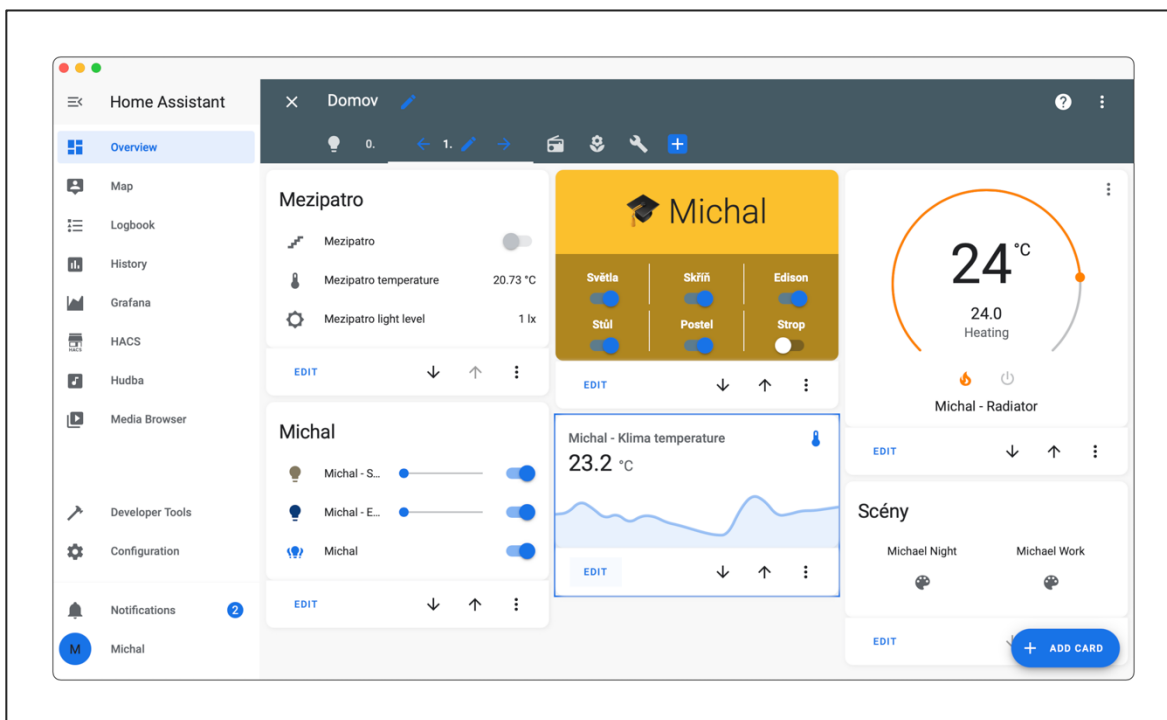
Obr. 33 Test připojení RF tlačítka do Home Assistant

Kód zpracovaný OpenMQTTGateway byl po každém stisku tlačítka stejný a propal se do MQTT tématu *OpenMQTTGateway/SRFB/SRFBtoMQTT*. V Home Assistant se po každém stisku změnil stav senzoru.

7.1.8 Test uživatelské samosprávy a kastomizace

Ve specifikacích požadavků byla definována schopnost uživatelské úpravy grafického rozhraní a schopnost vytvářet automatizační scénáře bez potřeby přítomnosti návrháře systému. Testovací scénář předpokládá možnost změnit prvky grafického rozhraní pro ovládání chytré domácnosti a vytvoření automatizace pomocí grafických průvodců:

- Přidání tlačítka pro vypnutí/zapnutí zařízení
- Přidat grafický ukazatel čidla
- Přidání prvku pro ovládání termostatu
- Přidání tlačítka pro spuštění vybrané akce



Obr. 34 Samospráva grafického prostředí Home Assistant

Prostředí Home Assistant obsahoval editor grafického prostředí, kde se podařilo pomocí grafického rozhraní přidat definované prvky do obrazovky. Testovací scénář proběhl úspěšně.

7.1.9 Test automatizačního scénáře

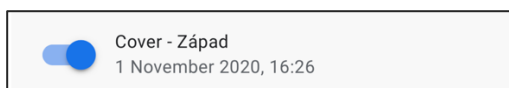
Připravený automatizační skript využívá jako spouštěč interní entitu *Slunce* dostupnou v Home Assistant, která na základě souřadnic chytré domácnosti propočítává aktuální přesný úhel elevace Slunce. Automatizace se má spustit v případě, kdy elevace Slunce spadne 1° pod horizont. Jako akce je nastaveno zavolání služby pro zavření dvou definovaných rolet, kde jedna operuje na Zigbee protokolu a druhá na RF protokolu. Dle výpočtu na základě online kalkulatoru SunCalc [65] mělo 1. listopadu na testované lokalitě dojít k této situaci mezi 16:26 a 16:27 místního času.

Výpis 7.4 Připravený automatizační skript v Home Assistant

```
alias: Cover - Západ
trigger:
  - below: '-1'
    entity_id: sun.sun
    platform: numeric_state
    value_template: '{{ state.attributes.elevation }}'
action:
  - entity_id:
    - cover.predsins
```

```
- cover.chodba
service: cover.close_cover
mode: single
```

Záznam o čase poslední aktivace spouštěče ukazuje, že automatizace byla provedena ve správný okamžik.



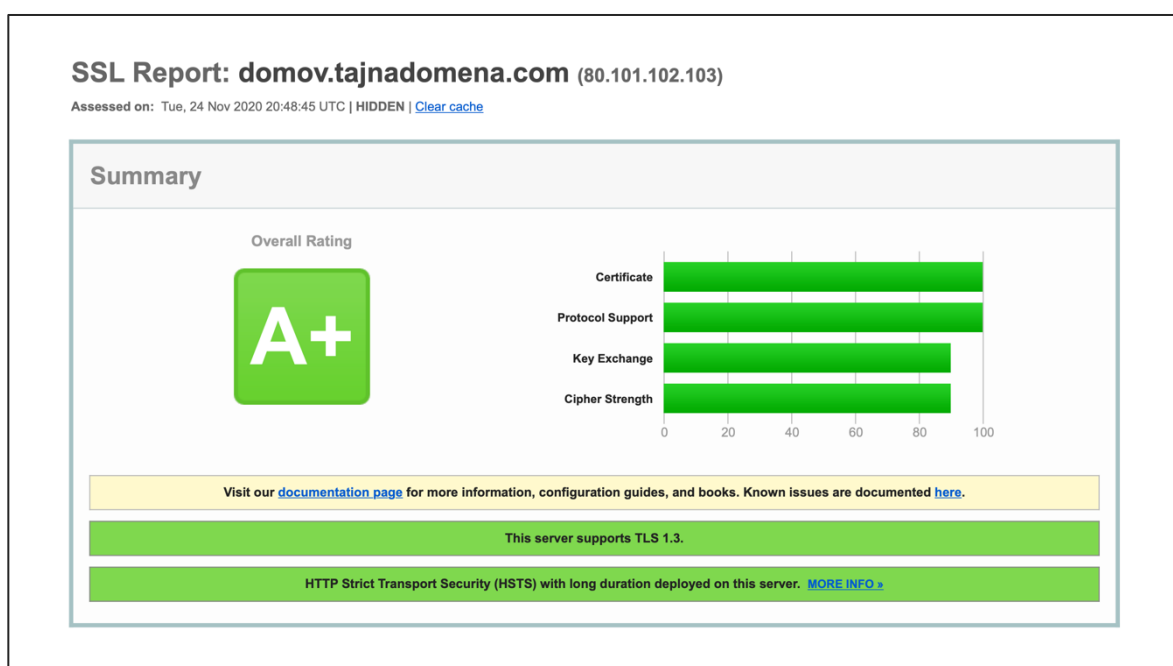
Obr. 35 Aktuální stav automatizačního scénáře v Home Assistant

7.1.10 Test zabezpečení vzdáleného přístupu

Kvalita implementace HTTPS

Kvalita implementace HTTPS je ověřena pomocí testovacího nástroje SSL Labs firmy Qualys. Očekávané hodnocení je A+, které splňuje dostatečnou obranu proti dnešním útokům, které mohou nastat na cestě mezi klientem a serverem. Součástí hodnocení je implementace bodů z návrhu řešení:

- Automatické přesměrování veškerých požadavků z HTTP na HTTPS
- Minimální povolené zabezpečení klienta – TLS verze 1.2
- Zapnutá funkce Strict Transport Security (HSTS)
- Povolená Forward Secrecy
- Serverem poskytovaný platný certifikát webu



Obr. 36 Test kvality implementace HTTPS

Všechny body byly dle výsledků splněny. Aktuální implementace HTTPS dosahuje stanovených požadavků.

Zabezpečení přihlášení v Home Assistant

OAuth přihlašovací formulář byl při implementaci dále omezen. Testovací scénář předpokládá:

- Zakázání přihlášení po 5 neúspěšných pokusech se špatně zadanou kombinací uživatelského účtu a hesla
- Zobrazení neúspěšných pokusů v notificačním centru Home Assistant včetně IP adresy přihlašovaného uživatele



Obr. 37 Test automatické blokace uživatelů s neúspěšnými pokusy o přihlášení v Home Assistant

Na obrázku výše lze vidět, že Home Assistant úspěšně zobrazil neúspěšný pokus o přihlášení v notificačním centru včetně správné externí IP adresy. Připojení na Home Assistant ze sítě Internet bylo testováno pomocí mobilního datového připojení. Po 5 neúspěšných pokusech se IP adresa uložila do seznamu nepovolených adres, Home Assistant vložil do notificačního centra nové oznámení, že došlo k blokaci dané adresy, a z dané adresy již nebylo možné pokračovat v přihlašování. Server odmítal další spojení zasláním HTTP hlavičky s chybou *403 – Přístup zakázán*. Testovací scénář byl splněn.

7.1.11 Test zabezpečení vzdálené správy

Implementovaný SSH server má povolit v domácí síti LAN autentizaci pomocí hesla a v případě připojení ze sítě Internet musí akceptovat pouze ověření pomocí privátního/veřejného klíče.

Výpis 7.5 Test připojení na SSH server

```
// Přihlášení ze sítě Internet pomocí hesla
michal@Michals-MBP ~ % ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
-p 666 user@domov.tajnadomena.com
user@domov.tajnadomena.com: Permission denied (publickey).

// Přihlášení ze domácí sítě LAN pomocí hesla
michal@Michals-MBP ~ % ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
user@10.0.0.1
user@10.0.0.1's password:

// Přihlášení pomocí nahraného klíče
michal@Michals-MBP ~ % ssh user@domov.tajnadomena.com -p 666
```

SSH server korektně odmítl nežádoucí pokus o připojení ze sítě Internet s použitím hesla a povolil připojení s heslem uvnitř sítě LAN. Testovací scénář proběhl úspěšně.

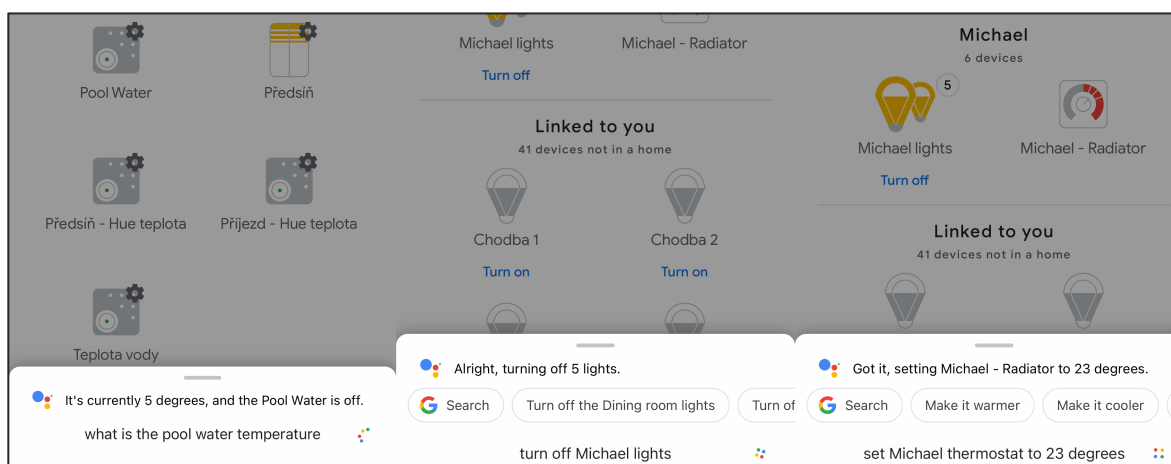
7.1.12 Test ovládání zařízení přes Hlasové asistenty

Testovací scénář očekává správné provedení vybraných hlasových příkazů:

- Dotaz na stav senzoru
- Změna teploty termostatu
- Rozsvícení světel

Google Assistant

Interakce s asistentem byla otestována pomocí aplikace Google Home a pomocí chytrých reproduktorů s integrovaným Google Assistant připojených ke Google účtu, na kterém byla zprovozněna integrace s Home Assistant.

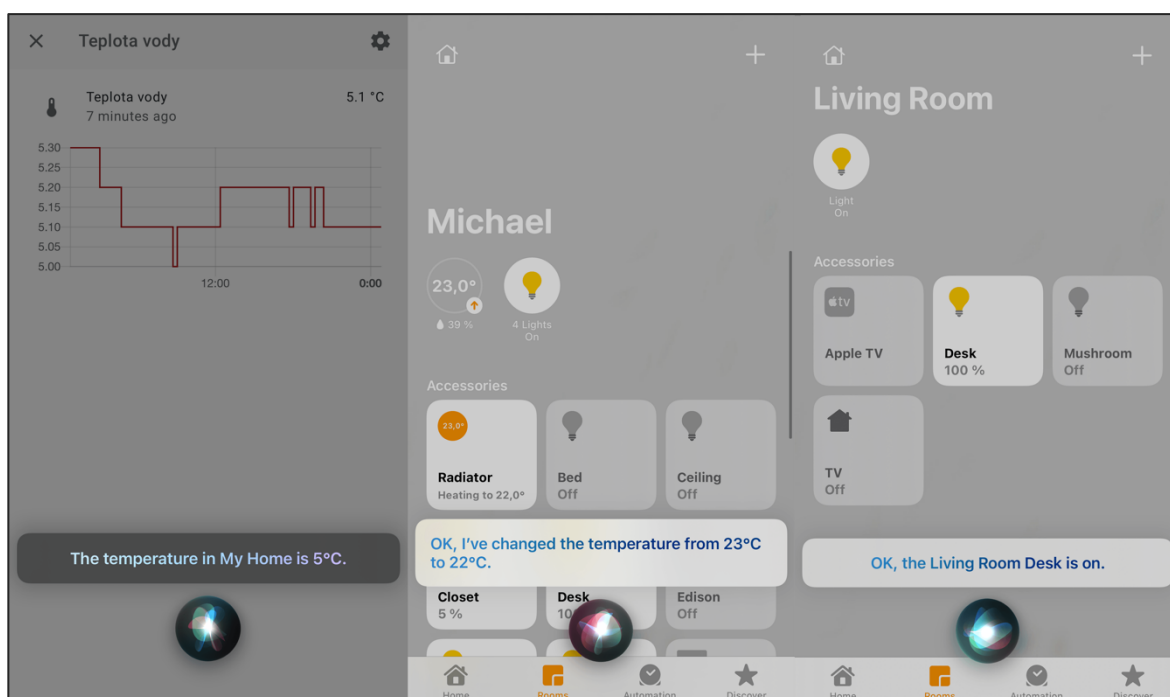


Obr. 38 Test hlasového rozhraní Google Assistant

Stav senzoru odpovídal hodnotě uvedené v Home Assistant. Došlo k zhasnutí světel v Home Assistant a posléze se na termostatu projevila změna teploty. Testovací scénář proběhl úspěšně.

Apple Siri

Interakce s asistentem Apple Siri probíhala přímo přes operační systém v zařízení iPhone. Do aplikace Apple Home, která je využívána asistentem jako platforma pro ovládání chytré domácnosti, byla integrována zařízení z Home Assistant (viz 4.5.7).



Obr. 39 Test hlasového asistenta Apple Siri

Všechny příkazy byly správně pochopeny, propsány do Home Assistant a poté vykonány. Testovací scénář byl dokončen bez chyb.

7.1.13 Test databáze časových řad

Správná funkčnost komponenty InfluxDB a Grafana je potvrzena v případě, zda bude v aplikaci Grafana úspěšně vykreslený graf ze zařízení, jejichž data jsou nastavena pro ukládání do databáze InfluxDB.



Obr. 40 Generování grafu teplotního čidla z databáze InfluxDB v Grafana

Přiložený obrázek výše ukazuje obrazovku v aplikaci Grafana generující graf teploty čidla, které je v Home Assistant nastavené pro ukládání (viz Výpis 6.43). Veškeré záznamy, které čidlo posílalo do Home Assistant, byly korektně zpracovány. Je zde vidět nepřetržitý provoz v řádu čtyř měsíců (mezery na začátku grafu jsou z důvodu neefektivního napět'ového regulátoru uvnitř sestaveného čidla, které ovlivnilo výdrž připojené baterie. Nejedná se o chybu v databázi InfluxDB). Testovací scénář proběhl úspěšně.

7.1.14 Test zálohování a obnovy

Pomocí výpisu systemd služby *backupsync.service* bylo zjištěno, zda došlo ve 3 hodiny ráno k úspěšné archivaci souborů a zálohy na Google Drive.

Výpis 7.6 Kontrola provedení automatické zálohy

```
user@SERVER: ~ $ systemctl status backupsync.service
Oct 21 03:02:49 SERVER rclone[18267]: Elapsed time:      2m0.0s
Oct 21 03:02:49 SERVER rclone[18267]: Transferring:
Oct 21 03:02:49 SERVER rclone[18267]: * spajzbackup.tar.bz2: 82% /155.040M, 1.078M/s, 25s
Oct 21 03:03:16 SERVER rclone[18267]: INFO : spajzbackup.tar.bz2: Copied (replaced existing)
Oct 21 03:03:16 SERVER rclone[18267]: INFO :
Oct 21 03:03:16 SERVER rclone[18267]: Transferred: 155.040M / 155.040 MBytes, 100%, 1.057
MBytes/s, ETA 0s
Oct 21 03:03:16 SERVER rclone[18267]: Transferred:      1 / 1, 100%
Oct 21 03:03:16 SERVER rclone[18267]: Elapsed time:      2m27.8s
Oct 21 03:03:16 SERVER systemd[1]: backupsync.service: Succeeded.
Oct 21 03:03:16 SERVER systemd[1]: Finished Synchronizace na GDrive.
```

Dle výpisu výše byl soubor úspěšně nahrán do cloudového úložiště.

Po stažení souboru z Google Drive úložiště byl pomocí archivační utility *tar* archiv znovu rozbalen. Soubory se shodovaly s těmi, které byly definovány pro zálohu v implementaci (viz 6.5). Testovací scénář byl úspěšně dokončen.

```
user@SERVER: ~ $ sudo tar -xvjf smarthomebackup.tar.bz2
```

7.2 Testovací report

Tabulka 3 Přehled výstupu testovacích scénářů

Číslo scénáře	Testovací scénář	Úspěšně dokončen
1	Síťová konfigurace	Ano
2	Využití HW prostředků	Ano
3	MQTT server	Ano
4	Zigbee zařízení	Ano
5	Z-Wave zařízení	Ano
6	HomeKit zařízení	Ano
7	RF zařízení	Ano
8	Uživatelská samospráva	Ano
9	Automatizační scénáře	Ano
10.1	Vzdálený přístup – Implementace HTTPS	Ano
10.2	Vzdálený přístup – Zabezpečení Home Assistant	Ano
11	Zabezpečení vzdálené správy	Ano
12.1	Hlasový asistent – Google Assistant	Ano
12.2	Hlasový asistent – Apple Siri	Ano
13	Databáze časových řad	Ano

Všechny testovací scénáře prošly. Vzhledem k první iteraci softwaru nejsou zaznamenány žádné regrese. Implementace funguje korektně a první iterace centrální jednotky chytré domácnosti může být nasazena do produkčního prostředí.

Závěr

V práci jsem si vytyčil cíl dosáhnout integrace chytrých zařízení napříč různými protokoly.

V teoretické části jsem se podíval, jaký je aktuální stav trhu s výrobky chytré domácnosti a které protokoly se používají. Rozebral jsem komerční řešení, která aktuálně používají výrobci. Zjistil jsem, že ačkoliv mnoho zařízení používá společné protokoly, výrobci nutí zákazníky své centrální jednotky, které mají silně omezené integrace s ekosystémy jiných výrobců. I univerzální centrální jednotky, které se tento problém snaží řešit, trpí zásadní chybou v konceptu těchto zařízení. Software vlastní výrobce, a zákazník má pouze licenci na jeho užívání. Ze samotné podstaty neustále se vyvíjejícího trhu chytré domácnosti je nutnost software udržovat. To nutí výrobce těchto zařízení přecházet z fixní ceny na předplatitelské modely, které nebyly původně zamýšleny ani komunikovány zákazníkům. Zákazník je pak nucen na takový model přistoupit, jinak se zařízení stane nefunkčním, anebo projít náročnou migrací na jiné řešení. Spousta centrálních jednotek také vyžaduje neustálé připojení k internetu, což dále omezuje samostatnost uživatele; navíc při vypnutí či výpadcích serverů, které se již několikrát staly, je uživatel ponechán s omezeně funkčním řešením.

Zcela lokální centrální jednotka postavená na volně dostupném softwaru řeší tyto největší nedostatky aktuálního stavu trhu. Pro vývoj řešení jsem zvolil plán v podobě životního cyklu softwaru, kde je definovaný postupný sled činností, které se v průběhu postupně aplikují. Vybraný evoluční model životního cyklu mi dovolil uvažovat nad tvorbou řešení tak, aby bylo do budoucna připravené na rozšíření nových technologií a protokolů, které dnes ještě nelze definovat. Nejdříve jsem specifikoval požadavky na centrální jednotku z pohledu architekta systému a z pohledu konečného uživatele. Tyto dva pohledy zajistily, aby nebylo myšleno jen na technickou stránku řešení, ale i na přínos a uživatelskou zkušenost uživatele. V části analýzy jsem podrobil požadavky hlubšímu zkoumání: Jak fungují jednotlivé protokoly zařízení chytré domácnosti a jaké jsou jejich volně dostupné implementace, jakým způsobem lze přistoupit k provozu jednotlivých softwarových aplikací v operačním systému nebo jakou variantu operačního systému zvolit.

V praktické části jsem připravil návrh řešení. Vybral jsem jednotlivé typy softwarových projektů, přístup k řešení operačního systému a provozu softwaru. Návrh jsem i zakreslil do komponentového pohledu a pohledu datových toků mezi jednotlivými softwarovými aplikacemi. V implementační části jsem popsal dokumentaci stavby celého prostředí od instalace operačního systému až po konfiguraci každé komponenty. Implementované řešení jsem nakonec podrobil připraveným testovacím scénářům, které ověřily připravenost řešení pro přesun do produkčního prostředí.

Výsledkem je nasazené řešení, které je plně lokální v místní síti, nezávislé na internetovém připojení, vůli a plánech výrobce komerčního řešení. Dává uživateli plně do rukou vlastnictví veškerého softwarového vybavení. Lokální zpracování příkazů umožňuje okamžitou reakci na příkazy uživatele. Nezávislost na uzavřeném řešení, ale použití svobodného softwaru umožňuje flexibilní rozšíření a pozvolnou evoluci celého systému. Veškerá data jsou uživateli volně přístupná k nahlédnutí, pouze v jeho vlastnictví a on rozhoduje, jak s daty naloží.

Nepřestává mě však fascinovat krása technologií, která se skrývá v možnosti vytvářet neustále více komplexní a propracované řešení, které posouvá pomyslnou laťku technologického vývoje ještě výše, a přináší větší užitek pro každodenní život běžných uživatelů. Umělá inteligence v podobě strojového učení implementovaná v kontextu právě vytvořené chytré domácnosti, ve které lze komunikovat mezi všemi zařízeními, skrývá obrovský potenciál. Schopnost naučit se vzorce chování uživatele při ručním ovládání domácnosti a poté na jejich základě vytvořit automatické automatizační scénáře by mohlo chytrou domácnost skutečně „oživit“. Nebo uplatnění metodik provozu softwaru z datacenter v podobě tzv. vysoké dostupnosti (High Availability), kdy software není vázán na jeden počítač, ale dokáže se přesunout na jiný počítačový uzel v případě katastrofického selhání hardwaru počítače původního. Uvedené podněty otevírají zajímavé možnosti k dalšímu výzkumu problematiky chytré domácnosti a mohou volně navázat na výsledek této práce.

Použitá literatura

- [1] *Smart Home* | Definition of Smart Home by Oxford Dictionary on Lexico.com [online]. 2020 [vid. 2020-12-08]. Dostupné z: https://www.lexico.com/definition/smart_home
- [2] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION AND THE INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 20924:2018. *ISO* [online]. prosinec 2018 [vid. 2020-04-05]. Dostupné z: <https://www.iso.org/standard/69470.html>
- [3] VINCENT BONNEAU, BERTRAND COPIGNEAUX, IDATE, LAURENT PROBST, BERTRAND PEDERSEN a PWC. *Smart Home: Technologies with a standard battle* [online]. B.m.: European Commission. 2017 [vid. 2020-04-05]. Dostupné z: https://ec.europa.eu/growth/tools-databases/dem/monitor/sites/default/files/DTM_Smart%20Home%20-%20Standard%20battle%20v1.pdf
- [4] OECD. *Consumer policy and the smart home* [online]. OECD Digital Economy Papers. 268. 2018 [vid. 2020-10-30]. *OECD Digital Economy Papers*. Dostupné z: doi:10.1787/e124c34a-en
- [5] INTERNET SOCIETY. *Policy Brief: The Internet of Things* [online]. 2016 [vid. 2020-11-01]. Dostupné z: <https://www.internetsociety.org/wp-content/uploads/2017/09/ISOC-PolicyBrief-IoT.pdf>
- [6] INTERNET SOCIETY. *The Internet of Things (IoT): An Overview* [online]. 2015 [vid. 2020-11-01]. Dostupné z: <https://www.internetsociety.org/wp-content/uploads/2017/08/ISOC-IoT-Overview-20151221-en.pdf>
- [7] Z-WAVE ALLIANCE. Silicon Labs and Z-Wave Alliance Expand Smart Home Ecosystem by Opening Z-Wave to Silicon and Stack Suppliers. *Z-Wave Alliance* [online]. 18. prosinec 2019 [vid. 2020-10-21]. Dostupné z: <https://z-wavealliance.org/z-wave-specification-press-release/>
- [8] JENNIFER PATTISON TUOHY. Samsung SmartThings: Use the app, features and hubs for a better smart home [obrázek]. *The Ambient* [online]. 7. říjen 2020 [vid. 2020-11-02]. Dostupné z: <https://www.the-ambient.com/guides/samsung-smarthings-guide-smart-home-163>
- [9] WINK LABS INC. *Introducing Wink Subscription — Wink Blog* [online]. 6. květen 2020 [vid. 2020-11-02]. Dostupné z: <https://web.archive.org/web/20200506203904/https://blog.wink.com/wink-blog/2020/5/6/introducing-wink-subscription>
- [10] REVOLV A NEST COMPANY. *Revolv* [online]. 13. března 2016 [vid. 2020-11-02]. Dostupné z: <https://web.archive.org/web/20160313115847/http://revolv.com/>
- [11] MICHELE TURNER. *We hear you: updates to Works with Nest* [online]. 14. srpen 2019 [vid. 2020-11-02]. Dostupné z: <https://web.archive.org/web/20190814024115/https://blog.google/products/google-nest/updates-works-with-nest/>
- [12] LINDEN TIBBETS. *Introducing IFTTT Pro* [online]. [vid. 2020-11-02]. Dostupné

z: https://ifttt.com/explore/introducing_ifttt_pro

[13] AARON SWIFT. Platform Outage Update. *SmartThings Blog* [online].

14. březen 2018 [vid. 2020-11-02]. Dostupné

z: <https://blog.smartthings.com/news/smartthings-updates/platform-outage-update/>

[14] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION AND THE INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC/IEEE

12207:2017. *ISO* [online]. listopad 2017 [vid. 2020-11-06]. Dostupné

z: <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/06/37/63712.html>

[15] BRUCKNER, Tomáš, VOŘÍŠEK, JIŘÍ a BUCHALCEVOVÁ, ALENA. *Tvorba informačních systémů: principy, metodiky, architektury*. Praha: Grada, 2012. ISBN 978-80-247-4153-6.

[16] PROF. ING. JIŘÍ VOŘÍŠEK, CSC. Tvorba a provoz informačních systémů.

In: [online]. B.m. 19. únor 2016 [vid. 2020-11-07]. Dostupné

z: https://nb.vse.cz/~vorisek/FILES/4IT215_materialy_k_predmetu/05MMDIS-Princip_multidimezionality_a_dimenze_reseni_IS.zip

[17] GOOGLE. *Google Github* [online]. 2020 [vid. 2020-11-05]. Dostupné

z: <https://github.com/google>

[18] MICROSOFT. *Microsoft Github* [online]. 2020 [vid. 2020-11-05]. Dostupné

z: <https://github.com/microsoft>

[19] *The Linux Kernel Archives - FAQ* [online]. [vid. 2020-10-22]. Dostupné

z: <https://www.kernel.org/category/faq.html>

[20] THE LINUX FOUNDATION. *2020 Linux Kernel History Report* [online]. 2020

[vid. 2020-10-22]. Dostupné z: <https://www.linuxfoundation.org/blog/2020/08/download-the-2020-linux-kernel-history-report/>

[21] LI, Zheng (Eddie), Maria KIHIL, Qinghua LU a Jens ANDERSSON. Performance Overhead Comparison between Hypervisor and Container Based Virtualization.

In: [online]. 2017, s. 955–962. Dostupné z: doi:10.1109/AINA.2017.79

[22] GARTNER. Forecast Analysis: Container Management (Software and Services), Worldwide. *Gartner* [online]. 25. červen 2020 [vid. 2020-11-08]. Dostupné

z: <https://www.gartner.com/en/newsroom/press-releases/2020-06-25-gartner-forecasts-strong-revenue-growth-for-global-co>

[23] IAN LEWIS. Container Runtimes Part 1: An Introduction to Container Runtimes - Ian Lewis. *Ian Lewis* [online]. 6. prosinec 2017 [vid. 2020-11-14]. Dostupné

z: <https://www.ianlewis.org/en/container-runtimes-part-1-introduction-container-r>

[24] THE LINUX FOUNDATION. *Open Container Initiative* [online]. 2020 [vid. 2020-11-14]. Dostupné z: <https://opencontainers.org/>

[25] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION AND THE INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 7498-1:1994. *ISO* [online]. listopad 1994 [vid. 2020-04-05]. Dostupné

z: <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/02/02/20269.html>

[26] MUSA UNMEHOPA a SHASHANK GOEL. *ZigBee interoperability* [online].

B.m.: ZigBee Alliance. 2017 [vid. 2020-04-06]. Dostupné

z: https://zigbeealliance.org/developer_resources/zigbee-interoperability/

- [27] ZIGBEE ALLIANCE. Zigbee Certified Products Archives. *Zigbee Alliance* [online]. 2020 [vid. 2020-04-06]. Dostupné z: https://zigbeealliance.org/product_type/certified_product/
- [28] INTERNATIONAL TELECOMMUNICATION UNION. *ARTICLE 1 - Terms and Definitions* [online]. 19. říjen 2009 [vid. 2020-04-05]. Dostupné z: <http://life.itu.int/radioclub/rr/art1.pdf>
- [29] JOEL CRANE. ZigBee and WiFi Coexistence — MetaGeek. *MetaGeek* [online]. 2018 [vid. 2020-04-06]. Dostupné z: <https://support.metageek.com/hc/en-us/articles/203845040-ZigBee-and-WiFi-Coexistence>
- [30] KANTERS, Koen. *Koenkk Github* [online]. JavaScript. 2020 [vid. 2020-10-21]. Dostupné z: <https://github.com/Koenkk/>
- [31] OPENZWAVE. *OpenZWave* [online]. 2020 [vid. 2020-11-08]. Dostupné z: <https://github.com/OpenZWave>
- [32] FRANK ZIRONE. *RFLink Gateway* [online]. 2020 [vid. 2020-11-26]. Dostupné z: <http://www.rflink.nl/>
- [33] RFXCOM. *RFXCOM* [online]. 2020 [vid. 2020-11-26]. Dostupné z: http://www.rfxcom.com/en_GB
- [34] FLORIAN. *ltechnophile/OpenMQTTGateway* [online]. C++. 2020 [vid. 2020-11-26]. Dostupné z: <https://github.com/ltechnophile/OpenMQTTGateway>
- [35] ARENDS, Theo. *arendst/Tasmota* [online]. C. 2020 [vid. 2020-11-26]. Dostupné z: <https://github.com/arendst/Tasmota>
- [36] PÉREZ, Xose. *xoseperez/espurna* [online]. C++. 2020 [vid. 2020-11-26]. Dostupné z: <https://github.com/xoseperez/espurna>
- [37] APPLE. HomeKit Accessory Protocol Specification (Non-Commercial Version). *Support - Apple Developer* [online]. [vid. 2020-11-07]. Dostupné z: <https://developer.apple.com/support/homekit-accessory-protocol/>
- [38] *HomeKit Accessory Development Kit* [online]. C. B.m.: Apple, 2020 [vid. 2020-11-08]. Dostupné z: <https://github.com/apple/HomeKitADK>
- [39] JC2K. *Jc2k/aiohomekit* [online]. Python. 2020 [vid. 2020-11-09]. Dostupné z: <https://github.com/Jc2k/aiohomekit>
- [40] LUSIARDI, Joachim. *jlusiardi/homekit_python* [online]. Python. 2020 [vid. 2020-11-09]. Dostupné z: https://github.com/jlusiardi/homekit_python
- [41] STEGEMAN, Michael. *mrstegeman/hap-controller-node* [online]. JavaScript. 2020 [vid. 2020-11-09]. Dostupné z: <https://github.com/mrstegeman/hap-controller-node>
- [42] GOOGLE. Overview | Actions on Google Smart Home. *Google Developers* [online]. 2020 [vid. 2020-11-08]. Dostupné z: <https://developers.google.com/assistant/smarthome/overview>
- [43] APPLE. Set up your HomePod, Apple TV, or iPad as a home hub. *Apple Support* [online]. 2020 [vid. 2020-11-08]. Dostupné z: <https://support.apple.com/en-us/HT207057>
- [44] OPENJS FOUNDATION. *node-red/node-red* [online]. 2020 [vid. 2020-11-09]. Dostupné z: <https://github.com/node-red/node-red>
- [45] *domoticz/domoticz* [online]. C++. B.m.: Domoticz, 2020 [vid. 2020-11-09]. Dostupné z: <https://github.com/domoticz/domoticz>

- [46] CZ.NIC, Z. S. P. O. *Domoticz screenshot [Project: Turris | Gadgets]* [online]. 10. srpen 2015 [vid. 2020-11-09]. Dostupné z: https://doc.turris.cz/gadgets/_detail/domo1.png?id=domoticz
- [47] *home-assistant/core* [online]. Python. B.m.: Home Assistant, 2020 [vid. 2020-11-09]. Dostupné z: <https://github.com/home-assistant/core>
- [48] FADHEL, Muntazir, Emil SEKERINSKI a Shucai YAO. A Comparison of Time Series Databases for Storing Water Quality Data. In: [online]. 2019, s. 302–313. ISBN 978-3-319-89796-7. Dostupné z: doi:10.1007/978-3-030-11434-3_33
- [49] GRAFANA LABS. Data sources. *Grafana Labs* [online]. 2020 [vid. 2020-11-13]. Dostupné z: <https://grafana.com/docs/grafana/latest/datasources/>
- [50] FABISIAK, Radoslaw. What is a reverse proxy and how it works? *Medium* [online]. 26. červenec 2020 [vid. 2020-11-15]. Dostupné z: <https://medium.com/duomly-blockchain-online-courses/what-is-a-reverse-proxy-and-how-it-works-f20d00533df2>
- [51] VIXIE, P., S. THOMSON, Y. REKHTER a J. BOUND. *Dynamic Updates in the Domain Name System (DNS UPDATE)* [online]. RFC2136. B.m.: RFC Editor. 1997 [vid. 2020-11-15]. Dostupné z: doi:10.17487/rfc2136
- [52] ELVIS PLESKY. NGINX vs Apache – Which Is the Best Web Server in 2020? *Plesk* [online]. 7. srpen 2020 [vid. 2020-11-13]. Dostupné z: <https://www.plesk.com/blog/various/nginx-vs-apache-which-is-the-best-web-server/>
- [53] INTERNET SECURITY RESEARCH GROUP (ISRG). *Let's Encrypt - Free SSL/TLS Certificates* [online]. [vid. 2020-11-12]. Dostupné z: <https://letsencrypt.org/>
- [54] JUDD VINET a AARON GRIFFIN. *Arch Linux* [online]. 2002 [vid. 2020-10-24]. Dostupné z: <https://www.archlinux.org/>
- [55] Arch Linux Wiki - Main Page. *Arch Linux Wiki* [online]. 10. říjen 2020 [vid. 2020-10-24]. Dostupné z: <https://wiki.archlinux.org/>
- [56] Installation guide - ArchWiki. *Arch Linux Wiki* [online]. 28. září 2020 [vid. 2020-10-25]. Dostupné z: https://wiki.archlinux.org/index.php/installation_guide
- [57] MAX FERRER a ERAZEM KOKOT. *Anarchy Installer* [online]. 2020 [vid. 2020-10-25]. Dostupné z: <https://anarchyinstaller.org/>
- [58] AEOTEC. *Z-Stick Gen5+* [online]. 2020 [vid. 2020-11-14]. Dostupné z: <https://aeotec.com/z-wave-usb-stick/>
- [59] HOME ASSISTANT. Integrations. *Home Assistant* [online]. 2020 [vid. 2020-11-15]. Dostupné z: <https://www.home-assistant.io/integrations/>
- [60] INTEL CORPORATION. *USB 3.0* Radio Frequency Interference Impact on 2.4GHz Wireless Devices - White Paper* [online]. 2012 [vid. 2020-11-22]. Dostupné z: <https://www.usb.org/sites/default/files/327216.pdf>
- [61] SILICON LABORATORIES. *CP210x USB to UART Bridge VCP Drivers* [online]. 2020 [vid. 2020-11-26]. Dostupné z: <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>
- [62] GOOGLE. Cloud Console - Web UI Admin. *Google Cloud* [online]. 2020 [vid. 2020-11-21]. Dostupné z: <https://cloud.google.com/cloud-console>
- [63] HOME ASSISTANT. Google Assistant integration. *Home Assistant* [online]. 2020 [vid. 2020-11-22]. Dostupné z: https://www.home-assistant.io/integrations/google_assistant/

- [64] NETDATA, INC. Netdata - Monitor everything in real time for free with Netdata. *Netdata* [online]. 2020 [vid. 2020-11-29]. Dostupné z: <https://www.netdata.cloud/>
- [65] TORSTEN HOFFMANN. *SunCalc sun position- und sun phases calculator* [online]. 2020 [vid. 2020-12-01]. Dostupné z: <https://www.suncalc.org>

Přílohy

Příloha A: Postup instalátoru distribuce Arch Linux

Zde jsou popsány jednotlivé možnosti zvolené pro nainstalování minimální linuxové distribuce Arch Linux s instalátorem Anarchy Installer.

Výpis 7.7 Postup instalace operačního systému Arch Linux instalátorem Anarchy Installer

```
-- Výběr lokálních preferencí --
Výběr jazyka instalačního průvodce - English
Znaková sada - us (United States)
Seznam serverů pro aktualizace - Download & Rank New Mirrors
Jazyk systémového prostředí - en_US.UTF-8
Časová zóna - Europe/Prague

-- Příprava diskových oddílů --
Metoda rozdělení disku - Auto Partition Drive
Výběr disku pro instalaci - (sda, sdb,..)
Výběr souborového systému - ext4
Vytvoření SWAP oddílu - Polovina RAM (4096M)
Zapnutí UEFI módu - Ano
Potvrzení změn

-- Instalace základních balíčků softwaru -
Možnost instalace - Anarchy-Advanced (ruční výběr softwaru)
Typ základní skupiny balíčků - Arch-Linux-Base
Typ příkazového shellu - bash
Typ zavaděče systému - systemd-boot (již v systému, není třeba další instalace)
Dodatečná síťová utilita - Žádná (server je připojen Ethernetem, použijeme systemd)
Povolit 32-bitový softwarový repozitář - Ne
Povolit DHCP - Ne
Nainstalovat Wi-Fi utility - Ne
Nainstalovat PPoE utility - Ne
Nainstalovat pomocník pro dualboot - Ne
Nainstalovat desktopové grafické prostředí - Ne
Název počítače - SERVER
Potvrzení, že chceme pouze příkazový řádek - Ano

-- Nastavení uživatelů --
Heslo pro root účet - *****
```



```
Menu pro přidání dalšího uživatele - New User
Název uživatele - user
Jméno uživatele - Main User
Heslo pro uživatele user - *****
Povolit sudo oprávnění pro uživatele user - Ano
Menu pro přidání dalšího uživatele - Done
```

```
-- Zahájení instalace balíčků --
Chcete nainstalovat další software - Ne
Potvrzení zahájení instalace balíčků - Ano

-- Dokončení instalace -
Restartovat systém - Reboot Systém
```

Příloha B: Alternativní připojení Docker kontejnerů do sítě

V některých situacích může aplikace v kontejneru vyžadovat přímý přístup do fyzické vnější sítě. Macvlan umožňuje přiřadit kontejneru svoji vlastní IP adresu z vnější sítě. Pro tento režim je vyžadována síťová karta, která se může přepnout do tzv. promiskuitního módu, kde může být jedné kartě přiřazeno více MAC adres. MAC adresa je unikátní identifikační číslo každého síťového prvku. V této metodě není třeba u kontejneru definovat, jaké porty může využívat, protože je připojen přímo do vnější sítě. K zprovoznění stačí přes *docker network* vytvořit novou síť typu *macvlan*. Její nastavení bude stejné jako rozsah IP adres v domácí LAN síti, do které je připojen počítač, na které běží Docker.

Výpis 7.8 Vytvoření Docker sítě umožňující přiřazení IP adres domácí sítě kontejneru

```
[user@SERVER]: ~>$ docker network create -d macvlan -o parent=enp1s0\
--subnet 10.0.0.0/16 \
--gateway 10.0.0.10 \
--ip-range 10.0.0.16/28 \
Fyzicka
```

Výhodou této metody je tedy vlastní IP adresa pro kontejner. Hodí se pro aplikaci, která potřebuje k běhu mnoho portů. Tato metoda má i své nevýhody. Komunikace s kontejnery, které zase používají metodu *bridge* není možná napřímo. Lze komunikovat jen přes porty, které byly publikovány do vnější sítě, za předpokladu, že je metoda Macvlan implementována i v nastavení síťové konfigurace operačního systému. Jelikož žádná aplikace, z těch, které jsou implementovány v této práci nepotřebuje tuto metodu připojení, nebude využita v návrhu řešení.

Výpis 7.9 Připojení operačního systému do sítě se systemd-networkd pomocí metody macvlan

```
/etc/systemd/network/mac0.netdev
[NetDev]
```

```

Name=mac0
Kind=macvlan

[MACVLAN]
Mode=bridge

/etc/systemd/network/enp1s0.network
[Match]
Name=enp1s0

[Network]
MACVLAN=mac0

/etc/systemd/network/mac0.network
[Match]
Name=mac0

[Network]
IPForward=yes
Address=10.0.0.1/16
Gateway=10.0.0.10

```

Příloha C: Kontejnerový obraz pro firmware flash Zigbee rádia

V následujícím výpisu je zobrazena struktura nově vytvořených souborů, které slouží k vytvoření vlastního kontejneru pro flash Zigbee rádia.

Výpis 7.10 Souborová struktura pro vytvoření kontejneru na firmware flash Zigbee rádia

```

Složka /home/user/docker/

|-- builds
    |-- cc2538-bsl
        |-- build.sh
        |-- Dockerfile
        |-- firmware-update.sh

```

Nejdříve byl vytvořen soubor Dockerfile, který obsahuje definici Docker kontejnerového obrazu.

- Staví na kontejnerovém obrazu Python 3
- Do kontejneru se nainstalují závislosti pro běh aplikace a aplikace samotná
- Stáhnou se aktuální firmwary pro flashování od tvůrců Zigbee2MQTT softwaru.
- Jako aplikace, která se má spustit při startu kontejneru je nastaven skript *firmware-update.sh*, který provede uživatele bezpečným postupem flashování.

Výpis 7.11 Definice kontejnerového obrazu pomocí Dockerfile

Soubor /docker/builds/cc2538-bsl/Dockerfile

```
FROM python:3-alpine

RUN apk add py3-setuptools git unzip bash
RUN pip3 install intelhex git+https://github.com/JelmerT/cc2538-bsl
COPY firmware-update.sh /
WORKDIR /firmwares
RUN git clone https://github.com/Koenkk/Z-Stack-firmware.git
RUN find /firmwares/Z-Stack-firmware/coordinator/ -name "*.zip" | while read filename; do
unzip -o "$filename"; done;

ENTRYPOINT ["/firmware-update.sh"]
```

Skript `firmware-update.sh` se bezpečně dotáže na vybrání správného firmware souboru pro flashnutí v závislosti na typu používaného USB Zigbee rádia. A poté spustí flashovací nástroj s předpokládaným USB zařízením na místě `/dev/zigbeestick`. Skript byl pomocí `chmod +x` označen jako spustitelný.

Výpis 7.12 Skript pro flashnutí Zigbee rádia umístěný do kontejneru

Soubor /home/user/docker/builds/cc2538-bsl/firmware-update.sh

```
#!/bin/bash

FLASHER=cc2538-bsl.py
FW_PATH=/firmwares/
DEVICE_NAME=/dev/zigbeestick

echo "-----"
echo " "
echo "      Zigbee Texas Instruments radio flashovací nástroj"
echo " "
echo "-----"
echo " "
echo " "
echo "Dostupné firmwary:"
ls -l "$FW_PATH"

echo " "
echo "Opište název souboru včetně přípony"
echo "nebo stiskněte Enter pro ukončení."
echo " "
```

```

read -p "Název souboru: " fileName
echo " "
if [[ -z "${fileName// }" ]]; then
    echo "Ukončuji..."
    exit 1
fi
filePath="${FW_PATH%}/$fileName"

echo "-----"
echo " "
echo "Zařízení: $DEVICE_NAME"
echo " "
echo "Firmware: $fileName"
echo " "
echo "Jsou zařízení a firmware soubor správné?"
read -p "Stiskněte Y pro pokračování, cokoliv jiného pro ukončení: " correctVal
echo " "

if [[ $correctVal == [yY] ]]; then
    echo "Probíhá flashování..."
    echo " "
    $FLASHER -p $DEVICE_NAME -evw "$filePath"

    retVal=$?
    if (( retVal != 0 )); then
        echo " "
        echo "Chyba při flashování! Prosím zkuste znovu..."
        echo " "
        exit $retVal
    fi
else
    echo "Ukončuji..."
    echo " "
    exit 1
fi

```

Nakonec byl vytvořen krátký skript, který jen zavolá příkaz *docker build*, jenž automaticky zpracuje soubor Dockerfile a podle něho vytvoří nový kontejnerový obraz.

Výpis 7.13 Skript pro zahájení vytvoření nového kontejnerového obrazu

```

Soubor /home/user/docker/builds/cc2538-bsl/build.sh

#!/bin/bash

docker build . -t cc2538-bsl

```

- `-t` umožňuje definovat název výsledného obraz

Následující výpis ukazuje úspěšný výstup flashovacího skriptu ze spuštěného kontejneru.

Výpis 7.14 Ukázkový výpis aplikace úspěšného nahrání Zigbee firmwaru

```
user@SERVER: ~ $ /home/user/docker/servicescripts/zigbeeupdate.sh
```

Dostupné firmwary:

CC1352P_2_20201026.hex

CC2530ZNP-Prod.hex

CC2652RB_20201026.hex

Opište název souboru včetně přípony
nebo stiskněte Enter pro ukončení.

Název souboru: CC2652RB_20201026.hex

Zařízení: /dev/zigbeestick

Firmware: CC2652RB_20201026.hex

Jsou zařízení a firmware soubor správné?

Stiskněte Y pro pokračování, cokoliv jiného pro ukončení: y

Probíhá flashování...

Opening port /dev/zigbeestick, baud 500000

Reading data from /firmwares/CC2652RB_20201026.hex

Your firmware looks like an Intel Hex file

Connecting to target...

...

Erasing all main bank flash sectors

Erase done

Writing 360448 bytes starting at address 0x00000000

Write 104 bytes at 0x00057F980

Write done

Verifying by comparing CRC32 calculations.

Verified (match: 0x7e76e932)

Příloha D: Servisní skript pro aktualizaci kontejneru

Zde se nachází zdrojový kód skriptu, který interaktivně aktualizuje běžící kontejner.

Výpis 7.15 Servisní skript pro aktualizaci kontejneru

Soubor /home/user/docker/servicescripts/updatecontainer.sh

```
#!/bin/bash

echo "Tyto kontejnery aktuálně běží:"
echo ""
docker ps --format "{{.Names}}"
echo ""
echo "Jaký kontejner chcete aktualizovat?"
read -p "Název kontejneru: " containerName
echo "Zastavuji kontejner..."
docker stop $containerName
echo "Odebíram kontejner..."
docker rm $containerName
echo "Stavím nový kontejner..."
/home/user/docker/scripts/$containerName.sh
echo "Odebíram staré kontejnerové obrazy..."
docker image prune --force
```